Chapter 3 Numpy and Pandas

```
import numpy as np
np.random.seed(10)
```

TBD: split the sections better

Base python does not include true vectorized data structures–vectors, matrices, and data frames. For small things one can use lists, lists of lists, and list comprehensions. However, such code will be bulky and slow.

This deficiency is addressed by additional libraries, in particular *numpy* and *pandas*. Numpy is the primary way to handle matrices and vectors in python. This is the way to model either a variable or a whole dataset so vector/matrix approach is very important when working with datasets. Even more, these objects also model the vectors/matrices as mathematical objects. Matrix computations are extremely important in statistics and hence also in machine learning.

3.1 Numpy

Numpy is the most popular python library for matrix/vector computations. Due to python's popularity, it is also one of the leading libraries for numerical analysis, and a frequent target for computing benchmarks and optimization.

Numpy logo. Isabela Presedo-Floyd, CC BY-SA 4.0, via Wikimedia Commons.

It is important to keep in mind that *numpy* is a separate library that is not part of the base python. Unlike R, base python is not vectorized, and one has to load *numpy* (or another vectorized library, such as *pandas*) in order to use vectorized operations. This also causes certain differences between the base python approach and the way to do vectorized operations.

3.1.1 Importing numpy

Numpy is typically imported as `np`:

```
import numpy as np
```

`np` is pretty much the standard acronym for the numpy and widely used in online documentation. Below we assume numpy has been imported as `np`.

3.1.2 Array: The Fundamental Data Structure in Numpy

Numpy is fundamentally based on arrays, N-dimensional data structures. Here we mainly stay with one- and two-dimensional structures (vectors and matrices) but the arrays can also have higher dimension (called *tensors*). Besides arrays, numpy also provides a plethora of functions that operate on the arrays, including vectorized mathematics and logical operations.

Arrays can be created with `np.array`. For instance, we can create a 1-D vector of numbers from 1 to 4 by feeding a list of desired numbers to the `np.array`:

```
a = np.array([1,2,3,4])
print("a:\n", a)
```

```
## a:
##  [1 2 3 4]
```

Note that it is printed in brackets as list, but unlike a list, it does not have commas separating the components.

If we want to create a matrix (two-dimensional array), we can feed `np.array` with a list of lists, one sublist for each row of the matrix:

```
b = np.array([[1,2], [3,4]])
print("b:\n", b)
```

```
## b:
##  [[1 2]
##  [3 4]]
```

The output does not have the best formatting but it is clear enough.

One of the fundamental property of arrays its dimension, called *shape* in numpy. Shape is array's size along all of its dimensions. This can be queried by attribute `.shape` which returns the sizes in a form of a tuple:

```
a.shape
```

```
## (4,)
```

```
b.shape
```

```
## (2, 2)
```

One can see that vector `a` has a single dimension of size 4, and matrix `b` has two dimensions, both of size 2 (remember: `(4,)` is a tuple of length 1!).

One can also *reshape* arrays, i.e. change their shape into another compatible shape. This can be achieved with `.reshape()` method. `.reshape` takes one argument, the new shape (as a tuple) of the array. For instance, we can reshape the length-4 vector into a 2x2 matrix as

```
a.reshape((2,2))
```

```
## array([[1, 2],
##        [3, 4]])
```

and we can "straighten" matrix `b` into a vector with

```
b.reshape((4,))
```

```
## array([1, 2, 3, 4])
```

3.1.3 Creating Arrays

Sometimes it is practical to create arrays manually as we did above, but usually it is much more important to make those by computation. Below we list a few options.

`np.arange` creates sequences, quite a bit like `range`, but the result will be a numpy vector. If needed, we can reshape the vector into a desired format:

```
np.arange(10)  # vector of length 10
```

```
## array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
np.arange(10).reshape((2,5))  # 2x5 matrix
```

```
## array([[0, 1, 2, 3, 4],
##        [5, 6, 7, 8, 9]])
```

`np.zeros` and `np.ones` create arrays filled with zeros and ones respectively:

```
np.zeros((5,))
```

```
## array([0., 0., 0., 0., 0.])
```

```
np.ones((2,4))
```

```
## array([[1., 1., 1., 1.],
##        [1., 1., 1., 1.]])
```

Arrays can be combined in different ways, e.g. `np.column_stack` combines them as columns (next to each other), and `np.row_stack` combines these as rows (underneath each other). For instance, we can combine a column of ones and two columns of zeros as follows:

```
oneCol = np.ones((5,))  # a single vector of ones
zeroCols = np.zeros((5,2))  # two columns of zeros
np.column_stack((oneCol, zeroCols))  # 5x3 columns
```

```
## array([[1., 0., 0.],
##        [1., 0., 0.],
##        [1., 0., 0.],
##        [1., 0., 0.],
##        [1., 0., 0.]])
```

Note that `column_stack` expects all arrays to be passed as a single tuple (or list).

Exercise 3.1 Use `np.zeros` , `np.ones` , mathematical operations and concatenation to create the following array:

```
## array([[-1., -1., -1., -1.],
##        [ 0.,  0.,  0.,  0.],
##        [ 2.,  2.,  2.,  2.]])
```

See the solution

3.1.4 Vectorized Functions (Universal Functions)

It is possible to use loops to do computation with numpy objects exactly in the same way when working with lists. However, one should use vectorized operations instead whenever possible. Vectorized operations are easier to code, easier to read, and result in faster code.

Numpy offers a plethora of vectorized functions and operators, called *universal functions*. Many of these work as expected. For instance, mathematical operations. We create a matrix, and then add “100” to it, and then rise “2” to the power of the values:

```
a = np.arange(12).reshape((3,4))
print(a)
```

```
## [[ 0  1  2  3]
##  [ 4  5  6  7]
##  [ 8  9 10 11]]
```

```
print(100 + a, "\n")
```

```
## [[100 101 102 103]
##  [104 105 106 107]
##  [108 109 110 111]]
```

```
print(2**a, "\n")  # remember: exponent with **, not with ^
```

```
## [[   1    2    4    8]
##  [  16   32   64  128]
##  [ 256  512 1024 2048]]
```

Both of these mathematical operations, `+` and `**` are performed elementwise[2] for every single element of the matrix.

Exercise 3.2 Create the following array:

```
## array([[ 2,  4,  6,  8, 10],
##        [12, 14, 16, 18, 20],
##        [22, 24, 26, 28, 30],
##        [32, 34, 36, 38, 40]])
```

See the solution

Comparison operators are vectorized too:

```
a > 6
```

```
## array([[False, False, False, False],
##        [False, False, False,  True],
##        [ True,  True,  True,  True]])
```

```
a == 7
```

```
## array([[False, False, False, False],
##        [False, False, False,  True],
##        [False, False, False, False]])
```

As comparison operators are vectorized, one might expect that the other logical operators, *and*, *or* and *not*, are also vectorized. But this is not the case. There are vectorized logical operators, but they differ from the base python version. These are more similar to corresponding operators in R or C, namely `&` for logical and, `|` for logical or, and `~` for logical not:

```
(a < 3) | (a > 8)  # logical or
```

```
## array([[ True,  True,  True, False],
##        [False, False, False, False],
##        [False,  True,  True,  True]])
```

```
(a > 4) & (a < 7)  # logical and
```

```
## array([[False, False, False, False],
##        [False,  True,  True, False],
##        [False, False, False, False]])
```

```
~(a > 6)  # logical not
```

```
## array([[ True,  True,  True,  True],
##        [ True,  True,  True, False],
##        [False, False, False, False]])
```

There is no vectorized multi-way comparison like `1 < x < 2`.

3.1.5 Array Indexing and Slicing

Indexing refer to extracting elements based on their position or certain criteria. This is one of the fundamental operations with arrays. There are two ways to extract elements: based on position, and based on logical criteria. Unfortunately, this also makes indexing somewhat confusing, and it needs some time to become familiar with.

3.1.5.1 Extracting elements based on position

Array indexing is very similar to list indexing. As matrices have two dimensions, we need two indices.

```
a = np.arange(12)
print(a[::2])  # every second element
```

```
## [ 0  2  4  6  8 10]
```

However, unlike lists, one can do vectorized assignments in numpy:

```
a[5:11] = -1  # assign multiple elements
a
```

```
## array([ 0,  1,  2,  3,  4, -1, -1, -1, -1, -1, -1, 11])
```

One can also extract multiple elements from a vector:

```
a[[4,5,7]]  # extract 3 elements in one go
```

```
## array([ 4, -1, -1])
```

When working with matrices (2-D arrays), we need two indices, separated by comma. Comma separates two slices

```
c = np.arange(12).reshape((3,4))
c
```

```
## array([[ 0,  1,  2,  3],
##        [ 4,  5,  6,  7],
##        [ 8,  9, 10, 11]])
```

```
c[1,2]  # 2nd row, 3rd column
```

```
## 6
```

```
c[1] # 2nd row
```

```
## array([4, 5, 6, 7])
```

Comma can separate not just two indices but two slices, so we can write

```
c[:,2]  # all rows, 3rd column
```

```
## array([ 2,  6, 10])
```

```
c[:2]  # 1st, 2nd row
```

```
## array([[0, 1, 2, 3],
##        [4, 5, 6, 7]])
```

```
c[:2, :3]  # 1s, 2nd row, first three columns
```

```
## array([[0, 1, 2],
##        [4, 5, 6]])
```

Exercise 3.3 Create matrix and access rows and columns

- create a 4x5 array of even numbers: 10, 12, 14, …
- extract third column
- set the fourth row to 1,2,3,4,5

Note: there are many ways to achieve this.

See the solution

3.1.5.2 Boolean indexing

An extremely widely used approach is to extract elements of an array based on a logical criteria. Fundamentally, it is just using a logical vector for indexing. The vector must be of the same lengts as the array in question, and the results *contains only those elements the correspond to* `True` *in the indexing vector*. Here is an example how we can do this manually:

```
a = np.array([1,2,7,8])
i = np.array([True, False, True, False])
a[i]  # 1, 7
```

```
## array([1, 7])
```

It is important you understand what is going on here: arrays `a` and `i` will be "matched", so each element of `a` will have its "match" in `i`. Next, only those elements of `a` that are matched with `True` are extracted, in this case just 1 and 7.

The previous example–manually creating a logical index vectors of trues and falses is hardly ever useful. Almost always we use logical operations instead. For instance, we can extract all elements of `a` that are greater than 5:

```
i = a > 5
i
```

```
## array([False, False,  True,  True])
```

```
a[i]
```

```
## array([7, 8])
```

This is often written in a more compact manner by skipping explicit logical vector `i`:

```
a[a > 5]
```

```
## array([7, 8])
```

New users of numpy (and other languages that support logical indexing) sometimes forget that the logical condition does not have to be related to the same array that we are attempting to extract. For instance, we can extract all results for a certain person:

```
names = np.array(["Cyrus", "Darius", "Xerxes", "Artaxerxes", "Cyrus", "Darius"])
results = np.array([17, 14, 20, 18, 13, 15])
results[names == "Darius"]
```

```
## array([14, 15])
```

Here index vector is based on the variable `name` only and is not directly related to `results`. However, we use it to extract values from the latter.

Finally, we also can extract rows (or columns) from a 2-D array in a fairly similar fashion:

```
names = np.array(["Cyrus", "Darius", "Xerxes"])
results = np.array([[17, 14], [20, 18], [13, 15]])
results
```

```
## array([[17, 14],
##        [20, 18],
##        [13, 15]])
```

```
results[names == "Darius",:]
```

```
## array([[20, 18]])
```

The results is the second row of the 2-D array `results`, corresponding to the name “Darius”.

Logical indexing can also be used on the left-hand-side of the expression, in order to replace elements. Below is an example where we replace all the negative elements of `a` with zero.

```
a = np.random.randn(2,3)
a
```

```
## array([[ 1.3315865 ,  0.71527897, -1.54540029],
##        [-0.00838385,  0.62133597, -0.72008556]])
```

```
a[a < 0] = 0
a
```

```
## array([[1.3315865 , 0.71527897, 0.        ],
##        [0.        , 0.62133597, 0.        ]])
```

When replacing elements in such fashion then we need to supply the replacement vector that is either length 1 (all elements are replaced by "0" in the example above), or alternatively we should supply a replacement vector of correct length. For instance, we can replace the positive numbers left in `a` with 1, 2, 3:

```
a[a > 0] = np.array([1, 2, 3])
a
```

```
## array([[1., 2., 0.],
##        [0., 3., 0.]])
```

Exercise 3.4 consider two vectors

```
names = np.array(["Roxana", "Statira", "Roxana", "Statira", "Roxana"])
score = np.array([126, 115, 130, 141, 132])
```

Do the following using a single one-line vectorized operation.

- Extract all test scores that are smaller than 130
- Extract all test scores by Statira
- Add 10 points to Roxana's scores. (You need to extract it first.)

See the solution

3.1.6 Random numbers

TBD: make a separate section with some simulations and related examples.

Numpy offer a large set of random number generators. These can be invoked as `np.random.` *generator* `(` *params* `, size)`. For instance, `np.random.choice(N)` can be used to create random numbers from 0 to $N-1$. `size` determines the shape of the resulting object.

NB! The argument is **size**, not *shape*, although it determines the output *shape*!

Here is an example to simulate roll of a die for 5 times:

```
x = np.random.choice(6, size=5)
x
```

```
## array([0, 2, 0, 4, 3])
```

But maybe we prefer not to label the results as 0..5 but 1..6. So we can just add one to the result. Here is an example that creates 2-D array of die rolls:

```
1 + np.random.choice(6, size=(2,4))
```

```
## array([[1, 5, 4, 1],
##        [4, 3, 2, 1]])
```

Numpy offers a large set of various random values. Here we list a few more:

3.1.6.1 Random elements from list

`random.choice` can also extract random elements from a list:

```
nucleotides = ["A", "G", "C", "T"]
dna = np.random.choice(nucleotides, 20)
"".join(dna)
```

```
## 'ACGTCGGGTGCGACCCGAGT'
```

As the example demonstrates, `random.choice` picks random elements with replacement (use `replace` option to change this behavior).

3.1.6.2 Random normals

`random.normal(loc, scale, size)` generates normally distributed random numbers. The distribution is centered at *loc* and its variance is *scale*:

```
np.random.normal(1000, 100, size=10)
```

```
## array([ 969.13995143,  984.62645147, 1067.89311948,  808.64457868,
##          905.75419444,  825.4457021 ,  897.90547332,  983.79309738,
##          934.20291005, 1042.21130617])
```

3.1.6.3 Binomial random numbers

`random.binomial(n, p, size)` creates random binomials where probability of success is *p* and sample size is *n*:

```
np.random.binomial(2, 0.5, size=(2,4))
```

```
## array([[2, 2, 1, 1],
##        [2, 2, 1, 2]])
```

Exercise 3.5 We can describe a coin toss as *Binomial(1, 0.5)* where *1* refers to the fact that we toss a single coin, and *0.5* means it has probability 0.5 to come heads up. So such random variables are sequences of zeros and ones. But how can we get a sequence of -1 and 1 instead? Demonstrate it on computer!

See the solution

3.1.6.4 Uniform random numbers

`random.uniform(low, high, size)` creates uniformly distributed random numbers in the interval *[low, high]*:

```
np.random.uniform(-1, 1, size=(3,4))  # random numbers in [-1, 1]
```

```
## array([[-0.4078626 , -0.73741789,  0.68563587,  0.31807261],
##        [ 0.19087921, -0.1272926 , -0.28749935,  0.17426185],
##        [-0.70105733, -0.6575228 , -0.20567095,  0.27590313]])
```

3.1.6.5 Repeating the exact same random sequence

The random numbers are often called *pseudorandom* as they are not truly random–they are computed based on a well-defined algorithm, so when feeding the same initial values to the algorithm, one always gets the same random numbers. However, normally the initial values are taken from certain hart-to-control parameters outside of the program control, such as time in microseconds and hard disk serial number, so in practice it is impossible to replicate the same sequence.

However, if you need to replicate your results exactly, you have to set the initial values explicitly using `random.seed(value)`. This re-initializes RNG-s to the given initial state:

```
np.random.seed(1)
np.random.uniform(size=5)  # 1st batch of numbers
```

```
## array([4.17022005e-01, 7.20324493e-01, 1.14374817e-04, 3.02332573e-01,
##        1.46755891e-01])
```

```
np.random.uniform(size=5)  # 2nd batch is different
```

```
## array([0.09233859, 0.18626021, 0.34556073, 0.39676747, 0.53881673])
```

```
np.random.seed(1)
np.random.uniform(size=5)  # repeat the 1st batch
```

```
## array([4.17022005e-01, 7.20324493e-01, 1.14374817e-04, 3.02332573e-01,
##        1.46755891e-01])
```

3.1.7 Statistical functions

Numpy offers a set of basic statistical functions, including *sum*, *mean*, and standard deviations *std*. These can be applied to the array as a whole, or separately to rows or columns. In the latter case one has to specify the argument `axis`, where the value 0 means to apply the operation row-wise (and preserve

columns) and `axis=1` means to apply the operation column-wise (and preserve rows). Here is an example:

```
a = np.arange(12).reshape((3,4))
a  # 3 rows, 4 columns
```

```
## array([[ 0,  1,  2,  3],
##        [ 4,  5,  6,  7],
##        [ 8,  9, 10, 11]])
```

```
a.sum()  # total sum
```

```
## 66
```

```
a.sum(axis=0)  # add rows, preserve columns
```

```
## array([12, 15, 18, 21])
```

```
a.sum(axis=1)  # add columns, preserve rows
```

```
## array([ 6, 22, 38])
```

The functions come in two forms: as a method `x.sum()`, and as a separate function `np.sum(x)`. These two ways are pretty much equivalent.

By default, a missing value of an array causes the function to return missing:

```
a = a.astype(float)  # as np.nan is float, need a float array
a[1,2] = np.nan
a
```

```
## array([[ 0.,  1.,  2.,  3.],
##        [ 4.,  5., nan,  7.],
##        [ 8.,  9., 10., 11.]])
```

```
np.sum(a)
```

```
## nan
```

NB! This differs from the corresponding functionality in pandas where missings are ignored by default!

The other statistical functions include

- `mean` for average
- `median` for median
- `var` for variance
- `std` for standard deviation
- `np.percentile` and `np.quantile` for quantiles

3.2 Pandas

Pandas is the standard python library to work with dataframes. Unlike in R, this is not a part of base python and must be imported separately. It is typically imported as `pd`:

```
import pandas as pd
```

Pandas logo. Marc Garcia, BSD license, via Wikimedia Commons.

```
## /home/otoomet/R/x86_64-pc-linux-gnu-library/4.4/reticulate/python/rpytools/loader.py:120: UserWa
##   return _find_and_load(name, import_)
```

Pandas relies heavily on numpy but is a separate package. Unfortunately, it also uses a somewhat different syntax and somewhat different defaults. However, as it is "made of" numpy, it works very well together with the latter.

Pandas contains two central data types: *Series* and *DataFrame*. Series is often used as a second-class citizen, just as a single variable (column) in data frame. But it can also be used as a vectorized dict that links keys (indices) to values. *DataFrame* is broadly similar to other dataframes as implemented in R or spark. When you extract its individual columns and rows you normally get those in the form of *Series*. So it is extremely useful to know the basics of *Series* when working with data frames. Both *DataFrame* and *Series* include *index*, a glorified row name, which is very useful for extracting information based on names, or for merging different variables into a data frame (See Section Concatenating data with `pd.concat`).

We start by introducing *Series* as this is a simpler data structure than *DataFrame*, and allows us to introduce *index*.

3.2.1 Series

Series is a one-dimensional positional column (or row) of values. It is in some sense similar to list, but from another point of view it is more like a dict, as it contains *index*, and you can look up values based on index as a key. So it allows not only positional access but also index-based (key-based) access. In terms of internal structure, it is implemented with vectorized operations in mind, so it supports vectorized arithmetic, and vectorized logical, string, and other operations. Unlike dicts, it also supports multi-element extraction.

Let's create a simple series:

```
s = pd.Series([1,2,5,6])
s
```

```
## 0    1
## 1    2
## 2    5
## 3    6
## dtype: int64
```

Series is printed in two columns. The first one is the index, the second one is the value. In this example, index is essentially just the row number and it is not very useful. This is because we did not provide any specific index and hence pandas picked just the row number. Underneath the two columns, you can also see the data type, in this case it is 64-bit integer, the default data type for integers in python.

Now let's make another example with a more informative index:

```
pop = pd.Series( [ 38, 26, 19, 19],
                 index = ['ca', 'tx', 'ny', 'fl'])
# population, in millions
pop
```

```
## ca    38
## tx    26
## ny    19
## fl    19
## dtype: int64
```

Now the index is helpful: we are looking at state populations, and index tells us which state is in which row. Another advantage of possessing index is that even when we filter and manipulate the series, it's index will still retain the original row label. So we know that index "fl" will always correspond to Florida. But if we have removed a few cases, or re-ordered the series, then Florida may not be on the fourth position any more.

Exercise 3.6 Create a series of 4 capital cities where the index is the name of corresponding country.

See the solution

We can extract values and index using the corresponding attributes:

```
pop.values
```

```
## array([38, 26, 19, 19])
```

```
pop.index
```

```
## Index(['ca', 'tx', 'ny', 'fl'], dtype='object')
```

Note that values are returned as np array, and index is a special index object. If desired, this can be converted to a list:

```
list(pop.index)
```

```
## ['ca', 'tx', 'ny', 'fl']
```

Series also supports ordinary mathematics, e.g. we can do operations like

```
pop > 20
```

```
## ca     True
## tx     True
## ny    False
## fl    False
## dtype: bool
```

the result will be another series, here of logical values, as indicated by the "bool" data type.

3.2.2 DataFrame

DataFrame is the central data structure for holding 2-dimensional rectangular data. It is in many ways similar to R dataframes. However, it also shares a number of features with Series, in particular the index, so you can imagine a data frame is just a number of series stacked next to each other. Also, extracting single rows or columns from DataFrames typically results in a series.

3.2.2.1 Creating data frames

DataFrame can be created manually as a dict of lists (or series). The keys of the list are the variable names and values are the variable values, normally these are lists or series. As an example, let's create a data frame with three variables, *ca*, *tx* and *md*, and three rows:

```
df = {'ca': [35, 37, 38], 'tx': [23, 24, 26], 'md': [5,5,6]}
pop = pd.DataFrame(df)
print('population:\n', pop, '\n')
```

```
## population:
##     ca  tx  md
## 0  35  23   5
## 1  37  24   5
## 2  38  26   6
```

The data frame is printed as four columns. Exactly as in case of *series*, the first column is index. In the example above we did not specify the index and hence pandas picked just row numbers. But we can provide an explicit index, for instance the year of observation:

```
pop = pd.DataFrame(df, index = [2010,2012,2014])
print('population:\n', pop, '\n')
```

```
## population:
##        ca  tx  md
## 2010  35  23   5
## 2012  37  24   5
## 2014  38  26   6
```

In this case the index is rather useful.

Exercise 3.7 Create a dataframe of (at least 4) countries, with 2 variables: population and capital. Country name should be the index.

Hint: feel free to invent populations!

See the solution

3.2.2.2 Read data from file

To create data frames manually is useful for testing and debugging, in real applications we typically read data from disk. This can be done with `pd.read_csv` that takes the file name as the first argument, and also supports many other options. In the example below, we read data about G.W.Bush approval rate in fall 2001. `pd.read_csv` assumes files are comma-separated by default, but as this example file is tab-separated we have to declare it using `sep="\t"` as an extra argument. We also read the first 10 rows only for demonstration:

```
approval = pd.read_csv("../data/gwbush-approval.csv", sep="\t", nrows=10)
approval
```

```
##              date  approve  disapprove  dontknow
## 0  2001 Dec 14-16       86          11         3
## 1    2001 Dec 6-9       86          10         4
## 2  2001 Nov 26-27       87           8         5
## 3   2001 Nov 8-11       87           9         4
## 4    2001 Nov 2-4       87           9         4
## 5  2001 Oct 19-21       88           9         3
## 6  2001 Oct 11-14       89           8         3
## 7    2001 Oct 5-6       87          10         3
## 8  2001 Sep 21-22       90           6         4
## 9  2001 Sep 14-15       86          10         4
```

Exercise 3.8 In the example above: how many columns are printed? How many variables does the dataframe contain?

See the solution

What happens if we use a wrong separator? This can be easily checked with printing the number of columns, and printing a few lines of data. Here is an example:

```
a = pd.read_csv("../data/gwbush-approval.csv")  # wrong separator
a.shape
```

```
## (31, 1)
```

```
a.head(2)
```

```
##   date\tapprove\tdisapprove\tdontknow
## 0           2001 Dec 14-16\t86\t11\t3
## 1             2001 Dec 6-9\t86\t10\t4
```

Two problems are immediately visible: first, the file contains a single column only (because it does not consider *tab* symbols as separators), and the two lines we printed look weird. If you ask for variable names, you can also see that all variable names are combined together into a single weird name:

```
a.columns
```

```
## Index(['date\tapprove\tdisapprove\tdontknow'], dtype='object')
```

The *tab* markers `\t` in printout give strong hints that the correct separator is *tab*.

It may initially be quite confusing to understand how to specify the file name. If you load data in a jupyter notebook, then the working directory is normally the same directory where the notebook is located[3]. Notebook also let's you to complete file names with *TAB* key. But in any case, the working directory can be found with `os.getcwd` (**get c**urrent **w**orking **d**irectory):

```
import os
os.getcwd()
```

```
## '/home/otoomet/tyyq/lecturenotes/machinelearning-py'
```

This helps to specify the relative path if your data file is not located in the same place as your code. You can also find which files does python find in a given folder, e.g. in `../data/` :

```
files = os.listdir("../data/")
files[:5]
```

```
## ['covid-scandinavia.csv.bz2', 'gsaf5-2020.csv.bz2', 'java-earthquakes.csv.bz2', 'country-concep
```

As we see, this function returns a list of file names it finds in the given location.

Exercise 3.9 Refresh your knowledge of relative paths!

- What is your current working directory?
- List all files in
  - your current folder
  - in the parent folder of it.

See the solution

As another complication, notebooks are often run on a separate server or in a docker container. These may have no access to files in your computer (as the server), or only have a limited access (like docker container). In such a case, you should upload the file to notebook, *even if it is running on your computer*!

3.3 Indexing data frames and *series*

Indexing refers to selecting data from data frames and series based on variable names, logical conditions, and position. It is a complex task with many different methods, and unfortunately also with many caveats. Below, the topic is split into several subsections:

- Select variables explains how to select desired variables from a data frame
- Filter observations with logical operations describes how to filter rows
- Positional indexing of Series introduces positional indexing, indexing based on row number, and how to do it with *series*
- Positional indexing of data frames explains positional indexing, indexing based on both row and column numbers, for data frames
- Modifying data frames: there are slight differences when modifying data instead of extracting, these are discussed here.
- Indexing: summary and comparison provides a summary of all methods.

Fortunately, *Series* and data frames behave in a broadly similar way, e.g. selecting cases by logical conditions, based on index, and location are rather similar. As series do not have columns, we cannot access elements by column name or by column position though.

These notes do not provide a comprehensive overview, consult e.g. McKinney "Python for Data Analysis for more details.

3.3.1 Select variables in data frames

We use the G.W.Bush approval data we loaded above to demonstrate variable access. For a refresher, the first lines of the data frame look like

```
approval.head(4)
```

```
##             date  approve  disapprove  dontknow
## 0  2001 Dec 14-16       86          11         3
## 1    2001 Dec 6-9       86          10         4
## 2  2001 Nov 26-27       87           8         5
## 3   2001 Nov 8-11       87           9         4
```

To begin with, data frames have variable names. We can extract a single variable either with `["varname"]` or a shorthand as attribute `.varname` (note: replace *varname* with the name of the relevant variable):

```
approval["approve"]  # approval, as series
```

```
## 0    86
## 1    86
## 2    87
## 3    87
## 4    87
## 5    88
## 6    89
## 7    87
## 8    90
## 9    86
## Name: approve, dtype: int64
```

```
approval.approve  # the same, as series
```

```
## 0    86
## 1    86
## 2    87
## 3    87
## 4    87
## 5    88
## 6    89
## 7    87
## 8    90
## 9    86
## Name: approve, dtype: int64
```

These constructs return the column as a series. If we prefer to get a single-column data frame, we can wrap the variable name into a list:

```
approval[["approve"]]  # approval, as data frame
```

```
##    approve
## 0       86
## 1       86
## 2       87
## 3       87
## 4       87
## 5       88
## 6       89
## 7       87
## 8       90
## 9       86
```

The attribute shorthand is usually the easier way, but it does not work if you need to use indirect variable name (variable name that is stored in another variable) or if the variable name contains spaces or other special characters. It also does not work for creating new variables in the data frame. See more in Section 3.3.5.

The previous example where we extracted a single column as a data frame instead of *Series* also hints how to extract more than one variable: just wrap all the required variable names into a list:

```
vars = ["date", "approve"]
approval[vars]
```

```
##              date  approve
## 0  2001 Dec 14-16       86
## 1    2001 Dec 6-9       86
## 2  2001 Nov 26-27       87
## 3   2001 Nov 8-11       87
## 4    2001 Nov 2-4       87
## 5  2001 Oct 19-21       88
## 6  2001 Oct 11-14       89
## 7    2001 Oct 5-6       87
## 8  2001 Sep 21-22       90
## 9  2001 Sep 14-15       86
```

There are no attribute shortcuts to extract multiple columns.

3.3.2 Filter observations with logical operations

Filtering refers to extracting only a subset of rows from the dataframe based on certain conditions. The conditions are logical operations that can be either true or false, depending on the values in each row. Filtering produces a sub-dataframe where only those observations that meet the selection criteria are

present: Here is an example:

```
approval[approval.approve > 88]
```

```
##              date  approve  disapprove  dontknow
## 6  2001 Oct 11-14       89           8         3
## 8  2001 Sep 21-22       90           6         4
```

Note that we have to refer to data variables as `approval.approve`, not just `approve`, unlike in R *dplyr* where one can just write `approve`. This is somewhat harder to write but it is less ambiguous and produces fewer hard-to-find bugs.

Obviously we can use more complex selection conditions, for instance we can look for very low or very high approval rates as follows:

```
approval[(approval.approve < 86) | (approval.approve > 89)]
```

```
##              date  approve  disapprove  dontknow
## 8  2001 Sep 21-22       90           6         4
```

Note that we are using the vectorized "or" operator `|`, not the base python `or`. We also need to wrap both the "less than" and "greater than" parts in parenthesis.
See more in Section 3.1.4.

Exercise 3.10 How many polls in the data show the president's approval rate at least 88%? At which dates are those polls conducted?

See the solution

NB! The filtered object is not a new data frame but a *view* of the original data frame. This may give you warnings and errors later when you attempt to modify the filtered data. If you intend to do that, perform a *deep copy* of data using the `.copy` method. See more in Section 3.3.5.

3.3.3 Positional indexing of Series

Besides selecting variables and filtering by logical conditions, we occasionally need to access elements by index, or by position (location). Here we demonstrate the positional indexing using a series object, positional indexing of data frames is discussed in Section 3.3.4 below:

```
pop = pd.Series([32.7, 267.7, 15.3],  # in millions
                index=["MY", "ID", "KH"])
pop
```

```
## MY     32.7
## ID    267.7
## KH     15.3
## dtype: float64
```

We can access series' values in two ways: by position, and by index. In order to access elements by position, we have to use attribute `.iloc[]` where *i* loc refers to "integer". Unlike most other methods, `.iloc` expects arguments in brackets. A single number in brackets returns the element as an element (e.g. a single number), if brackets contain a list (this looks like double brackets), it returns a series, potentially containing only a single element. So in order to extract 2nd and 3rd element in the population series, we can write:

```
pop.iloc[1]  # extract 2nd element as a number
```

```
## 267.7
```

```
pop.iloc[[1,2]]   # extract 2nd, 3th as a series
```

```
## ID    267.7
## KH     15.3
## dtype: float64
```

Alternatively, we can also extract the elements by index. This works in a similar fashion, except we have to use `.loc[]` instead of `.iloc[]`. The rules for single and double brackets apply in the similar fashion as in case of positional access.

```
pop.loc["ID"]  # extract Indonesian population as a number
```

```
## 267.7
```

```
pop.loc[["ID", "MY"]]  # extract Indonesian and Malaysian population
```

```
## ID    267.7
## MY     32.7
## dtype: float64
```

```
# as a series
```

Exercise 3.11 Use your series of capital cities (see the exercise above). Extract:

- 1st, 3rd element by position as single elements (city names)
- 2nd element by country name as a 1-element series.

See the solution

One can also drop the `.loc[]` syntax and just use square brackets, so instead of writing `pop.loc[["ID", "MY"]]`, one can just write `pop[["ID", "MY"]]`.

The fact that there are several ways to extract positional data causes a lot of confusion for beginners. It is not helped by the common habit of not using indices and just relying on the automatic row-numbers. In this case positional access by `.iloc[]` produces exactly the same results as the index access by `.loc[]`, and one can conveniently forget about the index and use whatever feels easier. But sometimes the index changes as a result of certain operations and that may lead to errors or unexpected results. For instance, we can create an alternative population series without explicit index:

```
pop1 = pd.Series([np.nan, 26, 19, 13])  # index is 0, 1, ...
pop1
```

```
## 0     NaN
## 1    26.0
## 2    19.0
## 3    13.0
## dtype: float64
```

In this example, position and index are equivalent and hence it is easy to forget that `.loc[]` is index-based access, not positional access! So one may freely mix both methods (and remember, `.loc` is not needed):

```
pop1.loc[2]
```

```
## 19.0
```

```
pop1.iloc[2]
```

```
## 19.0
```

```
pop1[2]
```

```
## 19.0
```

This becomes a problem if a numeric index is not equivalent to row number any more, for instance after we drop missings:

```
pop2 = pop1.dropna()  # remove missings
pop2  # note: the first row has index 1
```

```
## 1    26.0
## 2    19.0
## 3    13.0
## dtype: float64
```

```
pop2.iloc[2]  # this is by position
```

```
## 13.0
```

```
pop2.loc[2]  # this is by index
```

```
## 19.0
```

```
pop2[2]  # also by index
```

```
## 19.0
```

Additionally, if `pop2` for some reason turns into a numpy array, then `pop2[2]` is is based on position as arrays do not have index!

3.3.4 Positional indexing of data frames

We use a small data frame of capital cities to demonstrate how indexing on data frames works. The data frame contains two variables, name of the capital city and population as variables, index is the country name:

```
countries = pd.DataFrame({"capital":["Kuala Lumpur", "Jakarta", "Phnom Penh"],
                          "population":[32.7, 267.7, 15.3]},  # in millions
                         index=["MY", "ID", "KH"])
countries
```

```
##          capital  population
## MY  Kuala Lumpur        32.7
## ID       Jakarta       267.7
## KH    Phnom Penh        15.3
```

(*MY* is Malaysia, *ID* Indonesia and *KH* is Cambodia).

Exactly as series, data frames allow positional access by `.iloc[]`. However, as data frames are two-dimensional objects, `.iloc` accepts two arguments (in brackets, separated by comma), the first one for rows, the second one for columns. So we can write

```
countries.iloc[2]  # 3rd row, as series
```

```
## capital       Phnom Penh
## population          15.3
## Name: KH, dtype: object
```

```
countries.iloc[[2]]  # 3rd row, as data frame
```

```
##        capital  population
## KH  Phnom Penh        15.3
```

```
countries.iloc[2,1]  # 3rd row, 2nd column, as a number
```

```
## 15.3
```

There is also an index-based extractor `.loc[]` that accepts one (for rows) or two (for rows and columns) indices. In case of data frames, the default row index is just the row number; but the column index is the variable names. So we can write

```
countries.loc["MY","capital"]  # Malaisian capital
```

```
## 'Kuala Lumpur'
```

```
countries.loc[["KH", "ID"], ["population", "capital"]]
```

```
##     population     capital
## KH        15.3  Phnom Penh
## ID       267.7     Jakarta
```

```
# Extract a sub-dataframe
```

Unfortunately, data frames add their confusing constructs. When accessing data frames with `.loc[]` then we have to specify rows first, and possibly columns second. If we drop `.loc` then we cannot specify rows. That is, unless we extract one variable with brackets, get a series and extract the desired row in the second set of brackets…

```
countries["capital"]
```

```
## MY    Kuala Lumpur
## ID         Jakarta
## KH      Phnom Penh
## Name: capital, dtype: object
```

```
countries["capital"]["MY"]
```

```
## 'Kuala Lumpur'
```

Finally, remember that 2-D numpy arrays will use similar integer-positional syntax as `.iloc[]`, just without `.iloc`.

In conclusion, it is very important to know what is your data type when using numpy and pandas. Indexing is all around us when working with data, there are many somewhat similar ways to extract elements, and which way is correct depends on the exact data type.

3.3.5 Modifying data frames

Modifying data frames can be done in a broadly similar way as extracting elements, you just need to put the expression on the left-hand side.

We can create a new column by assigning a list to a new column name:

```
countries["temperature"] = [27.7, 26.1, 26.6]  # daily mean, January
countries
```

```
##          capital  population  temperature
## MY  Kuala Lumpur        32.7         27.7
## ID       Jakarta       267.7         26.1
## KH    Phnom Penh        15.3         26.6
```

One can also overwrite an existing column in a similar fashion.

However, there are several exceptions and caveats. Let's demonstrate this by modifying the data frame of three countries we created above.

3.3.5.1 One cannot create variables with dot-attribute

We can extract a single series as `countries.capital`, but we cannot create a new variable with dot-notation. For instance:

```
countries.foo = [1, 2, 3]  # does not work
```

```
## <string>:1: UserWarning: Pandas doesn't allow columns to be created via a new attribute name - :
```

But after we have created a column with bracket-notation, we can access it using dot-notation.

3.3.5.2 Explicitly make copy when working with filtered data

A typical data science workflow consists of a) filtering data to relevant cases only, and b) modifying the resulting subset. The first step often involves removing missing values, or limiting the analysis to a certain subset of interest. It is important to realize that *Pandas*' filtering does not copy the interesting cases in memory, it may instead just create a *view*, i.e. re-use the same location in computer memory but just limit access to certain part of it.[4] A view is another way to make a shallow copy (see Section 2.5.1), but the resulting slice does not use all the original memory but just a part of it. This is a very good idea in terms of conserving memory and avoiding unnecessary copy operations. However, this may cause warnings and errors when modifying the filtered data later. We demonstrate this on the same dataset.

Select only large countries (population over 20M):

```
large = countries[countries.population > 20]
large
```

```
##          capital  population  temperature
## MY  Kuala Lumpur        32.7         27.7
## ID       Jakarta       267.7         26.1
```

We got a subset of Malaysia and Indonesia. Now let's add another variable to these large countries:

```
large["language"] = ["Malay", "Indonesian"]
```

```
## <string>:1: SettingWithCopyWarning:
## A value is trying to be set on a copy of a slice from a DataFrame.
## Try using .loc[row_indexer,col_indexer] = value instead
##
## See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/i
```

```
large
```

```
##          capital  population  temperature    language
## MY  Kuala Lumpur        32.7         27.7       Malay
## ID       Jakarta       267.7         26.1  Indonesian
```

Note the warning: *A value is trying to be set on a copy of a slice…*. This tells you that filtering `countries[countries.population > 20]` did not create a new data frame but a view of the existing one in memory, and *Pandas* is unhappy with the code modifying just a part of the original data frame.

> NB!
>
> Although the result appears correct here, do not rely on this approach! It may or may not work, depending on the exact memory layout of the dataset!

Fortunately, the solution is very simple. We need to make an explicit copy with `.copy` method before we start any modifications:

```
large = countries[countries.population > 20].copy()  # explicit copy
large["language"] = ["Malay", "Indonesian"]
large
```

```
##          capital  population  temperature    language
## MY  Kuala Lumpur        32.7         27.7       Malay
## ID       Jakarta       267.7         26.1  Indonesian
```

Now the modification works without a warning.

Explicit copy is not needed before you start modifying data, you can do various filtering steps without `.copy` as long as you make the copy before modifications.

3.3.5.3 Modifying index

The index that is attached to series' and data frames is potentially a useful and iformative tool. But sometimes it is not very useful. For instance, when you load data from disk, then the index defaults to be the row number, and this is rarely what we are interested in. In such cases one may want to change the index. If you want to create a new index then you can just assign it to `df.index`. For instance, we can just assign country names as index to our data frame of large countries:

```
large.index = ["Malaysia", "Indonesia"]
large
```

```
##                capital  population  temperature    language
## Malaysia   Kuala Lumpur        32.7         27.7       Malay
## Indonesia       Jakarta       267.7         26.1  Indonesian
```

Alternatively, we can convert a column to index with `.set_index()` method:

```
large.set_index("capital")
```

```
##               population  temperature    language
## capital
## Kuala Lumpur        32.7         27.7       Malay
## Jakarta            267.7         26.1  Indonesian
```

This will remove the column "capital" from data frame as its values will be in index instead. Note that by default, `.set_index()` returns a new data frame instead of modifying it in place, so if you want to preserve it, you have to store it in a new variable. The opposite–converting the index into a column can be done with `.reset_index()` .

Exercise 3.12 Take the data frame of capital-population data frame from Section 3.3.4.

- Replace the index by country names
- Convert the index into a variable "country"

Ensure that you store and print the final data frame!

See the solution

3.3.6 Indexing: summary and comparison

Indexing data is complex. Here we repeat and summarize the main methods we have discussed so far. First create three objects, a numpy matrix, a data frame, and a series. The first two are 2-dimensional but the last one 1-dimensional.

```
M = np.array([[1507, 12478],
              [-500, 11034],
              [1537, 8443],
              [1591, 6810]])
M
```

```
## array([[ 1507, 12478],
##        [ -500, 11034],
##        [ 1537,  8443],
##        [ 1591,  6810]])
```

```
df = pd.DataFrame(M, columns=["established", "population"],
                  index=["Mumbai", "Delhi", "Bangalore", "Hyderabad"])
df
```

```
##            established  population
## Mumbai            1507       12478
## Delhi             -500       11034
## Bangalore         1537        8443
## Hyderabad         1591        6810
```

```
s = pd.Series(M[:,0], index=df.index)
s
```

```
## Mumbai       1507
## Delhi        -500
## Bangalore    1537
## Hyderabad    1591
## dtype: int64
```

(This is data about four cities, the year when those were established, and population in thousands).

Exercise 3.13 Create another numpy matrix and a data frame about cities in a similar fashion: create a matrix of data, and create a data frame from it using `pd.DataFrame`. Specify index (row names) and columns (variable names). Include at least 3 cities and 3 variables (e.g. population in millions, size in km2, and population density people per km2).

Hint: you may invent both city names and the figures!

See the solution

Extract rows/columns by number (integer):

- Numpy array: just use the numbers in brackets:

```
M[1,0]  # second row, first column
```

```
## -500
```

```
M[2,:]  # third row
```

```
## array([1537, 8443])
```

- Data frames: use `iloc` and brackets:

```
df.iloc[1,0]  # second row, first column
```

```
## -500
```

```
df.iloc[2,:]  # third row
```

```
## established    1537
## population     8443
## Name: Bangalore, dtype: int64
```

- Series: use `iloc` and brackets (but these are just 1-dimensional):

```
s.iloc[1]  # second row
```

```
## -500
```

Extract using index (city names/column names):

- numpy array: not possible
- Data frames: use `loc` and brackets:

```
df.loc["Delhi","established"]  # second row, first column
```

```
## -500
```

```
df.loc["Bangalore",:]  # third row
```

```
## established    1537
## population     8443
## Name: Bangalore, dtype: int64
```

- Series: use `loc` (but not columns here):

```
s.loc["Delhi"]
```

```
## -500
```

If we want to extract individual columns, we can do the following:

- Numpy arrays: use brackets and use a colon `:` in row indicators place:

```
M[:,0]
```

```
## array([1507, -500, 1537, 1591])
```

- Data frames: you can use `iloc` and brackets, exactly as in case of numpy arrays. You can also use brackets and column names (column index) without `iloc`, or dot-column name:

```
df.iloc[:,0]
```

```
## Mumbai       1507
## Delhi        -500
## Bangalore    1537
## Hyderabad    1591
## Name: established, dtype: int64
```

```
df["established"]
```

```
## Mumbai       1507
## Delhi        -500
## Bangalore    1537
## Hyderabad    1591
## Name: established, dtype: int64
```

```
df.established
```

```
## Mumbai       1507
## Delhi        -500
## Bangalore    1537
## Hyderabad    1591
## Name: established, dtype: int64
```

If you want to extract rows and columns in a mixed, e.g. rows by number, and columns by column names (index), you can use double extraction (two sets of brackets) and chain your extractions into a single line:

```
df.iloc[:3,:]["population"]
```

```
## Mumbai       12478
## Delhi        11034
## Bangalore     8443
## Name: population, dtype: int64
```

Exercise 3.14 Take your own city matrix and city data frame. From both of these extract:

- population density (for all cities)
- data for the third city. For the data frame do it in two ways: using index, and using row number!
- area of the second city. For the data frame, do it in two ways: using column name (column index), and column number!

See the solution

Finally, if asking for a single entry (singleton), pandas simplifies the result into a lower-ranked object (series instead of data frame, or a number instead of series). If you want to retain a similar data structure as the original one, wrap your selector in a list. For instance, the previous example that returns a data frame: single line:

```
df.iloc[:3,:][["population"]]
```

```
##            population
## Mumbai          12478
## Delhi           11034
## Bangalore        8443
```

All these methods can create rather confusing situations sometimes. For instance, if we do not specify index, it will be automatically created as row numbers (but starting from 0, not 1). In that case `df.iloc[i]` and `df.loc[i]` give the same result (assuming `i` is a list of row numbers). Even worse, if the index skips some numbers, then `df.loc[i]` may or may not work, and even where it works, it may give wrong results! In a similar fashion, `M[i,j]` works but `df[i,j]` does not work, `df.loc[i,j]` works but `M.loc[i,j]` does not work. In order to tell if the syntax is correct it is necessary to know what is the data structure.

2. There are also operations that are not performed elementwise when using array, in particular matrix product↩
3. If you run your code from command line, the working directory is the directory where you run the command, not the directory where the program is located.↩
4. *Pandas* decides whether to make a copy or a view in each case separately, depending on what is the more efficient approach.↩

DATA WRANGLING AND PREPROCESSING

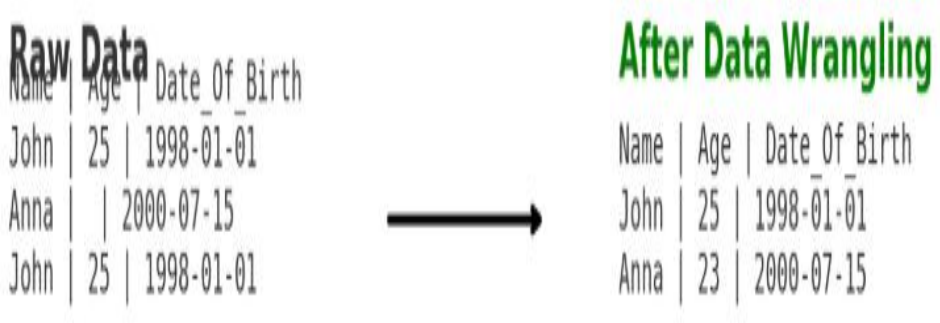
UNIT-II

Data Wrangling:

Data wrangling is nothing but cleaning, organizing, and transforming raw data into a usable format. Data Wrangling is used to remove duplicates handle missing values and fix errors. it converts raw data into proper structure.

Example:

Name	Age	Date_Of_Birth
John	25	1998-01-01
Anna		2000-07-15
John	25	1998-01-01



After data wrangling,

We fill Anna's missing age.

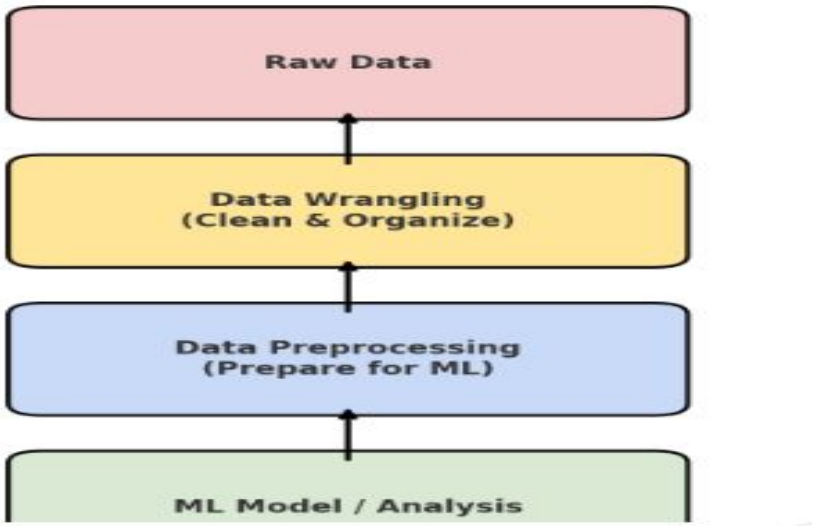
Remove duplicate rows.

Convert date formats if needed.

Data Preprocessing:

Raw data may have missing values and **wrong formats**. Data preprocessing makes the data clean**,** standardized**,** and **ready to use**.

Preparing raw data for machine learning or analysis by cleaning, transforming, and organizing it.



Handling Missing Data:

Missing data means some values in your dataset are empty or not available (in Python like NaN).

We have different methods to handle missing data. the different methods are

1. Mean
2. Median
3. Drop
4. Interpolation

MEAN:

Mean is one type of method to handle missing data. Mean means to fill missing value with column average.

EX:

```
import pandas as pd
data = {  'Name': ['Alice', 'Bob', 'Charlie', 'David'],
            'Age': [25, None, 30, 35]}
df = pd.DataFrame(data)
print(“Before Filling Missing Values:\n",df)
df[‘Age'] = df[‘Age'].fillna(df[‘Age'].mean())
print("\nAfter Filling Missing Values with Mean:\n", df)
```

OUTPUT:

Before Filling Missing Values:

	Name	Age
0	**Alice**	**25.0**
1	**Bob**	**NaN**
2	**Charlie**	**30.0**
3	**David**	**35.0**

After Filling Missing Values with Mean:

	Name	Age
0	Alice	25.0
1	Bob	30.0
2	Charlie	30.0
3	David	35.0

Median:

Median is another method to handle missing Data.Median means to Fill with middle value of the column.

Ex:

```
import pandas as pd
data = {  'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
          'Age': [25, 30, None, 35, 40]}
```

 # Create DataFrame

```
df = pd.DataFrame(data)
print("Before Filling Missing Values:\n", df)
```

 # Fill missing Age with the median of the Age column

```
df['Age'] = df['Age'].fillna(df['Age'].median())
print("\nAfter Filling Missing Values with Median:\n", df)
```

Output :

```
Before Filling Missing Values:
      Name       Age
0     Alice      25.0
1         Bob        30.0
2         Charlie    NaN
3     David      35.0
4      Eve       40.0
```

After Filling Missing Values with Median:

	Name	Age
0	Alice	25.0
1	Bob	30.0
2	Charlie	32.5
3	David	35.0
4	Eve	40.0

Drop:

Drop is on more type to handle missing data. Drop Means Remove rows or columns with missing values.

Ex:

```
import pandas as pd
data = {
    'Name': ['Alice', 'Bob', 'Charlie', None],
    'Age': [25, None, 30, 35]
}
```

 # Create DataFrame

```
df = pd.DataFrame(data)
print("Before Dropping Missing Values:\n", df)

# Drop rows with any missing (NaN) values
df_cleaned = df.dropna()
print("\nAfter Dropping Rows with Missing Values:\n", df_cleaned)
```

Output:

```
Before Dropping Missing Values:
      Name        Age
0      Alice       25.0
1      Bob         NaN
2     Charlie      30.0
3     None         35.0
```

After Dropping Rows with Missing Values:

	Name	Age
0	Alice	25.0
2	Charlie	30.0

Interpolation: Interpolation is one type of method to handle missing data. Interpolation means Estimate values based on nearby data.

Ex:

```
importpandas as pd
data = {
    'Day': [1, 2, 3, 4, 5],
    'Temperature': [30, 32, None, 36, 38]
}
```

```
# Create DataFrame
df = pd.DataFrame(data)
print("Before Interpolation:\n", df)
# Interpolate missing values
df['Temperature'] = df['Temperature'].interpolate()
print("\nAfter Interpolation:\n", df)
```

Output:

Before Interpolation:

	Day	Temperature
0	1	30.0
1	2	32.0
2	3	NaN
3	4	36.0
4	5	38.0

After Interpolation:

Day	Temperature
0 1	30.0
1 2	32.0
2 3	34.0
3 4	36.0
4 5	38.0

Dealing with Duplicates:

In large data set, the same row information repeated more than once in a table.These repeated rows are called duplicate entries.

Pandas provides several methods to find and remove duplicate entries in DataFrames.

Find Duplicate Entries:

duplicated () method is used to find the duplicates in the dataframe. If row is duplicate, it returns true. Otherwise it returns false.

Ex:

```
import pandas as pd
# create dataframe
data = {
    'Name': ['John', 'Anna', 'John', 'Anna', 'John'],
    'Age': [28, 24, 28, 24, 19],
    'City': ['New York', 'Los Angeles', 'New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)
```

 # check for duplicate entries

print(df.duplicated())

Output:

0 False

1 False

2 True

3 True

4 False

Remove Duplicate Entries:

drop_duplicates() method is used to remove the duplicates entries.

Ex:

```
import pandas as pd
# create dataframe
data = {
    'Name': ['John', 'Anna', 'John', 'Anna', 'John'],
    'Age': [28, 24, 28, 24, 19],
    'City': ['New York', 'Los Angeles', 'New York', 'Los Angeles', 'Chicago']
}
```

```
df = pd.DataFrame(data)
# remove duplicates
df.drop_duplicates(inplace=True)
print(df)
```

Output:

	Name	Age	City
0	**John**	**28**	**New York**
1	**Anna**	**24**	**Los Angeles**
4	**John**	**19**	**Chicago**

Overall Example:

```
import pandas as pd
# Sample dataset with duplicates
data = {
    'Name': ['Alice', 'Bob', 'Alice', 'David', 'Eve', 'Bob'],
    'Age': [25, 30, 25, 35, 28, 30],
    'City': ['New York', 'Los Angeles', 'New York', 'Chicago', 'Houston', 'Los Angeles']
}
# Creating DataFrame
df = pd.DataFrame(data)
print("Original DataFrame with duplicates:\n")
print(df)
```

```
# Detecting duplicate rows
print("\nDetecting duplicate rows (excluding first occurrence):")
print(df.duplicated())
# Detecting duplicate rows (excluding last occurrence):
print("\nDetecting duplicate rows (keep='last'):")
print(df.duplicated(keep='last'))
# Showing only duplicate rows
duplicates = df[df.duplicated()]
print("\nOnly duplicate rows:\n")
print(duplicates)
# Removing duplicates (keep first occurrence)
df_no_duplicates = df.drop_duplicates()
print("\nDataFrame after removing duplicates (keep='first'):\n")
print(df_no_duplicates)
# Removing duplicates based on specific column(s)
df_unique_names = df.drop_duplicates(subset=['Name'])
print("\nDataFrame after removing duplicates based on 'Name' column:\n")
print(df_unique_names)
```

Output:

Original DataFrame with duplicates:

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Alice	25	New York
3	David	35	Chicago
4	Eve	28	Houston
5	Bob	30	Los Angeles

Detecting duplicate rows (excluding first occurrence):

```
0    False
1    False
2     True
3    False
4    False
5     True
dtype: bool
```

Detecting duplicate rows (keep='last'):

```
0     True
1     True
2    False
3    False
4    False
5    False
dtype: bool
```

Only duplicate rows:

	Name	Age	City
2	Alice	25	New York
5	Bob	30	Los Angeles

DataFrame after removing duplicates (keep='first'):

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
3	David	35	Chicago
4	Eve	28	Houston

DataFrame after removing duplicates based on 'Name' column:

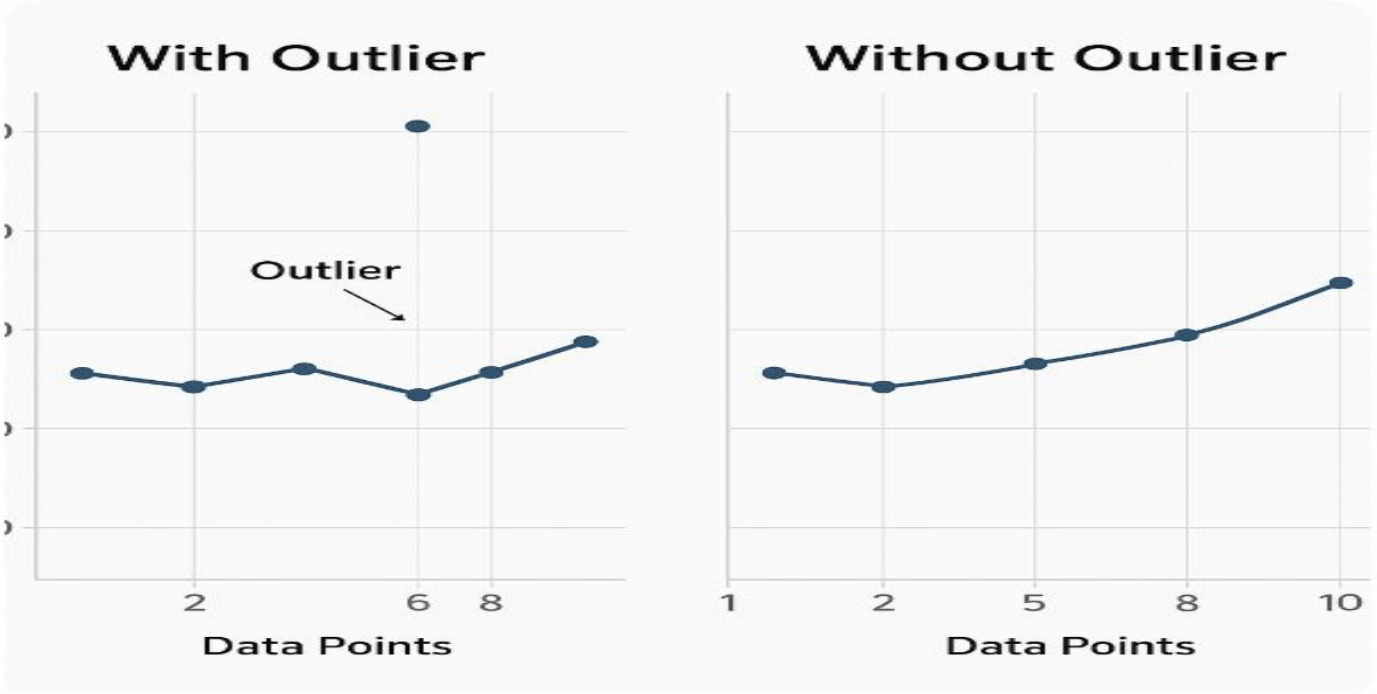
	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
3	David	35	Chicago
4	Eve	28	Houston

Outliers:

Outlier means a data point that is significantly higher or lower than the other values. These can mislead data analysis.

Example:

In a monthly sales dataset, if sales are usually between ₹50,000 and ₹60,000, but in one month sales are ₹5,00,000, then that month's sales is an **outlier.**



Why Removing Outliers:

Based on two factors Outliers removing.

1) Impact on Analysis
2) Statistical Significance

Main Causes of Outliers:

Outliers can arise from various sources

Data Entry Errors:

Simple human errors in entering data can create extreme values.

Measurement Error:

When a device is faulty or the experiment is disturbed, it can result in unusually high or low readings.

Experimental Errors:

If there are flaws in the experimental method, the data may not accurately reflect what is actually meant to be measured.

Data Processing Errors:

During data collection or processing, technical issues can lead to incorrect data.

How to identified Outliers:

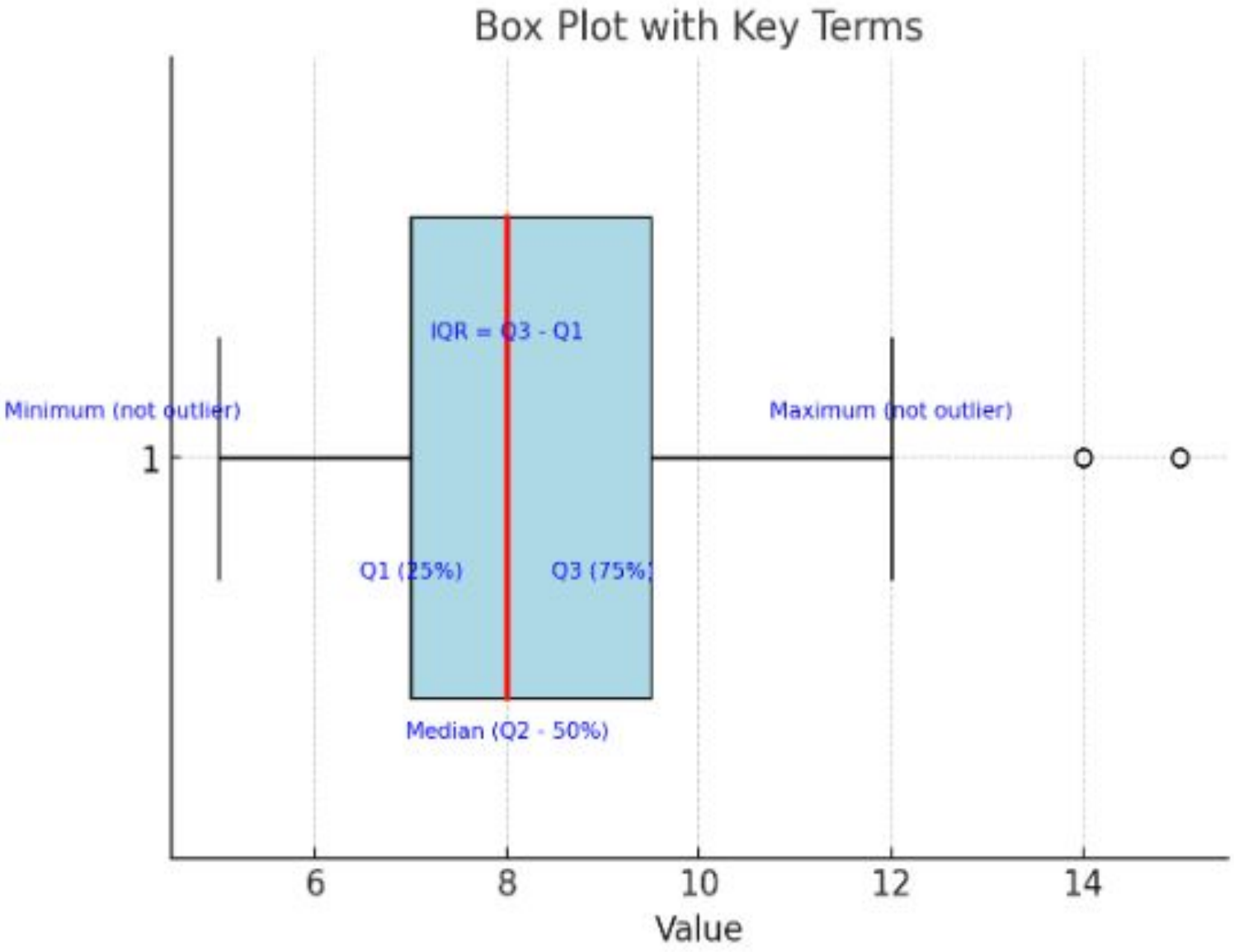
By identifying outliers, we can detect errors, unusual values, or useful information in the data. To identify outliers, graphs or charts are commonly used, as they show values that differ from the rest of the data. Graphs like box plots and scatter plots help easily identify outliers that differ from other values in the data.

Identifying outliers with box plots:

In Box Plots, we have different terminologies,

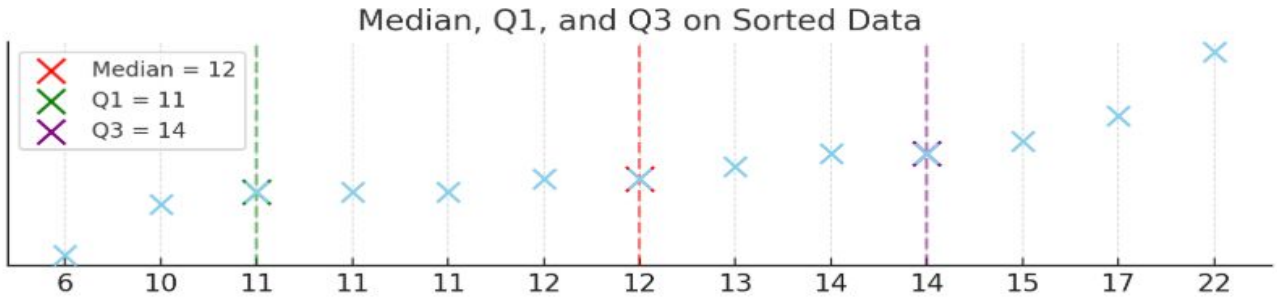
1. Minimum
2. First Quartile (Q1 - 25%)
3. Median (Q2 - 50%)
4. Third Quartile (Q3 - 75%)
5. Interquartile Range (IQR = Q3 - Q1)
6. Whiskers
7. Outliers

Minimum:
The smallest data value within the acceptable range (not an outlier).
First Quartile (Q1 - 25%):
25% of the data falls below this value. Start of the box.
Median (Q2 - 50%):
Middle value of the dataset; divides the data into two equal halves.
Third Quartile (Q3 - 75%)
75% of the data falls below this value. End of the box.
Interquartile Range (IQR = Q3 - Q1)
The range that covers the middle 50% of the data.
Whiskers
Lines extending from the box to show the range within $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$.
Outliers
Data points outside the whiskers; unusually high or low values.


Box Plot with Key Terms
IQR = Q3 - Q1
Minimum (not outlier)
Maximum (not outlier)
1
Q1 (25%)
Q3 (75%)
Median (Q2 - 50%)
6
8
10
12
14
Value

Ex :

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
# Sample dataset with some outliers
data = [10, 12, 11, 15, 11, 14, 13, 17, 12, 22, 14, 11, 6]
# Create the box plot
plt.figure(figsize=(8, 5))
sns.boxplot(data=data, color='skyblue')
# Add title and labels
plt.title("Box Plot Example", fontsize=14)
plt.xlabel("Sample Data")
plt.grid(True)
# Show the plot
plt.show()
```



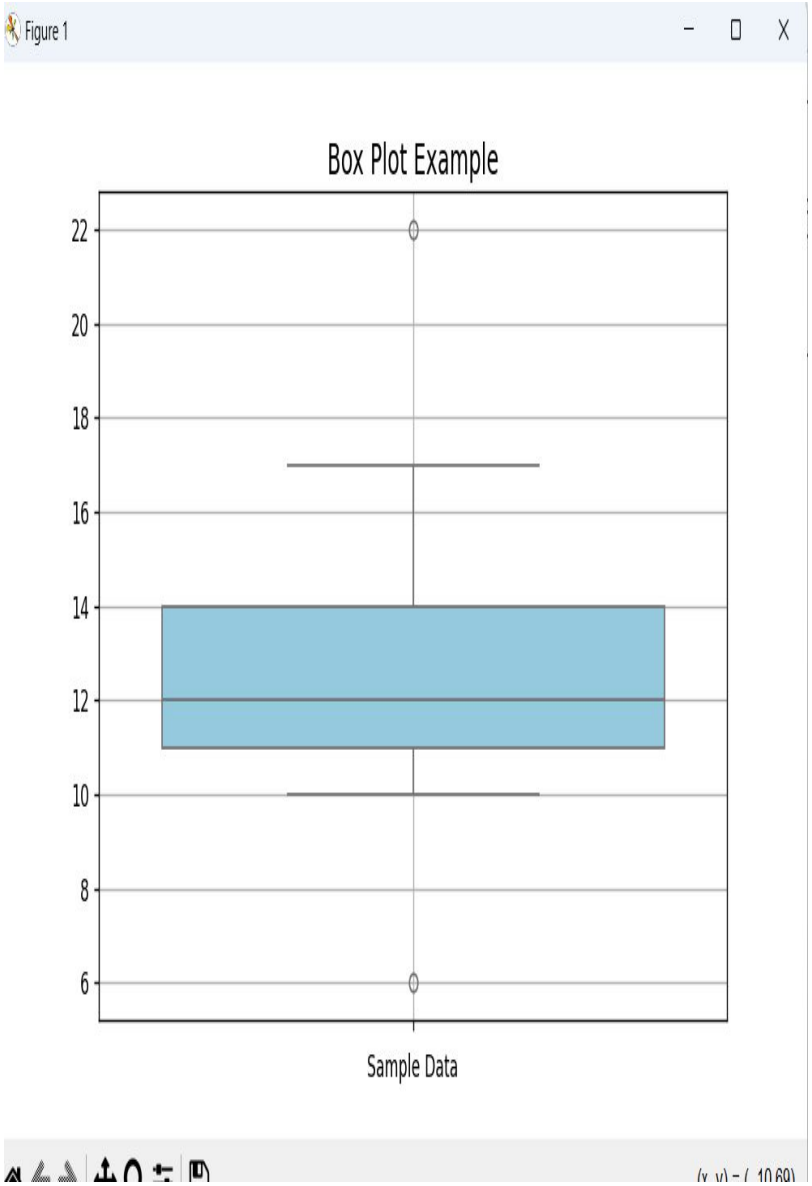
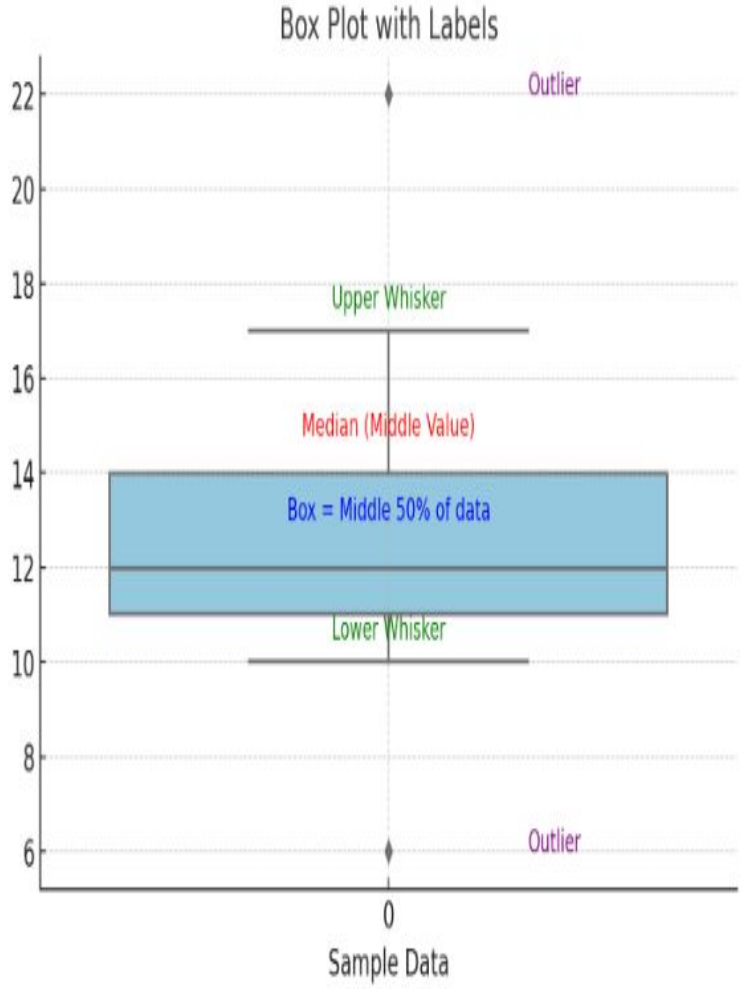


Figure 1
Box Plot Example
22
20
18
16
14
12
10
8
6
Sample Data


Box Plot with Labels
Outlier
Upper Whisker
Median (Middle Value)
Box = Middle 50% of data
Lower Whisker
Outlier
22
20
18
16
14
12
10
8
6
0
Sample Data

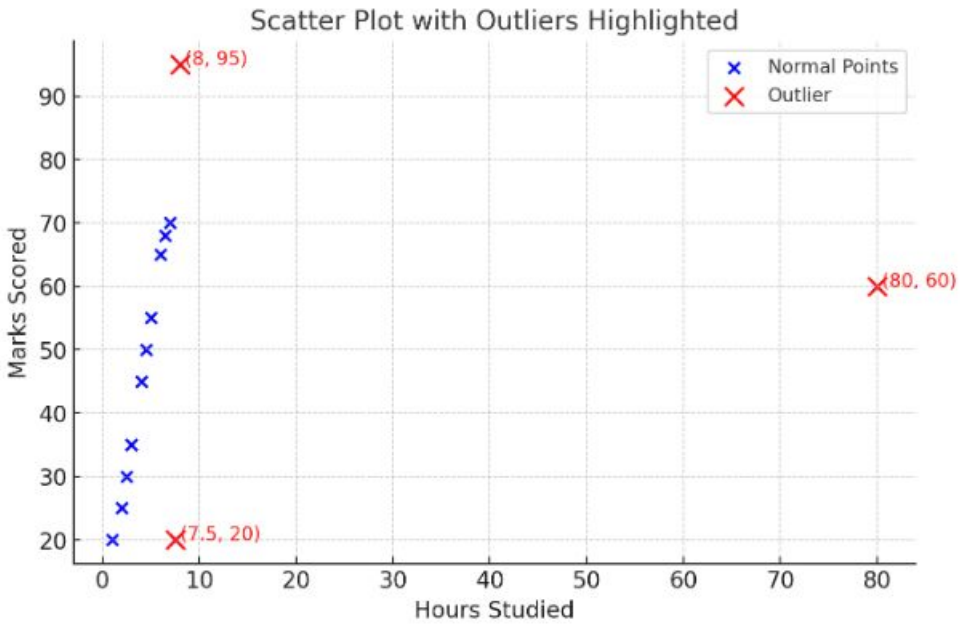
Identifying outliers with Scatter Plots

A scatter plot is a type of graph used to show the relationship between two numbers.

In this graph, each value is shown as a **dot**. One number is placed on the horizontal axis (x-axis), and the other number is placed on the vertical axis (y-axis).

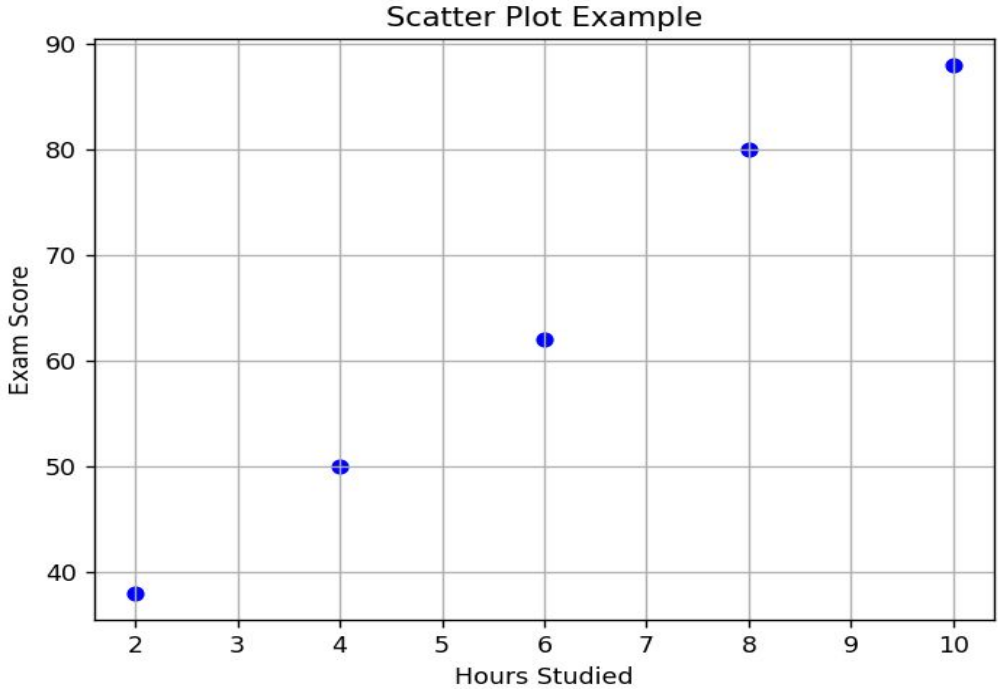
Usually, the data dots follow a certain pattern or direction. But if some dots are far away from that general pattern, they are called **outliers (unusual values)**.

X-axis → Independent variable
Y-axis → Dependent variable
Dots → Individual data points



Ex :

```
import matplotlib.pyplot as plt
x = [2, 4, 6, 8, 10]
y = [38, 50, 62, 80, 88]
plt.scatter(x, y, color='blue')
plt.xlabel("Hours Studied")
plt.ylabel("Exam Score")
plt.title("Scatter Plot Example")
plt.grid(True)
plt.show()
```



Anomalies:

An anomaly is an observation that clearly differs from the normal data. It often indicates a problem, an error, or an unusual event.

Ex:

In a dataset of server response times (in ms):

[120, 130, 125, 122, 128, 127, 121, 800]

The value **800 ms** is an **anomaly** — possibly due to a network delay or server overload.

Types of Anomalies

Point Anomalies: Individual data points that deviate significantly from the rest of the data. For example, detecting credit card fraud based on an unusually high 'amount spent'.

Contextual Anomalies: Depend on the surrounding context. In time-series data, what's normal at one time might be abnormal at another.

Collective Anomalies: A set of data points together indicate an anomaly. For example, unexpected data transfer activities might indicate a potential cyber attack.

Using Z-score Method:

$$\text{Z-score} = (x - \mu) / \sigma$$

Here x = individual data point

μ = mean of the dataset

σ = standard deviation of the dataset

Calculate the Mean & Calculate the Standard Deviation:

```
import numpy as np
data = [50, 52, 49, 51, 53, 500]
mean = np.mean(data)
std_dev = np.std(data)
print("Mean:", mean)
print("Standard Deviation:", std_dev)
```

Output:

```
Mean: 125.83333333333333
Standard Deviation: 167.33740034898224
```

pip install scipy

Example:

```
import numpy as np
from scipy.stats import zscore
# Sample data with one anomaly
data = [50, 52, 49, 51, 53, 500]
data_array = np.array(data)
# Instead of standard Z-score, use Modified Z-score (robust)
median = np.median(data_array)
mad = np.median(np.abs(data_array - median))  # Median Absolute Deviation
modified_z_scores = 0.6745 * (data_array - median) / mad
# Define threshold for anomaly
threshold = 3.5  # Commonly used for Modified Z-score
# Find anomalies
anomalies = data_array[np.abs(modified_z_scores) > threshold]
# Print results
print("Original Data:", data)
print("Modified Z-scores:", modified_z_scores)
print("Anomalies Detected:", anomalies)
```

Output:

```
================== RESTART: C:/Users/Asus-Pc/Desktop/Rubiya.py ================
Original Data: [50, 52, 49, 51, 53, 500]
Modified Z-scores: [ -0.6745      0.22483333  -1.12416667  -0.22483333   0.6745
 201.6755    ]
Anomalies Detected: [500]
```

Difference between outliers and anomalies:

An outlier is a value that is statistically much higher or lower compared to the other values in a dataset.

Ex:

Suppose the marks of students on a particular day are:

[45, 48, 50, 47, 49, 46, 98]

Here, **98** is an **outlier** because it is significantly higher than the rest of the marks.

An **anomaly** is a value or behavior that is **unusual based on context or circumstances**. It could indicate an error, fraud, or a rare event.

Ex:

A user usually spends less than **₹500** every day at **9 AM from India**.

One day, a transaction of **₹50,000** occurs at **2 AM from the USA**, when most people are asleep.

This is an **anomaly** because it shows a significant change in behavior. It could be a **fraudulent** transaction.

Key Difference Table

Feature	Outlier	Anomaly
Definition	Statistically far from other values	Contextually unusual event/value
Basis	Pure math/statistics	Behavior, meaning, or pattern
Context Needed?	No	Yes
Example	Marks: 98 when all others ~50	₹50,000 at 2 AM from another country
Use Case	Data cleaning, statistical modeling	Fraud detection, behavior monitoring

Encoding Categorical Variables:

In Python, encoding categorical variables means converting **non-numeric categorical data** into a **numeric format**.

We are using two variables:

1)Label Encoding.

2)One-Hot Encoding.

Label Encoding:

Label Encoding is a simple method used to convert **categorical data** into a **numerical format**.

Label Encoder transforms categories in **alphabetical order** by default:

'High' → 0

'Low' → 1

'Medium' → 2

Ex:

```
from sklearn.preprocessing import LabelEncoder
data = ['High', 'Low', 'Medium', 'Low', 'High']
le = LabelEncoder()
encoded_data = le.fit_transform(data)
print(encoded_data)
```

OUTPUT: 01210

Example 2:

```
from sklearn.preprocessing import LabelEncoder
# Sample categorical data
fruits = ['apple', 'banana', 'orange', 'banana', 'apple']
# Create a LabelEncoder object
encoder = LabelEncoder()
# Fit and transform the data
encoded_fruits = encoder.fit_transform(fruits)
# Display the results
print("Original categories:", fruits)
print("Encoded values:", encoded_fruits)
print("Class mapping:", list(encoder.classes_))
```

Output:

```
Original categories: ['apple', 'banana', 'orange', 'banana', 'apple']
Encoded values: [0 1 2 1 0]
Class mapping: ['apple', 'banana', 'orange']
```

One-Hot Encoding:

One-Hot Encoding is a technique used to convert **categorical data** (like names, colors, or labels) into a **numerical format** so that **machine learning algorithms** can understand it.

Machine learning models work with **numbers**, not text. But in real-world data, we often have categories like:

Colors: 'Red', 'Green', 'Blue'

Cities: 'Hyderabad', 'Mumbai', 'Delhi'

Brands: 'Nike', 'Adidas', 'Puma'

If we assign numbers directly (like Red=1, Green=2, Blue=3), the model might **misunderstand that 3 > 1**, which makes no sense for categories without order.

Example:

```
import pandas as pd

df = pd.DataFrame({'Color': ['Red', 'Green', 'Orange', 'Red', 'Green']})
onehot_encoded = pd.get_dummies(df, columns=['Color'])
print(onehot_encoded)
```

OUTPUT:

```
      Color_Green  Color_Orange  Color_Red
0           0             0           1
1           1             0           0
2           0             1           0
3           0             0           1
4           1             0           0
```

Data Transformation:
Data Transformation is the process of converting data from one format or structure into another for data analysis.

Why data transformation is important:
Data transformation is a very important step in data analysis and machine learning. It means changing raw (unprocessed) data into a clean and useful format so we can understand it better and build accurate models.

It helps in many ways, like:

Step	Before Transformation	After Transformation
1. Cleaning the data	`["apple", "apple", "aple"]`	`["apple"]`
2. Filling or removing missing values	`[10, 20, None, 30]`	`[10, 20, 25, 30]`
3. Changing the format of data	`"12-08-2025"`, `"2025/08/12"`	`"2025-08-12"`, `"2025-08-12"`
4. Scaling big or small numbers	`[5, 500, 10000]`	`[0.0, 0.05, 1.0]`
5. Encoding categories	`["Red", "Green", "Blue"]`	Label Encoding: `[0, 1, 2]` One-Hot Encoding: `[[1,0,0], [0,1,0], [0,0,1]]`

Without this step, the models may not give correct results. So, data transformation helps make the data ready for better analysis and decision-making.

Different Data Transformation Techniques:

We have different data transformation techniques used in python. They are

1) Renaming Columns
2) Changing Data Types or Data Types Conversion
3) Handling Missing Values
4) Scaling
5) Normalization

Renaming Columns:

Renaming columns is a way to change the names of columns in a dataset. It helps make the column names easier to understand and more meaningful.

Syntax:

DataFrame.rename (columns= {'old_name': 'new_name'}, inplace=True)

Here,

DataFrame.rename is a Method.

columns indicates dictionary with keys as old column names and values as new names

inplace=True Means updates the DataFrame directly

Ex:

```
import pandas as pd
# Sample DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})
print("Before Renaming:")
print(df)
```

 # Rename columns

```
df.rename (columns={'A': 'Math_Score', 'B': 'Science_Score'}, inplace=True)
print("\nAfter Renaming:")
print(df)
```

Output:

Before Renaming:

```
   A  B
0  1  4
1  2  5
2  3  6
```

After Renaming:

```
   Math_Score  Science_Score
0           1              4
1           2              5
2           3              6
```

Changing Data Types or Data Types Conversion:
Type Conversion (also called **data type casting**) means changing the data type of a column (e.g., from string to integer, float to string, etc.).

Common Data Types are **int, float, str, bool, datetime.**

Convert String to Integer in Python (Pandas)

In real-world datasets (like Excel, CSV), numbers are sometimes stored as **strings (text)** instead ofrealnumbers.

This can happen when:

1) Data is entered manually
2) It is read from a file where numbers are saved as text

Ex:

'25' → this is a string, not a number

```
import pandas as pd
# Step 1: Create a DataFrame with string numbers
df = pd.DataFrame({
    'Age': ['21', '22', '23']  # These look like numbers, but are strings
})
# Step 2: Check data type (optional)
print("Before Conversion:\n", df.dtypes)
# Step 3: Convert string to integer
df['Age'] = df['Age'].astype(int)
# Step 4: Check data type again
print("After Conversion:\n", df.dtypes)
```

Output:

Before Conversion:

```
Age    object     # object means string in pandas
After Conversion:
Age    int64      # Now it's properly converted to integer
```

Convert float to string:

Converting float to string means changing a number with decimal points into text.

Ex :

```
import pandas as pd
# Step 1: Create DataFrame with float values
df = pd.DataFrame({'Marks': [88.5, 92.0, 75.25] })

# Step 2: Check data types
print("Before Conversion:")
print(df.dtypes)

# Step 3: Convert float to string
df['Marks'] = df['Marks'].astype(str)

# Step 4: Check data types again
print("After Conversion:")
print(df.dtypes)
```

Output:

Before Conversion:

Marks float64

dtype : object

After Conversion:

Marks object

dtype : object

Handling Missing values:

Missing values are the empty spots in a dataset where the information is not given or is unknown.

In Pandas, missing values are usually represented as:

NaN (Not a Number)

We have different methods used to handling missing values. the methods are

1. Detect Missing Values
2. Remove Missing Values
3. Fill Missing Values
4. Check how many values are missing

Detect Missing Values:

Detecting missing values means checking where data is missing (i.e., where values are empty, null, or NaN) in a dataset.

Syntax:

df.isnull()

Ex:

```
import pandas as pd
df = pd.DataFrame({ 'Name': ['Alice', 'Bob', None], 'Score': [85, None, 90]})
print(df.isnull())
```

Remove Missing Values:

Remove missing values is the process of deleting parts of the data (rows or columns) that have no information, to clean the dataset.

Syntax:

Remove rows with missing values: df.dropna()

Ex :

```
import pandas as pd
df = pd.DataFrame({'Name': ['John', None, 'Sam'],'Marks': [85, 90, None]})
# Remove rows with any missing values
df_clean = df.dropna()
print(df_clean)
```

Output:

```
   Name  Marks
0  John   85.0
```

Remove columns with missing values: df.dropna(axis=1)

Ex:

```
import pandas as pd
# Create a DataFrame with missing values (NaN)
df = pd.DataFrame({ 'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, None, 30],
                    'Marks': [None, None, None]})
print("Original DataFrame:")
print(df)
# Remove columns with any missing values
df_clean = df.dropna(axis=1)

print("\nDataFrame after removing columns with missing values:")
print(df_clean)
```

Output:

```
Original DataFrame:
      Name      Age  Marks
0    Alice      25.0      None
1      Bob       NaN      None
2  Charlie      30.0      None
DataFrame after removing columns with missing values:
      Name
0    Alice
1      Bob
2  Charlie
```

Check how many values missing:

To understand how much data is missing in a dataset, you can count the number of missing (null/NaN) values in each column.

Syntax: df.isnull().sum()

Ex:

```
import pandas as pd
# Sample DataFrame with missing values
df = pd.DataFrame({'Name': ['Alice', 'Bob', None], 'Age': [25, None, 30],
                   'Marks': [85, None, None] })
# Count missing values column-wise
missing_counts = df.isnull().sum()
print("Missing values in each column:")
print(missing_counts)
```

Output:

```
Missing values in each column:
Name          1
Age           1
Marks         2
dtype     :       int64
```

Binning:
Binning is the process of converting **continuous numerical data** into **categorical groups** (called **bins** or **intervals**).those intervals with categories like "Low", "Medium", or "High".

It is also called **discretization**.
Syntax:
pd.cut(x, bins, labels=None, right=True, include_lowest=False)

Here,
x is a The column or list of numbers to bin (e.g., df['Marks'])
bins is a List of bin edges (ranges) or number of bins
labels are List of category names (optional)
right is If True, the bins include the right edge (default: True)
include_lowest is If True, the first bin includes the left edge

Ex:

```
import pandas as pd
df = pd.DataFrame({'Marks': [45, 67, 72, 88, 95, 50] })

# Define bin edges and labels
bins = [0, 50, 75, 100]
labels = ['Low', 'Medium', 'High']
# Apply binning
df['Grade'] = pd.cut(df['Marks'], bins=bins, labels=labels,
include_lowest=True)
print(df)
```

Output:

```
   Marks   Grade
0     45     Low
1     67  Medium
2     72  Medium
3     88    High
4     95    High
5     50     Low
```

Scaling is the process of converting numbers of different ranges into a **common range (like 0–1)** so that all features (columns) are treated equally during analysis or model training.

Example:
Before Scaling:
Age → 20 to 60
Salary → 20,000 to 1,00,000
These are in completely different ranges.
After scaling (0–1 range):
Age → 0.0, 0.25, 0.5, 0.75, 1.0
Salary → 0.0, 0.25, 0.5, 0.75, 1.0

How does Scaling work? (Min–Max Scaling)

Formula:

$$\text{Scaled Value} = \frac{\text{Value} - \text{Minimum}}{\text{Maximum} - \text{Minimum}}$$

Smallest number → becomes **0**

Largest number → becomes **1**

Middle numbers → come as fractions (0.25, 0.5, 0.75)

Age : 20,30,40,50,60

After Scaling:

$$20 \rightarrow \frac{20-20}{60-20} = \frac{0}{40} = 0.0$$

$$30 \rightarrow \frac{30-20}{40} = \frac{10}{40} = 0.25$$

$$40 \rightarrow \frac{40-20}{40} = \frac{20}{40} = 0.5$$

$$50 \rightarrow \frac{50-20}{40} = \frac{30}{40} = 0.75$$

$$60 \rightarrow \frac{60-20}{40} = \frac{40}{40} = 1.0$$

Salary :20,000, 40,000, 60,000, 80,000, 1,00,000

$$20{,}000 \rightarrow \frac{20000-20000}{100000-20000} = \frac{0}{80000} = 0.0$$

$$40{,}000 \rightarrow \frac{40000-20000}{80000} = \frac{20000}{80000} = 0.25$$

$$60{,}000 \rightarrow \frac{60000-20000}{80000} = \frac{40000}{80000} = 0.5$$

$$80{,}000 \rightarrow \frac{80000-20000}{80000} = \frac{60000}{80000} = 0.75$$

$$1{,}00{,}000 \rightarrow \frac{100000-20000}{80000} = \frac{80000}{80000} = 1.0$$

Program:

```
# Scaling Example in Python
from sklearn.preprocessing import MinMaxScaler
import pandas as pd
# Sample data
data = pd.DataFrame({
    'Age': [20, 30, 40, 50, 60],
    'Salary': [20000, 40000, 60000, 80000, 100000]})
print("Before Scaling:")
print(data)
# Create Min-Max Scaler
scaler = MinMaxScaler()
# Apply scaling
scaled_data = scaler.fit_transform(data)
# Convert scaled data back to DataFrame
scaled_df = pd.DataFrame(scaled_data, columns=['Age', 'Salary'])
print("\nAfter Scaling (0-1 range):")
print(scaled_df)
```

Output:

Before Scaling:

	Age	Salary
0	20	20000
1	30	40000
2	40	60000
3	50	80000
4	60	100000

After Scaling (0-1 range):

	Age	Salary
0	0.00	0.00
1	0.25	0.25
2	0.50	0.50
3	0.75	0.75
4	1.00	1.00

Types Scaling:

The different types are scaling are

1)Min-Max Scaling (Normalization)

2)Standardization (Z-score Scaling)

Min-Max Scaling:

Min-Max Scaling means adjusting all numbers so the smallest value becomes 0, the largest becomes 1, and all others fall in between**.** Min-Max Scaling also called Normalization.

Xscaled= X−Xmin/ X-max−Xmin

Here,

X = original value

Xmin = smallest value in the column

Xmax = largest value in the column

Xscaled= value after scaling (between 0 and 1)

Ex:

```
# Min-Max Scaling Example in Python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
# Sample data
data = pd.DataFrame({
    'Age': [20, 30, 40, 50, 60],
    'Salary': [20000, 40000, 60000, 80000, 100000]})
print("Before Scaling:")
print(data)
# ---------- Manual Min-Max Scaling ----------
def min_max_scale(column):
    min_val = column.min()     # find minimum
    max_val = column.max()    # find maximum
    return (column - min_val) / (max_val - min_val)
# Apply manual scaling
data['Age_Manual'] = min_max_scale(data['Age'])
data['Salary_Manual'] = min_max_scale(data['Salary'])
```

```
# ---------- Using sklearn MinMaxScaler ----------
scaler = MinMaxScaler()
scaled_values = scaler.fit_transform(data[['Age', 'Salary']])
scaled_df = pd.DataFrame(scaled_values,
columns=['Age_Sklearn', 'Salary_Sklearn'])

# Combine both results
final_df = pd.concat([data, scaled_df], axis=1)
print("\nAfter Min-Max Scaling (0 to 1):")
print(final_df)
```

Output:

Before Scaling:

	Age	Salary
0	20	20000
1	30	40000
2	40	60000
3	50	80000
4	60	100000

After Min-Max Scaling (0 to 1):

	Age	Salary	Age_Manual	Salary_Manual	Age_Sklearn	Salary_Sklearn
0	20	20000	0.00	0.00	0.00	0.00
1	30	40000	0.25	0.25	0.25	0.25
2	40	60000	0.50	0.50	0.50	0.50
3	50	80000	0.75	0.75	0.75	0.75
4	60	100000	1.00	1.00	1.00	1.00

Feature	Manual Scaling	Sklearn MinMaxScaler
Who writes formula?	You (custom function)	Already built-in
Easy for learning?	Yes (you understand formula clearly)	Yes (just use function, no math)
For small dataset	Works fine	Works fine
For big dataset	Slow, need more code	Very fast, optimized
Flexibility	Only what you code	Many options (like changing range from 0–1 to -1–1, etc.)

Standardization:

Standardization means **changing numbers so that**:

The **average (mean)** of the numbers becomes **0**

The **spread of numbers** (standard deviation) becomes **1**

This makes all features use the **same scale**, so they can be compared fairly even if they were originally in different units or ranges.

Example:

Suppose we have **heights (in cm)**:

160, 170, 180, 190, 200

Step 1 – Find Mean and Standard Deviation

Mean (μ) = (160 + 170 + 180 + 190 + 200) / 5 = **180**

Standard Deviation (σ) = **15.81** (approx)

✅ Formula of Standard Deviation (σ)

$$\sigma = \sqrt{\frac{\sum (X - \mu)^2}{N}}$$

- X = each value
- μ = mean
- N = number of values

✅ Step 1: Subtract mean from each value

- 160 – 180 = -20
- 170 – 180 = -10
- 180 – 180 = 0
- 190 – 180 = +10
- 200 – 180 = +20

✅ Step 2: Square each result

- $(-20)^2 = 400$
- $(-10)^2 = 100$
- $(0)^2 = 0$
- $(10)^2 = 100$
- $(20)^2 = 400$

✅ Step 3: Find average of these squares

$$\frac{400 + 100 + 0 + 100 + 400}{5} = \frac{1000}{5} = 200$$

✅ Step 4: Square root

$$\sigma = \sqrt{200} \approx 14.14$$

In my earlier example, I wrote **15.81** (approx),
but actually for this dataset, the correct
Standard Deviation = 14.14 (if we use population formula).

If we use **sample formula** (divide by N-1 instead of N), then:

$$\sigma = \sqrt{\frac{1000}{4}} = \sqrt{250} \approx 15.81$$

Difference
Population SD (divide by N) → 14.14
Sample SD (divide by N-1) → 15.81

In most statistics & machine learning, we use **sample SD**, that's why it became **15.81**.

Step 2 – Apply Standardization Formula

$z = x - \mu / \sigma$

Height (x)	Calculation	z-score
160	(160 − 180) / 15.81 = -1.26	-1.26
170	(170 − 180) / 15.81 = -0.63	-0.63
180	(180 − 180) / 15.81 = 0.00	0.00
190	(190 − 180) / 15.81 = 0.63	0.63
200	(200 − 180) / 15.81 = 1.26	1.26

Result after Standardization:

-1.26, -0.63, 0.00, 0.63, 1.26

Program:

```
import numpy as np
from sklearn.preprocessing import StandardScaler

# Original heights in cm
heights = np.array([[160], [170], [180], [190], [200]])

# Create the StandardScaler object
scaler = StandardScaler()

# Fit and transform the data
standardized_heights = scaler.fit_transform(heights)

# Display results
print("Original Heights:\n", heights.flatten())
      print("Standardized Heights:\n",
np.round(standardized_heights.flatten(), 2))
```

Output:

Original Heights:

[160 170 180 190 200]

Standardized Heights:

[-1.26 -0.63 0.   0.63 1.26]

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Sample Data
data = pd.DataFrame({
    'Height': [160, 170, 180, 190, 200]
})
print("Original Data:")
print(data)

# ---------- Min-Max Scaling ----------
minmax = MinMaxScaler()
data['Height_MinMax'] = minmax.fit_transform(data[['Height']])

# ---------- Standardization ----------
standard = StandardScaler()
data['Height_Standardized'] = standard.fit_transform(data[['Height']])

print("\nAfter Scaling:")
print(data)
```

Original Data:

	Height
0	160
1	170
2	180
3	190
4	200

After Scaling:

	Height	Height_MinMax	Height_Standardized
0	160	0.00	-1.26
1	170	0.25	-0.63
2	180	0.50	0.00
3	190	0.75	0.63
4	200	1.00	1.26

Normalization:

In data transformation, ***normalization*** means rescaling numerical values to a fixed range, usually **0 to 1** (sometimes -1 to 1).
It's also called **min-max scaling**.

Feature	**Normalization (Min–Max Scaling)**	**Standardization (Z-score Scaling)**
Definition	Rescales values into a fixed range (0–1 or -1–1)	Rescales values so that mean = 0 and standard deviation = 1
Formula	$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$	$X' = \frac{X - \mu}{\sigma}$
Output Range	Always between **0–1** (or -1–1 if chosen)	Values have **no fixed range** (can be -2, +3, etc.)
When to Use	When you need data in a **fixed scale** (e.g., neural networks, distance-based models like KNN)	When data has **different distributions** or outliers, and you want to **compare fairly**
Example	Marks [20, 40, 60, 80, 100] → [0.0, 0.25, 0.5, 0.75, 1.0]	Heights [160, 170, 180, 190, 200] → [-1.26, -0.63, 0, 0.63, 1.26]

UNIT-III

UNIVARIATE AND BIVARIATE ANALYSIS

Univariate Analysis:

Univariate analysis is the process of analyzing and describing a **single variable** in a dataset to understand its characteristics, such as distribution, central value (mean, median, mode), and spread (range, variance, standard deviation).

Here,

Uni means **one or Single**

Variate means **Variable**

Types of Univariate analysis:

Univariate analysis divided into two types

1. For Numerical (continuous) data
2. For Categorical data

For Numerical Data:

In Numerical Data, we have four components of Univariate analysis.

1. Measures of Central Tendency
2. Measures of Dispersion
3. Distribution Shape
4. Visualization

Measures of Central Tendency:

Measures of Central Tendency are statistical methods that describe the center point or the typical value of a dataset.

Here we have 3 measures

1. Mean (Average Value)
2. Median (Middle Value)
3. Mode (Most frequent value)

Mean:

Mean is the arithmetic average of a set of numbers. It is calculated by adding all the values and then dividing by the total number of values**.**

Mean = Sum of all Values / Number of Values.

Ex:

Marks of 5 students: 10, 20, 30, 40, 50

- Sum = 10 + 20 + 30 + 40 + 50 = 150
- Number of students = 5
- **Mean = 150 ÷ 5 = 30**

Median:

Median is the middle value of an ordered dataset (arranged in ascending or descending order).

- If the number of observations is **odd**, the median is the middle value.
- If the number of observations is **even**, the median is the average of the two middle values.

Ex:

1. **Odd number of values**
   Data: [10, 20, 30, 40, 50] (already in order)

- Middle value = 30
- Median = 30

2. **Even number of values**
   Data: [10, 20, 30, 40, 50, 60]

- Two middle values = 30 and 40
- Median = (30 + 40) ÷ 2 = 35
- Median = 35

Mode:

Mode is a measure of central tendency that represents the value which occurs most frequently in a dataset.

Ex:

1. Data: [10, 20, 20, 30, 40]

- Here, **20** appear twice (more than others).
- Mode = 20

2. Data: [5, 6, 7, 7, 8, 8, 9]

- Here, both **7 and 8** appear twice.
- This dataset is **bimodal** (two modes).

3. Data: [2, 4, 6, 8]

- All values occur only once.
- No mode.

Program:

```
import statistics as stats

# Sample data

data = [10, 20, 20, 30, 40, 50, 60]

# Mean

mean_value = stats.mean(data)

# Median

median_value = stats.median(data)

# Mode

try:

    mode_value = stats.mode(data)   # works if a clear mode exists

except stats.StatisticsError:
```

```
  mode_value = "No unique mode"

# Printing results

print("Data:", data)

print("Mean:", mean_value)

print("Median:", median_value)

print("Mode:", mode_value)
```

Output:

Data: [10, 20, 20, 30, 40, 50, 60]

Mean: 32.857142857142854

Median: 30

Mode: 20

Measures of Dispersion:

Dispersion means how spread out the data values are.

- If the numbers are **close together**, dispersion is **small**.
- If the numbers are **far apart**, dispersion is **large**.

Ex:

Marks = [48, 49, 50, 51, 52]

All marks are close → **small dispersion**

Marks = [10, 40, 70, 90]

Marks are very far apart → **large dispersion**

Common Measures of Dispersion:

The common measures of Dispersion are

1. Range
2. Variance
3. Standard Deviation (SD)
4. Quartile Deviation / IQR

Range:

It is defined as the difference between the highest value and the lowest value in a dataset.

Range = Maximum – Minimum

Ex:

Marks of students = [15, 25, 30, 35, 50]

- Maximum = 50
- Minimum = 15
- **Range = 50 – 15 = 35**

So, the range of marks = **35**

Variance:

It is the average of the squared differences between each value and the mean.

Ex:

Data = [2, 4, 6, 8]

1. Mean = (2 + 4 + 6 + 8) ÷ 4 = 20 ÷ 4 = **5**
2. Differences from mean = [2–5, 4–5, 6–5, 8–5] = [–3, –1, 1, 3]
3. Squared differences = [9, 1, 1, 9]

4. Average of squared differences = (9 + 1 + 1 + 9) ÷ 4 = 20 ÷ 4 = **5**

Variance = **5**

Standard Deviation:

Standard Deviation is the square root of variance and shows how much the data values differ from the mean.

Ex:

Data = [2, 4, 6, 8]

1. Mean = (2 + 4 + 6 + 8) ÷ 4 = **5**
2. Differences from mean = [–3, –1, 1, 3]
3. Squared differences = [9, 1, 1, 9]
4. Variance = (9 + 1 + 1 + 9) ÷ 4 = 20 ÷ 4 = **5**
5. **Standard Deviation = √5 ≈ 2.24**

Quartile Deviation/IQR:

Quartile Deviation is half of the difference between the third quartile and the first quartile.

$$\mathbf{QD = Q3\text{-}Q1/2}$$

Where:

- Q1= First Quartile (25% of data)
- Q3= Third Quartile (75% of data)

Ex:

Data = [10, 20, 30, 40, 50, 60, 70, 80]

1. Q1 = 25th percentile = 20
2. Q3= 75th percentile = 60
3. Quartile Deviation = (Q3–Q1)/2=(60–20)/2=40/2=20

4. So, Quartile Deviation = **20**

Program:

```
import numpy as np

# Sample data

data = [10, 20, 30, 40, 50, 60, 70, 80]

# 1. Range

range_val = max(data) - min(data)

# 2. Variance

variance_val = np.var(data)

# 3. Standard Deviation

std_dev = np.std(data)

# 4. Quartile Deviation

Q1 = np.percentile(data, 25)   # First Quartile

Q3 = np.percentile(data, 75)   # Third Quartile

quartile_deviation = (Q3 - Q1) / 2

# Display results

print("Data:", data)

print("Range:", range_val)

print("Variance:", variance_val)
```

```
print("Standard Deviation:", std_dev)

print("Quartile Deviation:", quartile_deviation)
```

Output:

Data: [10, 20, 30, 40, 50, 60, 70, 80]

Range: 70

Variance: 525.0

Standard Deviation: 22.9128784747792

Quartile Deviation: 17.5

Distribution Shape:

Distribution shape describes how data is spread — whether it is symmetrical (normal), skewed to left or right, or peaked/flat.

Visualization:

Visualization means showing data as pictures (like graphs or charts) so that we can understand it easily instead of only seeing numbers.

The common visualization techniques are

1. Histogram
2. Box plots
3. KDE (Kernel Density Estimation)

Histogram:

A histogram is a special type of bar graph used for numerical data**.**

First, the data is split into ranges of values (these are called **bins**).

Then we count how many numbers fall inside each bin**.**

Finally, we draw bars →

- **X-axis (horizontal)** shows the ranges (bins).
- **Y-axis (vertical)** shows the count (frequency).

The height of each bar tells us how many values are in that range**.**

Ex:

Student marks:

[25, 35, 40, 42, 45, 48, 50, 52, 55, 60]

Let's make bins of size 10 marks:

- **20–30** → 1 student (25)
- **30–40** → 1 student (35)
- **40–50** → 3 students (40, 42, 45, 48) → actually 4 students
- **50–60** → 4 students (50, 52, 55, 60)

So the histogram bars will look like this:

- 20–30 → bar of height **1**
- 30–40 → bar of height **1**
- 40–50 → bar of height **4**
- 50–60 → bar of height **4**

Program:

```
import matplotlib.pyplot as plt

# Student marks data

marks = [25, 35, 40, 42, 45, 48, 50, 52, 55, 60]
```

 # Create histogram

```
plt.hist(marks, bins=[20, 30, 40, 50, 60], edgecolor='black')
```

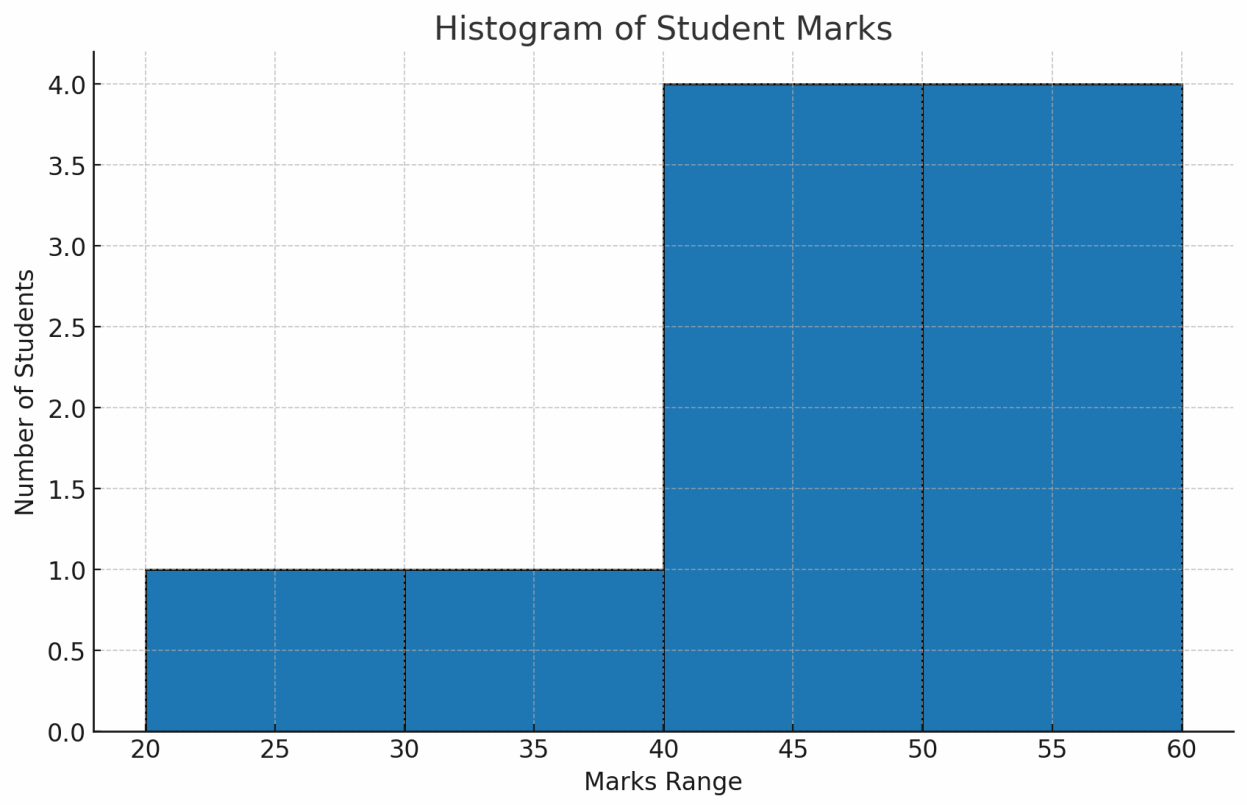
 # Add labels and title

```
plt.xlabel("Marks Range")

plt.ylabel("Number of Students")

plt.title("Histogram of Student Marks")
```

 # Show the graph

```
plt.show()
```

Output:



Box Plots:

A **box plot** (also called whisker plot) is a graph that shows how data is spread out.

In Box Plots, we have different terminologies,

1. Minimum
2. First Quartile (Q1 - 25%)
3. Median (Q2 - 50%)
4. Third Quartile (Q3 - 75%)
5. Interquartile Range (IQR = Q3 - Q1)
6. Whiskers
7. Outliers

Minimum:

The smallest data value within the acceptable range (not an outlier).

First Quartile (Q1 - 25%):

25% of the data falls below this value. Start of the box.

Median (Q2 - 50%):

Middle value of the dataset; divides the data into two equal halves.

Third Quartile (Q3 - 75%)

75% of the data falls below this value. End of the box.

Interquartile Range (IQR = Q3 - Q1)

The range that covers the middle 50% of the data.

Whiskers

Lines extending from the box to show the range within Q1 − 1.5×IQR and Q3 + 1.5×IQR.

Outliers

Data points outside the whiskers; unusually high or low values.

Ex:

 # Import required library

```
import matplotlib.pyplot as plt
```

 # Sample data

```
data = [2, 4, 5, 7, 8, 10, 12, 15, 18]
```

 # Create boxplot

```
plt.boxplot(data)
```

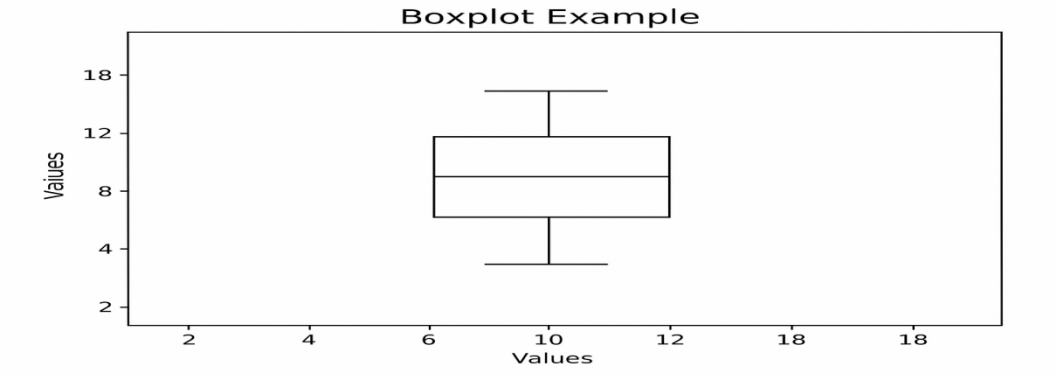
 # Add title and labels

```
plt.title("Boxplot Example")

plt.ylabel("Values")
```

 # Show plot

```
plt.show()
```

Output :



KDE:

KDE Means Kernel Density Estimation. KDE (Kernel Density Estimation) is a way to see how your data is spread out. Instead of assuming it follows a common pattern (like a bell curve), it draws a smooth curve that shows where the data points are concentrated and where they are sparse.

It smooths out the data points to show where data is concentrated.

The "kernel" is a function (usually Gaussian) that spreads each data point into a smooth curve, and combining all curves gives the overall density.

Ex:

Suppose we have exam scores:

data = [50, 60, 65, 70, 80]

Step 1: Each score gets a small smooth bump (kernel).
Step 2: Add all bumps → smooth curve.
Step 3: The curve shows **density**:

- Tall peaks → many data points nearby
- Short peaks → isolated points

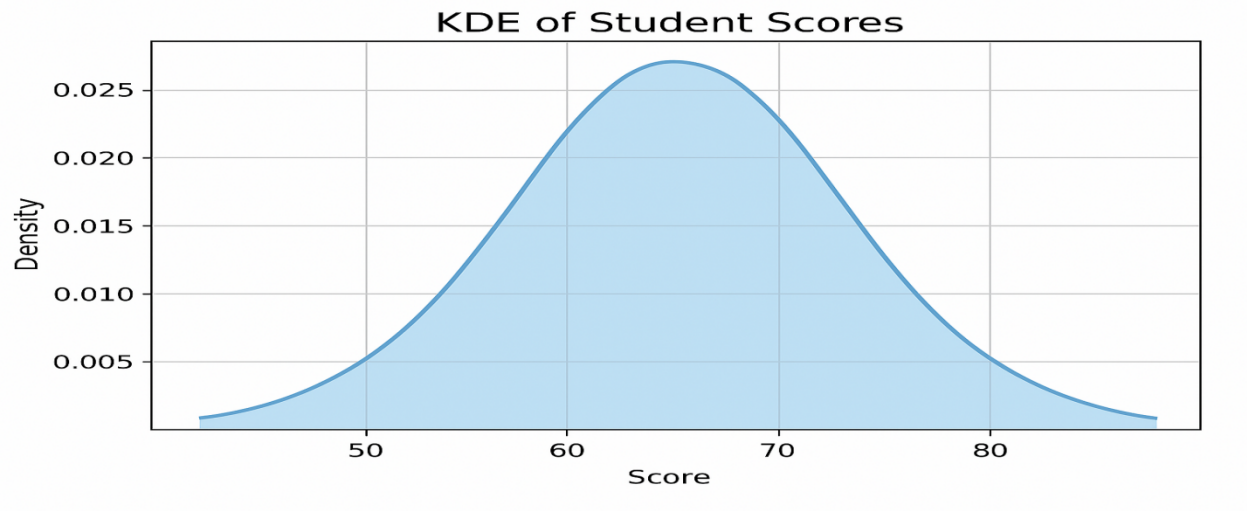
Program:

```
import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt

# Small dataset

data = np.array([50, 60, 65, 70, 80], dtype=float)
```

 # Plot KDE with adjusted bandwidth

```
plt.figure(figsize=(6,4))

sns.kdeplot(data, bw_adjust=0.5, fill=True, color='skyblue')

plt.title("KDE of Student Scores")

plt.xlabel("Score")

plt.ylabel("Density")

plt.grid(True)

plt.show()
```

Output:



For Categorical data:

Categorical data in univariate analysis means checking which categories are there and how often they happen, and then showing it with counts, percentages, or simple graphs.

Ex:

Suppose we collect people's favorite colors:
["Red", "Blue", "Green", "Red", "Red", "Blue"]

This is categorical data (because values are labels).

Components of Univariate Analysis for Categorical Data

Categories (Levels / Groups)

- The unique values in the data.
- Example: Gender → Male, Female, Other.

Frequency (Counts)

- Number of times each category occurs.
- Example: Male = 40, Female = 60.

Relative Frequency / Percentages

- Proportion of each category compared to total.
- Example: Male = 40%, Female = 60%.

Mode (Most Common Category)

- The category that occurs the most.
- Example: "Female" is mode if Female = 60%.

Visualization

- Graphical representation of distribution.

Visualization Techniques:

Different Visualization Techniques used for Categorical Data in Univariate analysis

1. Bar Chart
2. Pie Chart
3. Count Plot

Bar Chart:

A bar chart is a graph that shows categorical data using rectangular bars.

- Each bar represents a category.
- The height (or length) of the bar shows the frequency (count) or percentage of that category.
- Bars can be **vertical** (upwards) or **horizontal** (sideways).
- All bars have the same width but different heights/lengths.
- **X-axis (horizontal axis)** → categories (like fruits, subjects, colors).
- **Y-axis (vertical axis)** → frequency or percentage.
- Bars are separated by gaps

Ex:

A teacher asked **10 students** about their **favorite sport**. The results are:

- Cricket → 4 students
- Football → 3 students
- Basketball → 2 students
- Tennis → 1 student

Step 1: Frequency Table

Sport	Number of Students
Cricket	4
Football	3
Basketball	2
Tennis	1

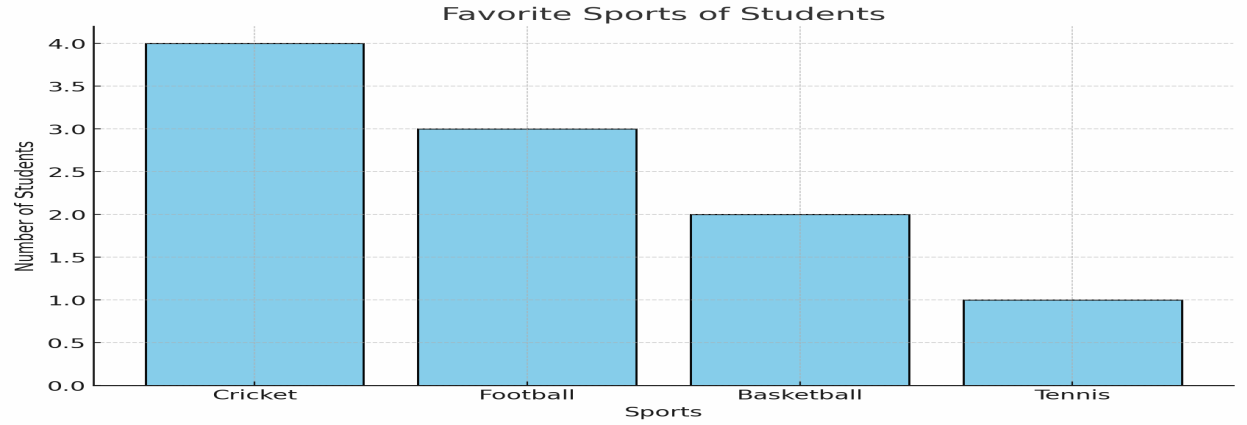
Step 2: Bar Chart Representation

- **X-axis (horizontal line)** → Sports (Cricket, Football, Basketball, Tennis)
- **Y-axis (vertical line)** → Number of Students (1 to 4)
- Draw bars:
    - Cricket bar = height 4
    - Football bar = height 3
    - Basketball bar = height 2
    - Tennis bar = height 1

Code:

```
import matplotlib.pyplot as plt

# Data

sports = ["Cricket", "Football", "Basketball", "Tennis"]

students = [4, 3, 2, 1]

# Create bar chart

plt.bar(sports, students, color="skyblue", edgecolor="black")

# Add labels and title

plt.title("Favorite Sports of Students")

plt.xlabel("Sports")

plt.ylabel("Number of Students")

# Show the chart

plt.show()
```

Output:



Pie Chart:

A pie chart is a circular diagram that shows data as slices of a circle, just like cutting a pizza or cake into pieces**.**

Each slice = one category**.**

The size (angle) of the slice depends on how much that category contributes to the total (percentage).

The whole circle = 100% of the data.

The circle is divided into slices (sectors)**.**Each slice represents a **category** of data.

The angle of each slice is calculated as:

Slice Angle = Category Value/Total Value * 360

Ex:

A teacher asked 6 students their **favorite fruit**:

- Apple → 3 students
- Mango → 2 students
- Banana → 1 student

Total = 6 students

Step 1: Find Percentages

- Apple = (3 ÷ 6) × 100 = **50%**
- Mango = (2 ÷ 6) × 100 = **33.3%**

- Banana = (1 ÷ 6) × 100 = **16.7%**

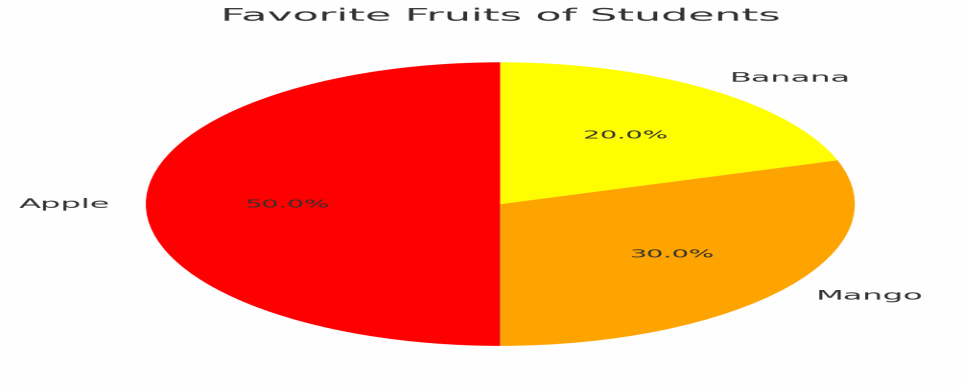
Step 2: Find Slice Angles

- Apple = (3 ÷ 6) × 360 = **180°**
- Mango = (2 ÷ 6) × 360 = **120°**
- Banana = (1 ÷ 6) × 360 = **60°**

Code:

```
import matplotlib.pyplot as plt


# Data

fruits = ["Apple", "Mango", "Banana"]

students = [5, 3, 2]   # Number of students

# Create Pie Chart

plt.pie (students, labels=fruits, autopct="%1.1f%%", startangle=90, colors=["red", "orange", "yellow"])

# Add Title

plt.title("Favorite Fruits of Students")

# Show the chart

plt.show()
```

Output:



Count Plot:

A Count Plot is a graph that shows the count (frequency) of each category in categorical data.

- It uses bars like a bar chart.
- The height of each bar tells how many times that category appears in the data.
- Works only for categorical variables**.**
- Automatically counts how many times each category appears.
- Similar to a bar chart, but you don't need to calculate counts yourself (the function does it).
- Very useful in exploratory data analysis (EDA)**.**

Ex:

Suppose a teacher asked **6 students** their favorite fruit:

- Apple → 3 students
- Mango → 2 students
- Banana → 1 student

In a **Count Plot**:

- Apple bar height = 3
- Mango bar height = 2
- Banana bar height = 1

Code:

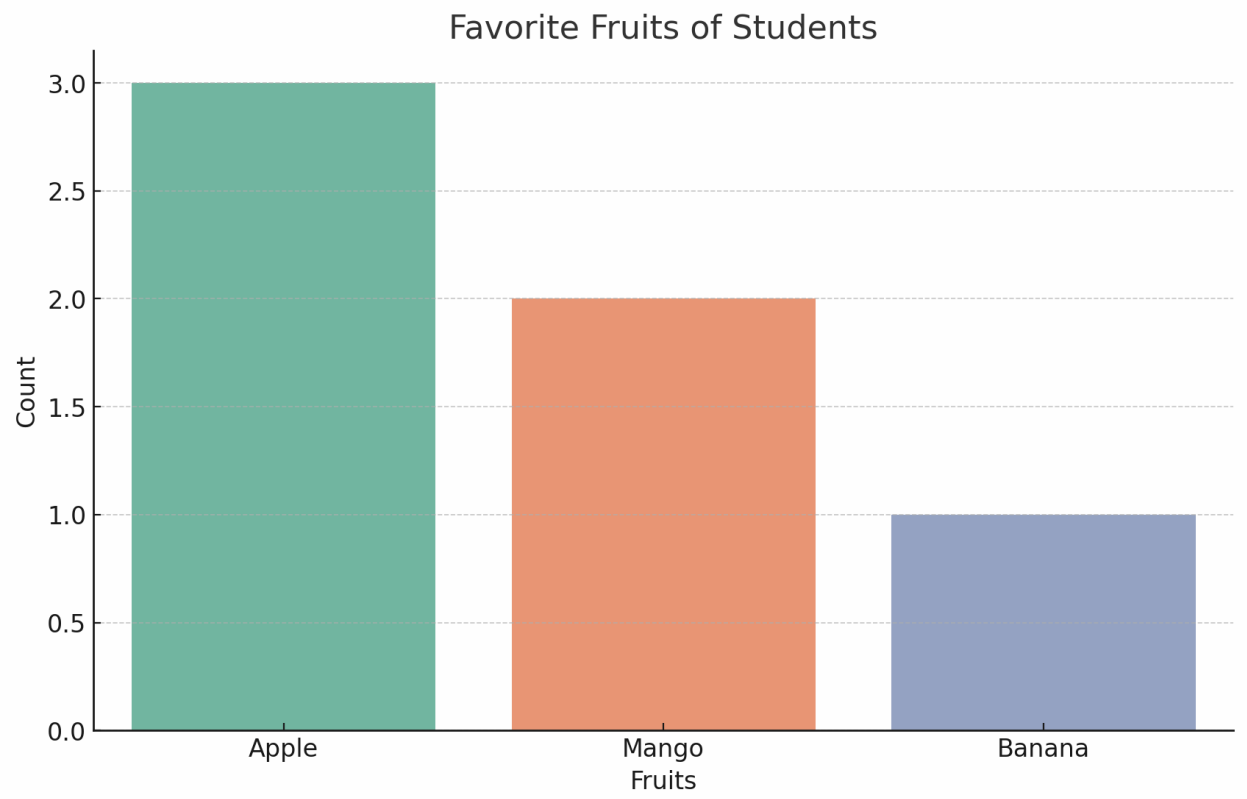
```
import seaborn as sns

import matplotlib.pyplot as plt

# Data

fruits = ["Apple", "Mango", "Apple", "Banana", "Apple", "Mango"]

# Create Count Plot

sns.countplot(x=fruits, palette="Set2")

# Add title

plt.title("Favorite Fruits of Students")

plt.xlabel("Fruits")
```

plt.ylabel("Count")

 # Show

plt.show()

Output:



Bivariate Analysis:

Bivariate analysis means analyzing the relationship between **two variables**.

Here,

"Bi" = two, "variate" = variables.

The Analysis depends on the type of variables. The types of variables are

1. Numerical – Numerical
2. Categorical – Categorical

Numerical – Numerical:

It means comparing two number-based variables (like height & weight, marks & study hours) to find patterns, correlations, or trends.

Common Methods to analyze Numerical-Numerical Data:

1. **Scatter Plots**
2. **Pair Plots**
3. **Heatmaps**
4. **Correlation**

Scatter Plots:

A scatter plot is a type of graph that shows the relationship between two numerical variables by plotting them as points (dots) on a chart.

Here,

We are using X-axis (one Variable) and Y-axis (Another Variable). X-axis also called as Horizontal and Y-axis also called as Vertical.

Why do we use:

- To check whether two numerical variables are related**.**
- To see if the relationship is positive, negative, or no relation**.**
- To detect outliers (unusual data points).

Ex:

Suppose we have 5 students' data:

Student	**Study Hours**	**Marks**
A	2	40
B	4	55
C	5	60
D	6	70

E 8 85

When we plot:

X-axis → Study Hours

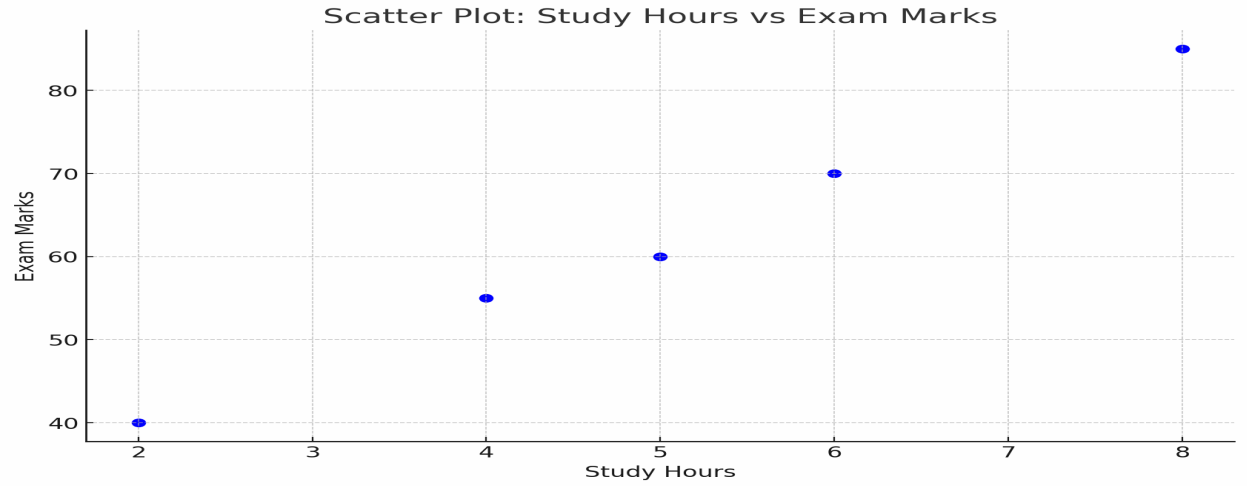
Y-axis → Marks

We get points like (2, 40), (4, 55), (5, 60), (6, 70), (8, 85).

Program:

```
import matplotlib.pyplot as plt
# Example data (Study Hours vs Marks)
study_hours = [2, 4, 5, 6, 8]
marks = [40, 55, 60, 70, 85]
# Create scatter plot
plt.scatter(study_hours, marks, color='blue', marker='o')
# Add labels and title
plt.xlabel("Study Hours")
plt.ylabel("Exam Marks")
plt.title("Scatter Plot: Study Hours vs Exam Marks")
# Add grid for better readability
plt.grid(True)
# Show plot
plt.show()
```

Output:



Pair Plots:

A pair plot (scatterplot matrix) is a collection of small plots that shows the relationships between all pairs of numerical variables in a dataset.

- If you have 3 or more numerical columns, the pair plot draws a scatter plot for every possible pair**.**
- On the diagonal, it shows the distribution (histogram or KDE) of each individual variable.
- Optionally, points can be colored by category to spot clusters or groups.

Why is it useful:

- To quickly explore how all variables relate to each other.
- To detect patterns, correlations, clusters, or outliers**.**
- To understand feature relationships before applying Machine Learning.

Example (Iris dataset):

Suppose we have 4 numerical variables:

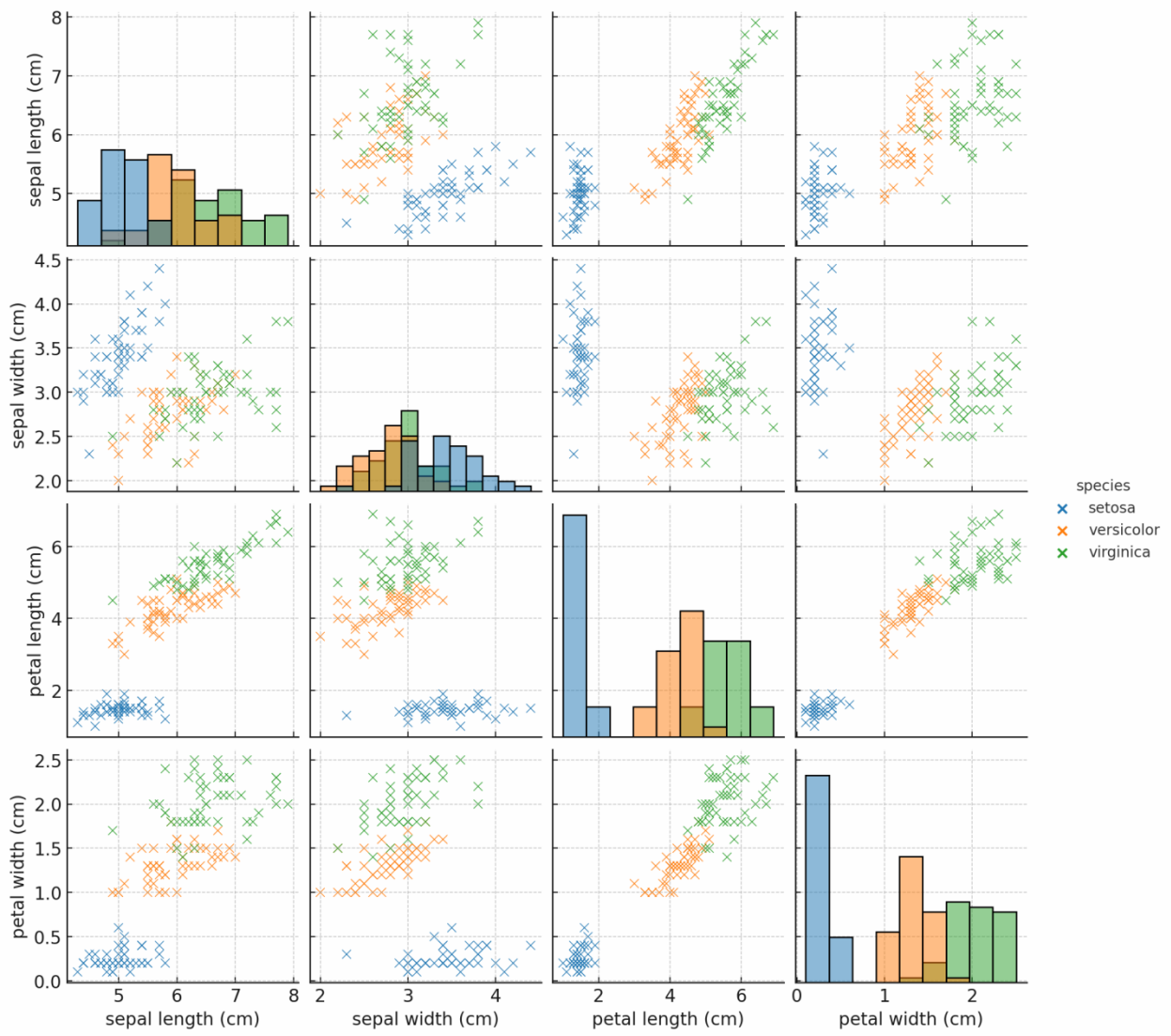
1. Sepal Length
2. Sepal Width
3. Petal Length

4. Petal Width

Code:

```
import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

import pandas as pd

# Load Iris dataset

iris = load_iris(as_frame=True)

df = iris.frame

# Add species column

df["species"] = df["target"].map(dict(enumerate(iris.target_names)))

# Create pair plot

sns.pairplot(df, vars=iris.feature_names, hue="species", diag_kind="hist")

plt.show()
```

Output:



HeatMaps:

A **heatmap** is a data visualization technique that represents values in a table or matrix format using **colors.**

- Darker or brighter colors = higher values
- Lighter colors = lower values

A heatmap is mainly Numerical–Numerical visualization, but sometimes it can also combine **Categorical–Numerical** (like subjects vs students marks).

Heatmaps are divided into two types. They are

1. Correlation Heatmap
2. Value Heatmap (Matrix form)

Correlation Heatmap:

The Correlation Heatmap shows how strongly two numerical variables are related. (e.g., Height vs Weight vs Age).

Ex:

In student marks, Math & Science marks may have high correlation.

Value Heatmap (Matrix form):

It shows numerical values inside a table (matrix) using colors.

Ex:

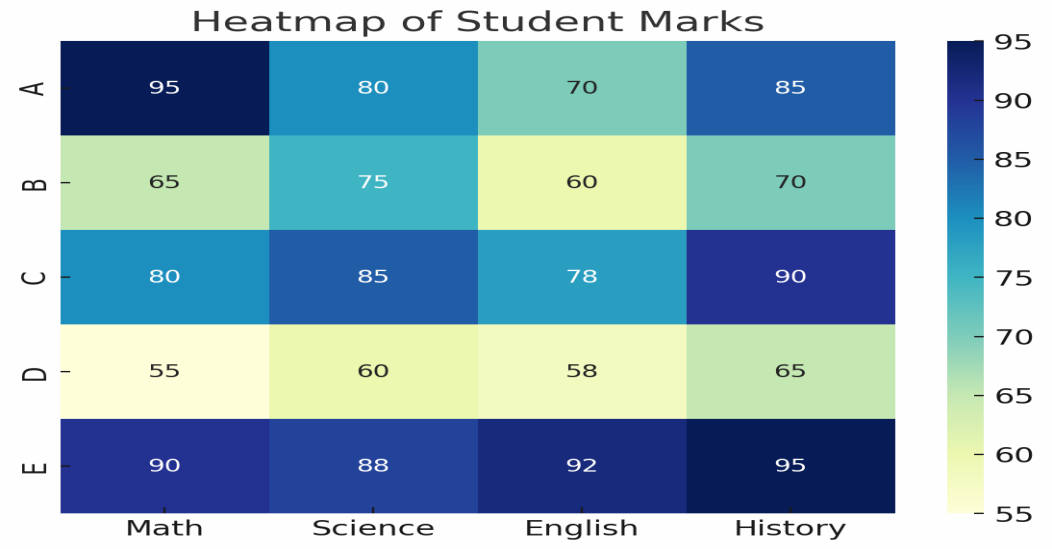
Student	Math	Science	English	History
A	95	80	70	85
B	65	75	60	70
C	80	85	78	90
D	55	60	58	65
E	90	88	92	95

Code:

```
import seaborn as sns

import matplotlib.pyplot as plt
```

```
import pandas as pd


# Example student marks dataset
data = {
    "Math": [95, 65, 80, 55, 90],
    "Science": [80, 75, 85, 60, 88],
    "English": [70, 60, 78, 58, 92],
    "History": [85, 70, 90, 65, 95]
}
students = ["A", "B", "C", "D", "E"]
df = pd.DataFrame(data, index=students)
# Create heatmap
plt.figure(figsize=(7, 5))
sns.heatmap(df, annot=True, cmap="YlGnBu", cbar=True)
plt.title("Heatmap of Student Marks")
plt.show()
```

Output:



In above,

Rows = Students (A–E)

Columns = Subjects (Math, Science, English, History)

Numbers = Marks

Colors = Show performance (darker = higher marks, lighter = lower marks)

Correlation Analysis:

Correlation Analysis is a statistical method used to measure the relationship between two numerical variables**.**

Types:

Correlation divided into 3 types .They are

1. Positive Correlation (+ve)
2. Negative Correlation (-ve)
3. Zero Correlation (0)

Positive Correlation (+ve):

Here one variable increases, the other also increases.

Ex:

Study Hours ⬆ → Exam Marks ⬆

Negative Correlation (-ve):

Here one variable increases, the other decreases.

Ex:

TV Watching Time ⬆ → Exam Marks ⬇

Zero Correlation (0):

No relationship between the variables.

Ex:

Shoe Size and Exam Marks

Correlation Coefficient (r)

A number between -1 and +1 that shows the strength and direction.

+1 → Perfect Positive correlation

-1 → Perfect Negative correlation

0 → No correlation

Example

Student	Study Hours	Marks
A	2	50
B	4	65
C	6	80
D	8	90

Code:

```
import numpy as np
import pandas as pd
# Student data
data = {
    "Study Hours": [2, 4, 6, 8],
    "Marks": [50, 65, 80, 90]
}
# Create DataFrame
df = pd.DataFrame(data)
# Calculate correlation
correlation = df["Study Hours"].corr(df["Marks"])
print("Correlation between Study Hours and Marks:", correlation)
```

Output:

```
Correlation between Study Hours and Marks: 0.998
```

Covariance Analysis:

DATA VISUALIZATION TECHNIQUES

UNIT-IV

Data Visualization Techniques :

Data Visualization is the graphical representation of information and data using visual elements like charts, graphs, and maps to help people understand the significance of the data.

Visualization with Matplotlib:

1) Matplotlib is a popular Python library used for creating data visualizations such as line charts, bar charts, histograms, scatter plots, pie charts, and many more.

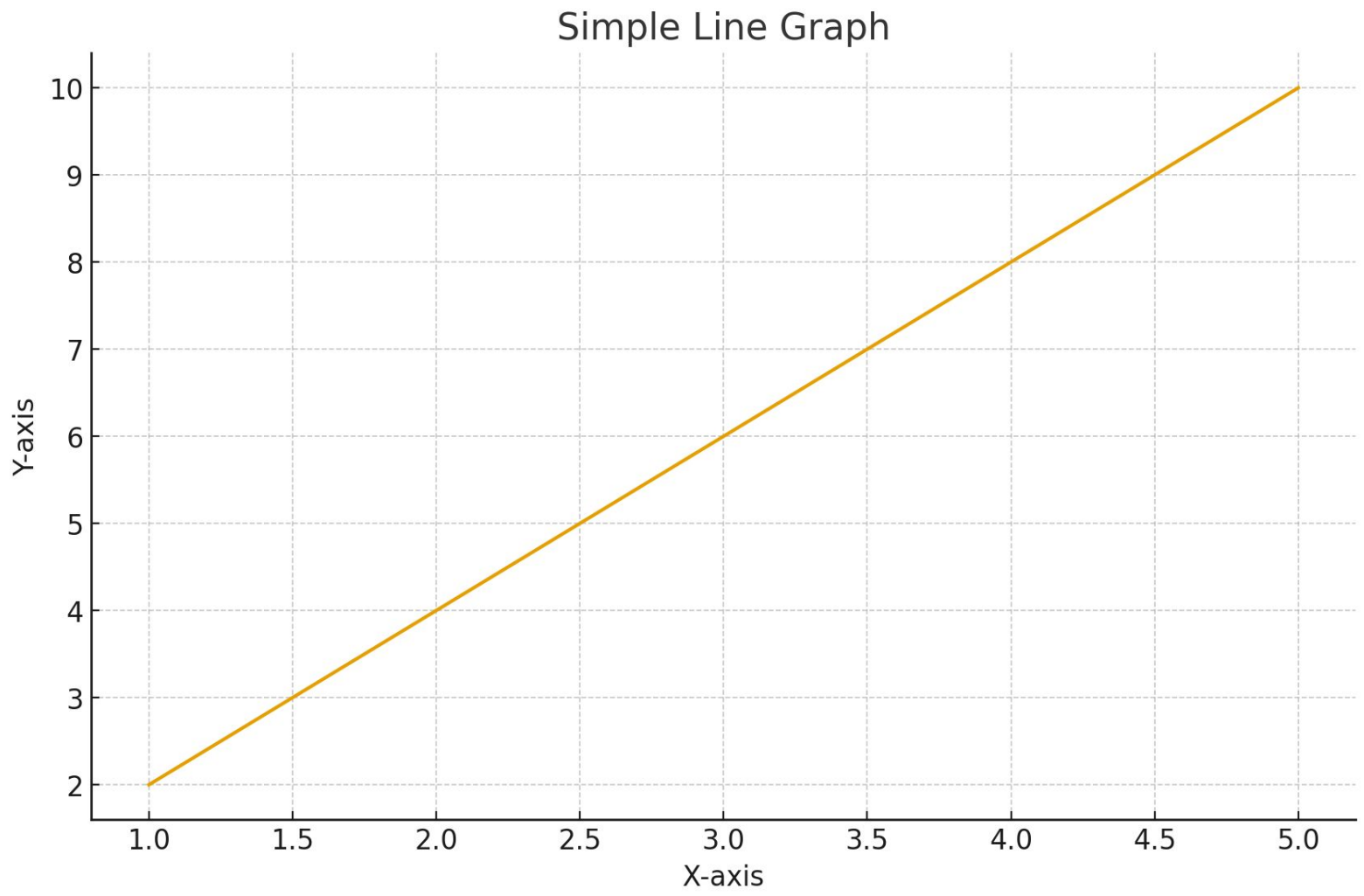
2) It is the foundation of visualization in Python, and many advanced libraries (like Seaborn, Plotly, and Pandas Visualization) are built on top of it.

It is highly flexible and works well with NumPy and Pandas.
The most commonly used module is matplotlib.pyplot.

Example:

```
import matplotlib.pyplot as plt
# 1. Prepare Data
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
# 2. Create Plot
plt.plot(x, y)
# 3. Add Labels & Title
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Simple Line Graph")
# 4. Show Plot
plt.show()
```

OUTPUT :


Simple Line Graph
Y-axis
X-axis
10
9
8
7
6
5
4
3
2
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0

Why use Matplotlib

· Easy to create basic charts quickly.
· Highly customizable (colors, labels, styles, markers).
· Works well with NumPy and Pandas.
· Can export plots to PNG, PDF, SVG, EPS formats.
· Used in data science, machine learning, and research for visualization.

Common Visualization Types in Matplotlib

The Common Visualization Types are

1. Bar Chart
2. Histogram
3. Scatter Plot
4. Pie Chart

Visualization with Seaborn

Seaborn is a Python data visualization library built on top of Matplotlib. It provides a high-level interface for creating beautiful, informative, and statistical plots with much simpler code than Matplotlib.

Comparison of Matplotlib and Seaborn:

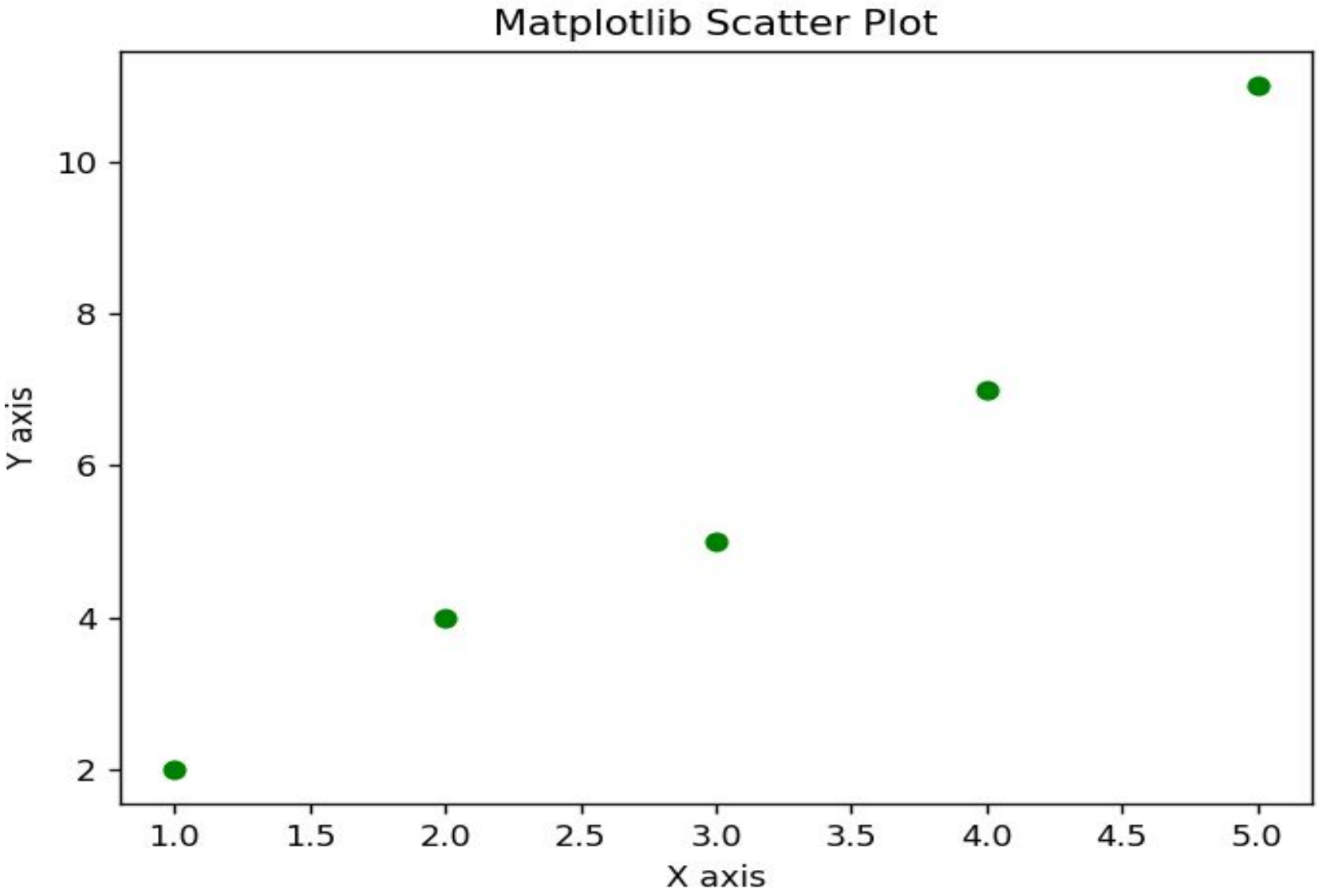
Matplotlib is a basic plotting library in Python.

1)We can create bar charts, scatter plots, histograms, etc.

2)We have to manually customize the plots (colors, markers, labels).

3)It gives high flexibility, but the code can get longer.

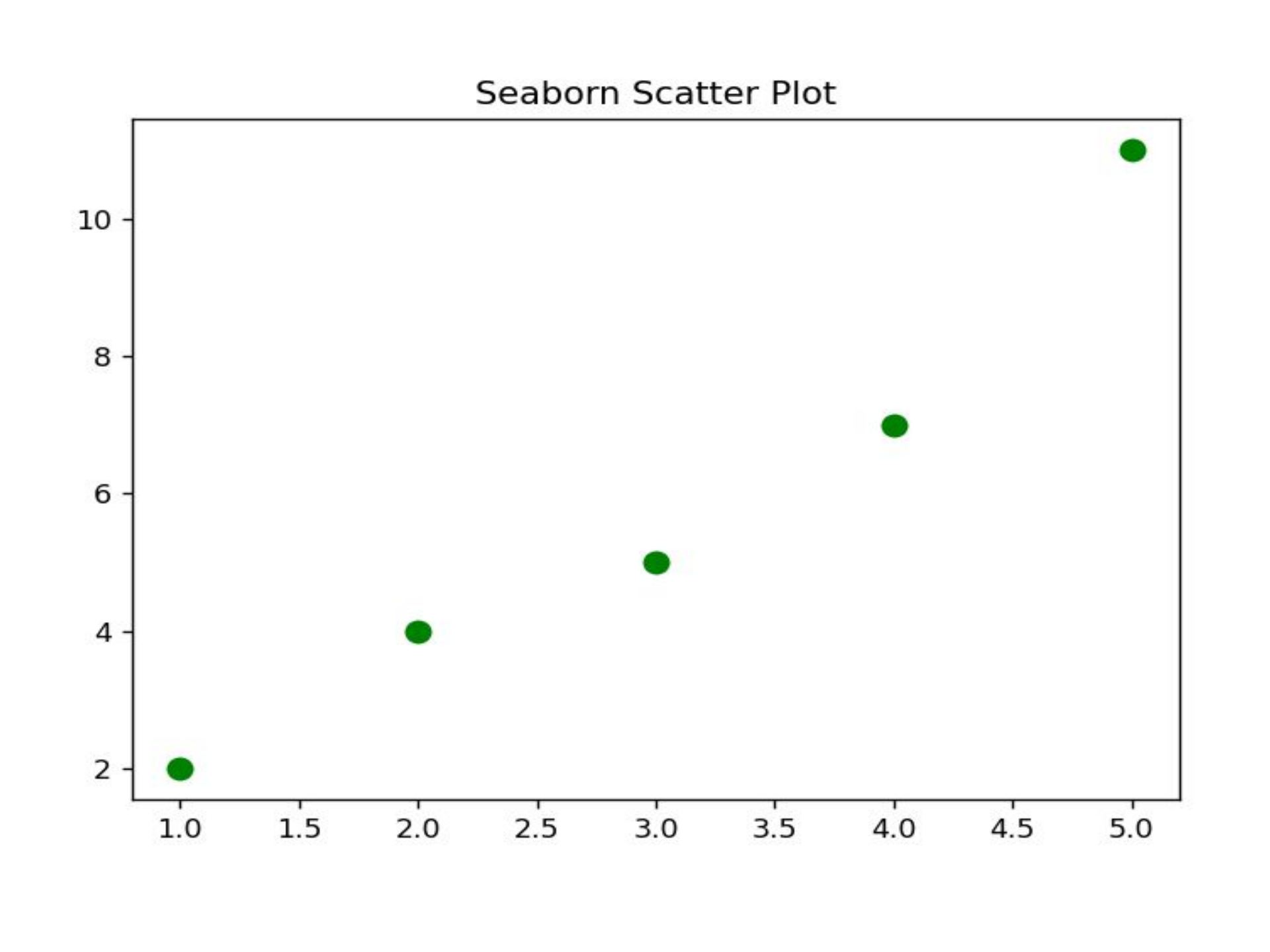
```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 4, 5, 7, 11]
plt.scatter(x, y, color='green') # Scatter plot
plt.xlabel("X axis")
plt.ylabel("Y axis")
plt.title("Matplotlib Scatter Plot")
plt.show()
```


Matplotlib Scatter Plot
Y axis
X axis
10
8
6
4
2
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0

Seaborn is built on top of Matplotlib, mainly for statistical data visualization.

- Plots are cleaner and prettier by default, no extra styling needed.
- Works directly with Pandas Data Frames.
- Advanced plots like heat maps, violin plots, pair plots are easy to create.

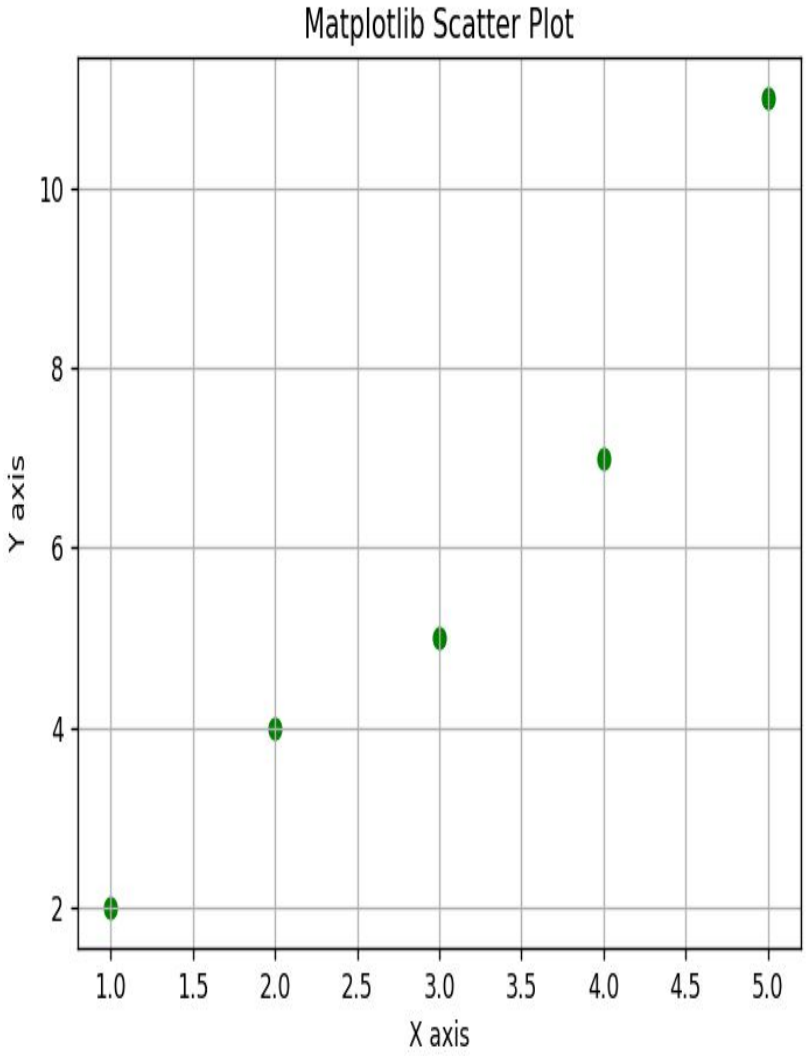
```
import seaborn as sns
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 4, 5, 7, 11]
sns.scatterplot(x=x, y=y, color='green', s=100)  # prettier scatter plot
plt.title("Seaborn Scatter Plot")
plt.show()
```


Seaborn Scatter Plot
10
8
6
4
2
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0

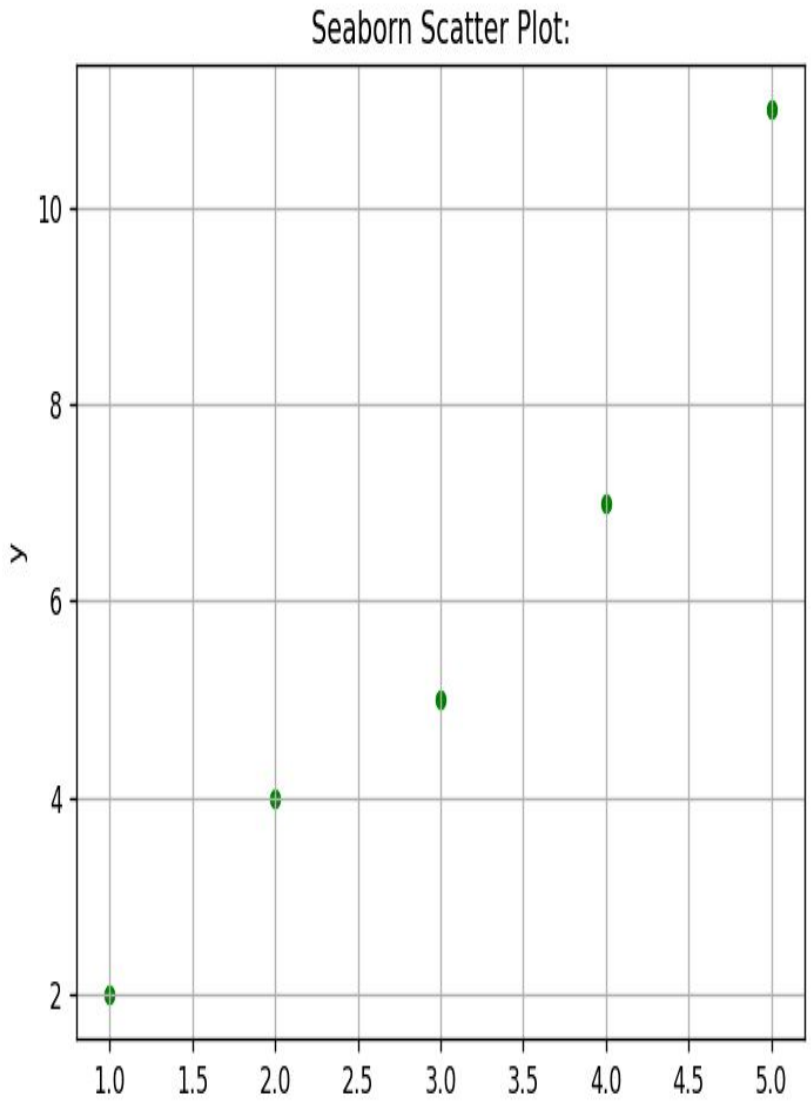
Matplotlib vs Seaborn

Feature	Matplotlib	Seaborn
Level	Basic plotting library	Built on top of Matplotlib
Style	Default style is plain/simple → needs manual customization	Default comes with neat & pretty plots
Ease of use	More code, requires a lot of customization	Less code, automatic styling
Data Handling	Works with Lists and Arrays	Works directly with **Pandas DataFrames**
Advanced Plots	Difficult (requires manual coding)	Easy (heatmap, violin plot, pair plot, etc.)
Flexibility	Very high (can manually change everything)	Less flexible, but gives high-level convenience

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 5, 7, 11]

plt.scatter(x, y, color='green')
plt.xlabel("X axis")
plt.ylabel("Y axis")
plt.title("Matplotlib Scatter Plot")
plt.show()
```



```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

data = pd.DataFrame({
 "x": [1, 2, 3, 4, 5],
 "y": [2, 4, 5, 7, 11]
})

sns.scatterplot(data=data,
x="x", y="y", color="green")
plt.title("Seaborn Scatter Plot“)
Plt.grid(True)
plt.show()
```



Customizing Plots:

Customized plots are graphs or charts that have been modified or styled to make them clearer, easier to understand, and visually attractive. This includes changing colors, markers, line styles, labels, titles, legends, grids, or figure size.

Why it is important

· Helps you understand data quickly.
· Makes your assignments and projects look neat.
· Helps others understand your results easily.

Things we can customize in a plot

1. Titles
2. Legends
3. Labels
4. Themes
5. Color
6. Marker & Line Styles

Titles:

A title is the name or heading of a graph that tells what the plot is about.

It helps the viewer understand the purpose of the graph at a glance.

Every good plot should have a descriptive title.

Ex: Student Marks Over 5 Months

Legends:

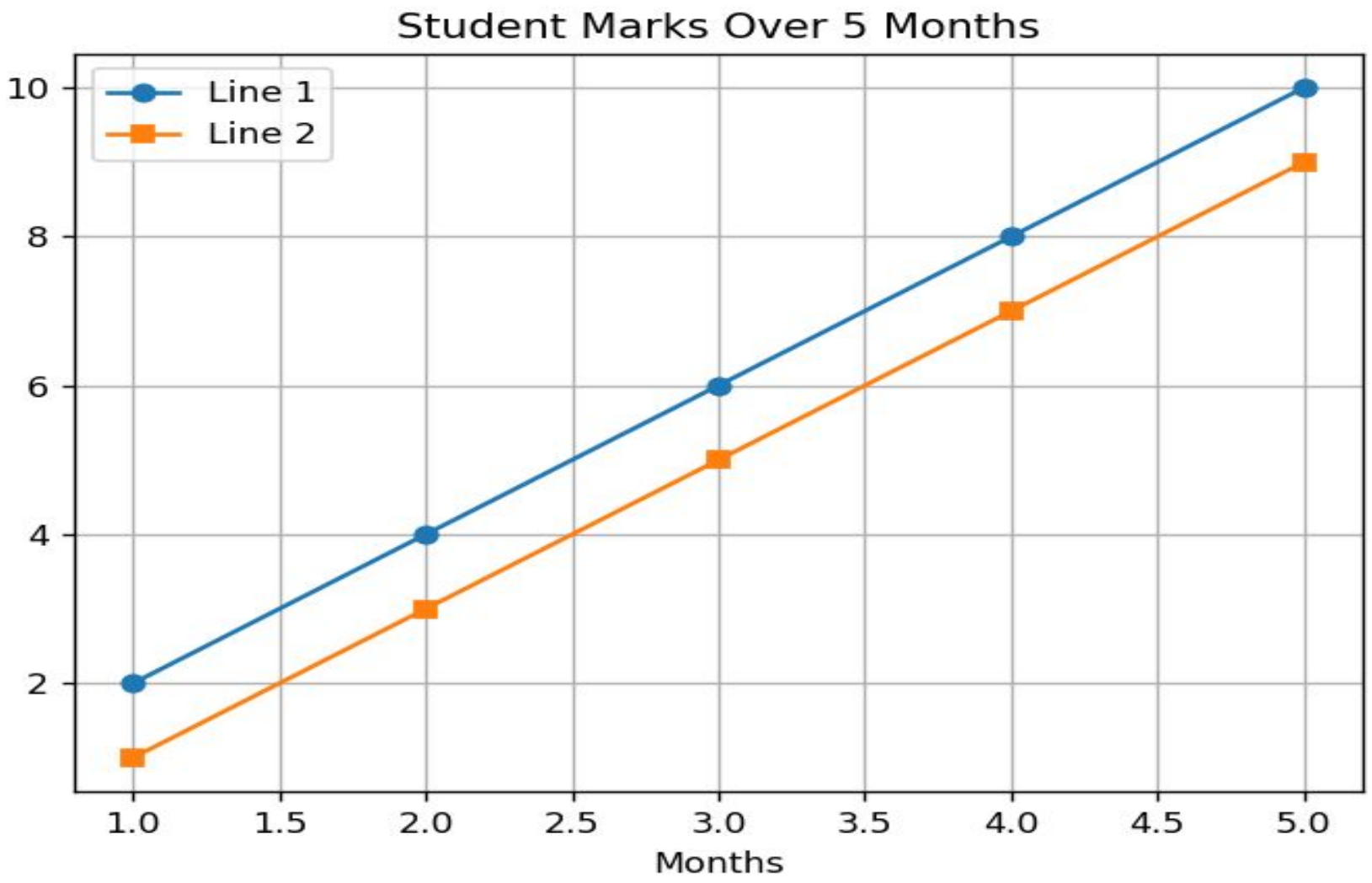
A legend is a small box or area on a graph that explains what different colors, lines, or markers represent.

1) It helps the viewer understand multiple data series in a single plot.

2) Without a legend, it can be confusing to know which line or color corresponds to which data.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]          # X-axis values
y1 = [2, 4, 6, 8, 10]        # Y-axis values for Line 1
y2 = [1, 3, 5, 7, 9]         # Y-axis values for Line 2
# Plot two lines
plt.plot(x, y1, label="Line 1", marker='o')  # First line with marker
plt.plot(x, y2, label="Line 2", marker='s')  # Second line with different
marker
# Add title and axis labels
plt.title("Student Marks Over 5 Months")  # Graph title
plt.xlabel("Months")                      # X-axis label
plt.ylabel("Marks")                       # Y-axis label
# Add a legend
plt.legend()
# Show grid (optional, makes graph clearer)
plt.grid(True)
plt.show()
```

OUTPUT:


Student Marks Over 5 Months
Line 1
Line 2
10
8
6
4
2
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0
Months

Labels:
In the context of data visualization or plotting, a label is a text description used to identify or name an element of the graph. Labels help anyone looking at the graph understand what the data represents.

Here are the main types of labels in plotting:

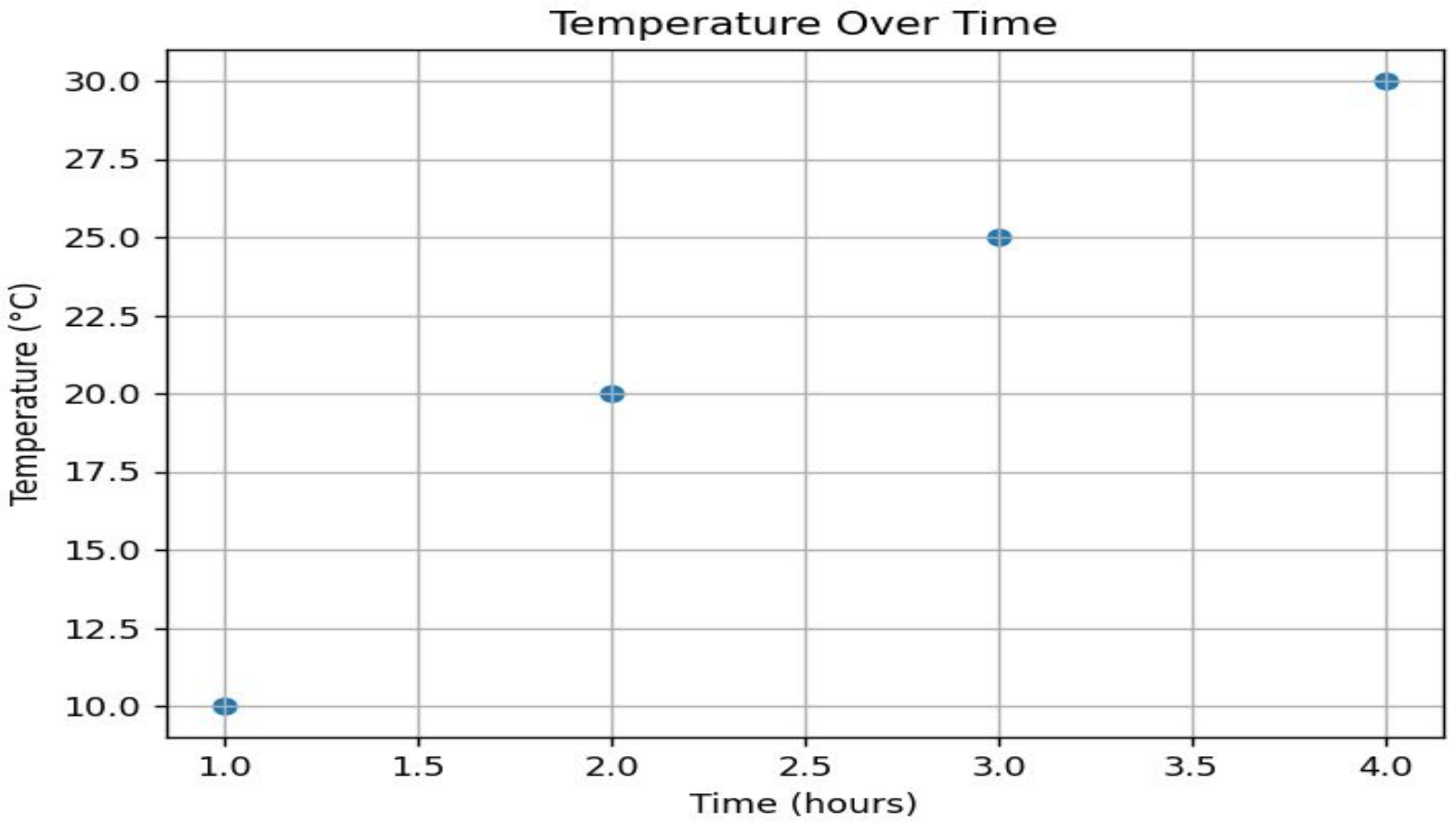
Axis Labels – Describe what each axis represents.

X-axis label: Describes the horizontal axis.
Y-axis label: Describes the vertical axis.

Ex:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]
plt.scatter(x, y)
plt.xlabel("Time (hours)")  # X-axis label
plt.ylabel("Temperature (°C)")  # Y-axis label
plt.title("Temperature Over Time")
plt.grid(“True”)
plt.show()
```

OUTPUT:


Temperature Over Time
Temperature (°C)
30.0
27.5
25.0
22.5
20.0
17.5
15.0
12.5
10.0
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Time (hours)

Themes:

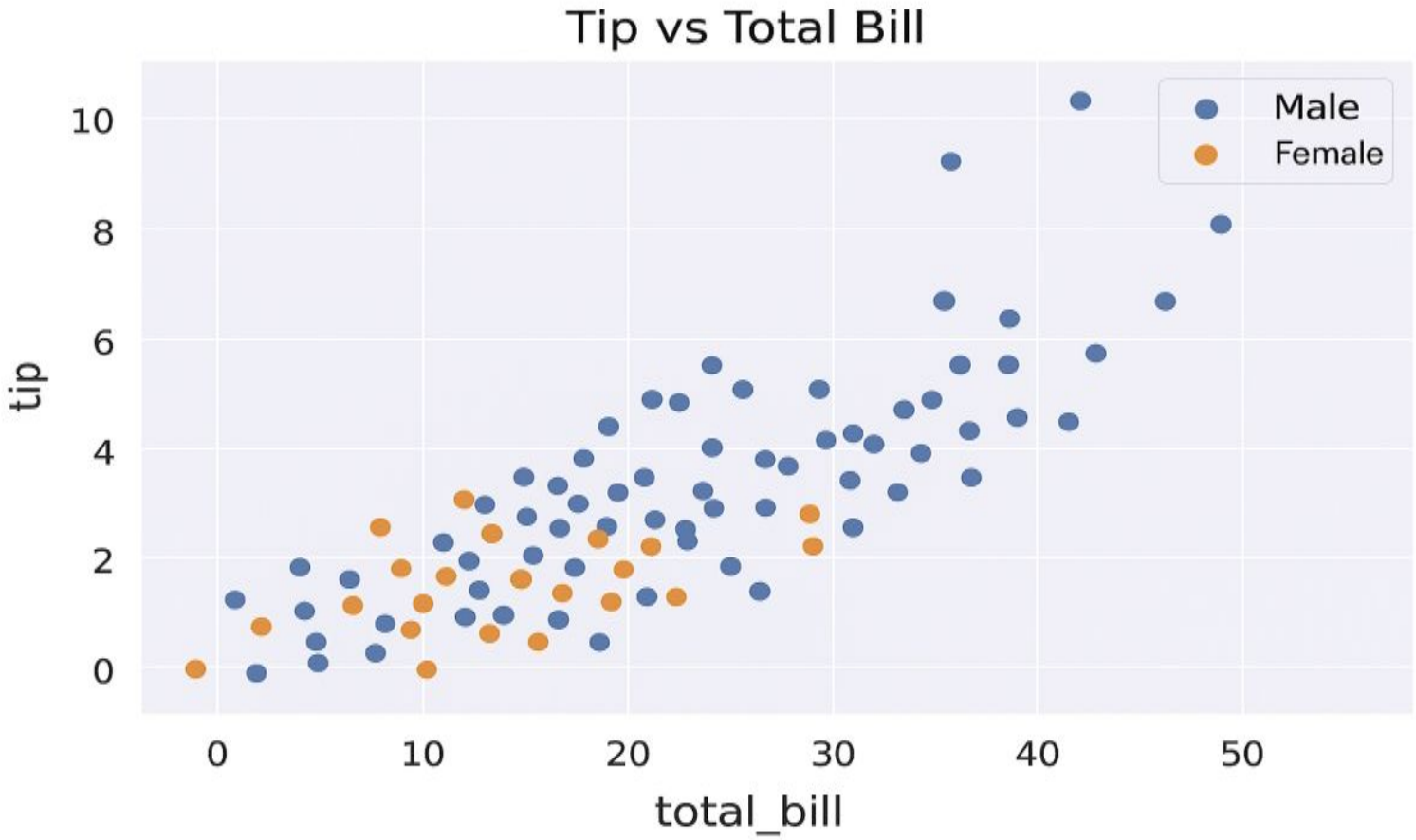
In the context of customized plots (like in Matplotlib or Seaborn in Python), a theme refers to the overall visual style or appearance of the plot.

It defines things like:

1) Background color of the plot
2) Grid lines style and visibility
3) Font style and size for titles and labels
4) Colors of lines, markers, bars, etc.

```
import seaborn as sns
import matplotlib.pyplot as plt
# Load example dataset
tips = sns.load_dataset("tips")
# Apply a theme
sns.set_theme (style="darkgrid") # Theme applied
# Create a plot
sns.scatterplot (data=tips, x="total_bill", y="tip", hue=“gender")
plt.title("Tip vs Total Bill")
plt.show()
```

OUTPUT:


Tip vs Total Bill
Male
Female
tip
total_bill
0
2
4
6
8
10
0
10
20
30
40
50

Advanced Visuals:

The advanced visuals are

1. Violin Plots
2. Strip Plots
3. Swarm Plots

Violin Plots:

A Violin Plot is an advanced data visualization technique that combines the features of a box plot and a density plot.

It is mainly used to understand both the summary statistics (like median, quartiles) and the distribution (spread and density) of the data.

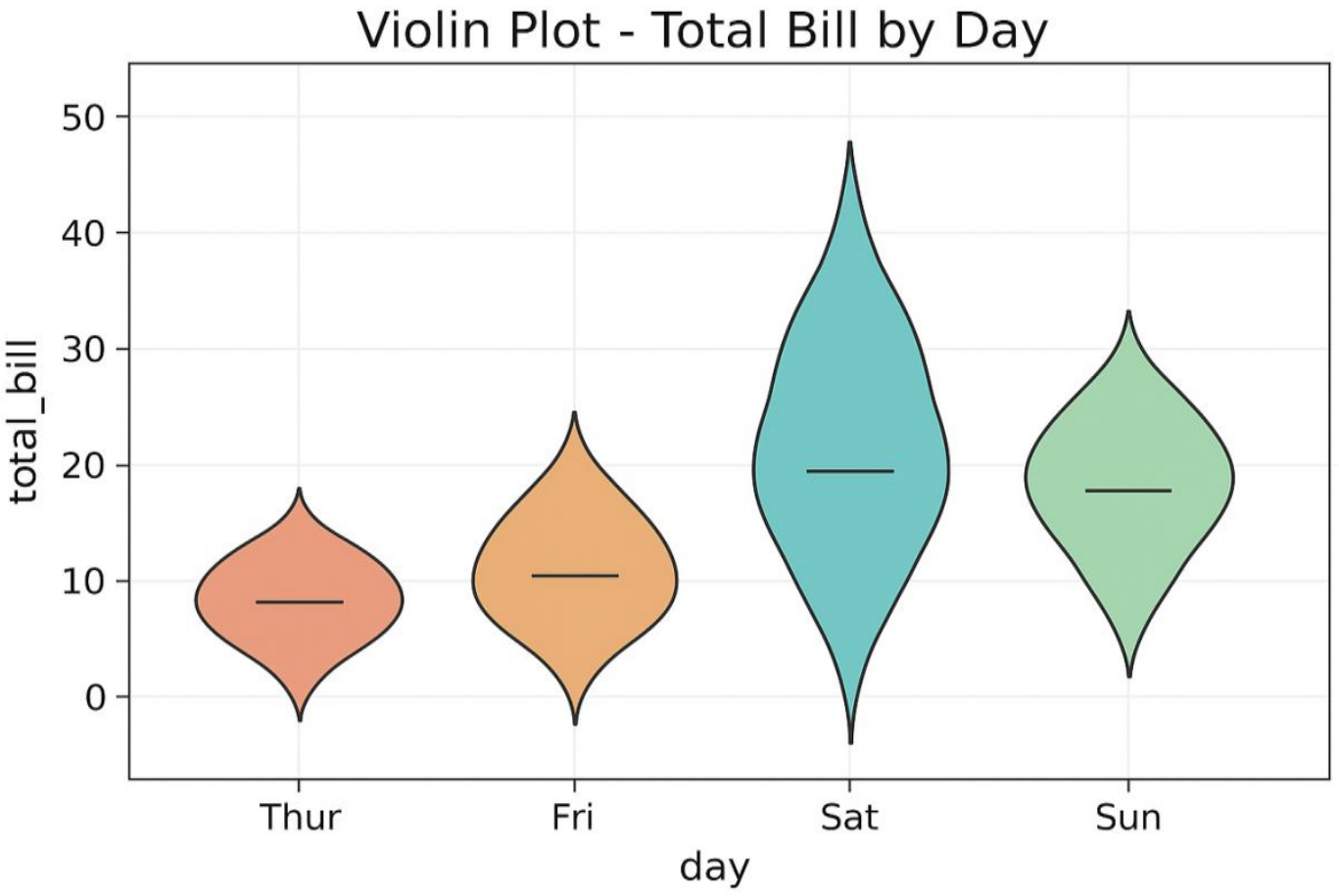
Difference from Box plot:

· Box plot shows only summary values (median, quartiles, min, and max).

· Violin Plot shows both summary values and distribution shape.

```
import seaborn as sns
import matplotlib.pyplot as plt
# Load dataset
tips = sns.load_dataset("tips")
# Set theme
sns.set_theme(style="darkgrid")
# Violin plot
plt.figure(figsize=(8,6))
sns.violinplot(x="day", y="total_bill", data=tips,
palette="Set2")
plt.title("Violin Plot - Total Bill by Day")
plt.show()
```

OUTPUT:


Violin Plot - Total Bill by Day
total_bill
50
40
30
20
10
0
Thur
Fri
Sat
Sun
day

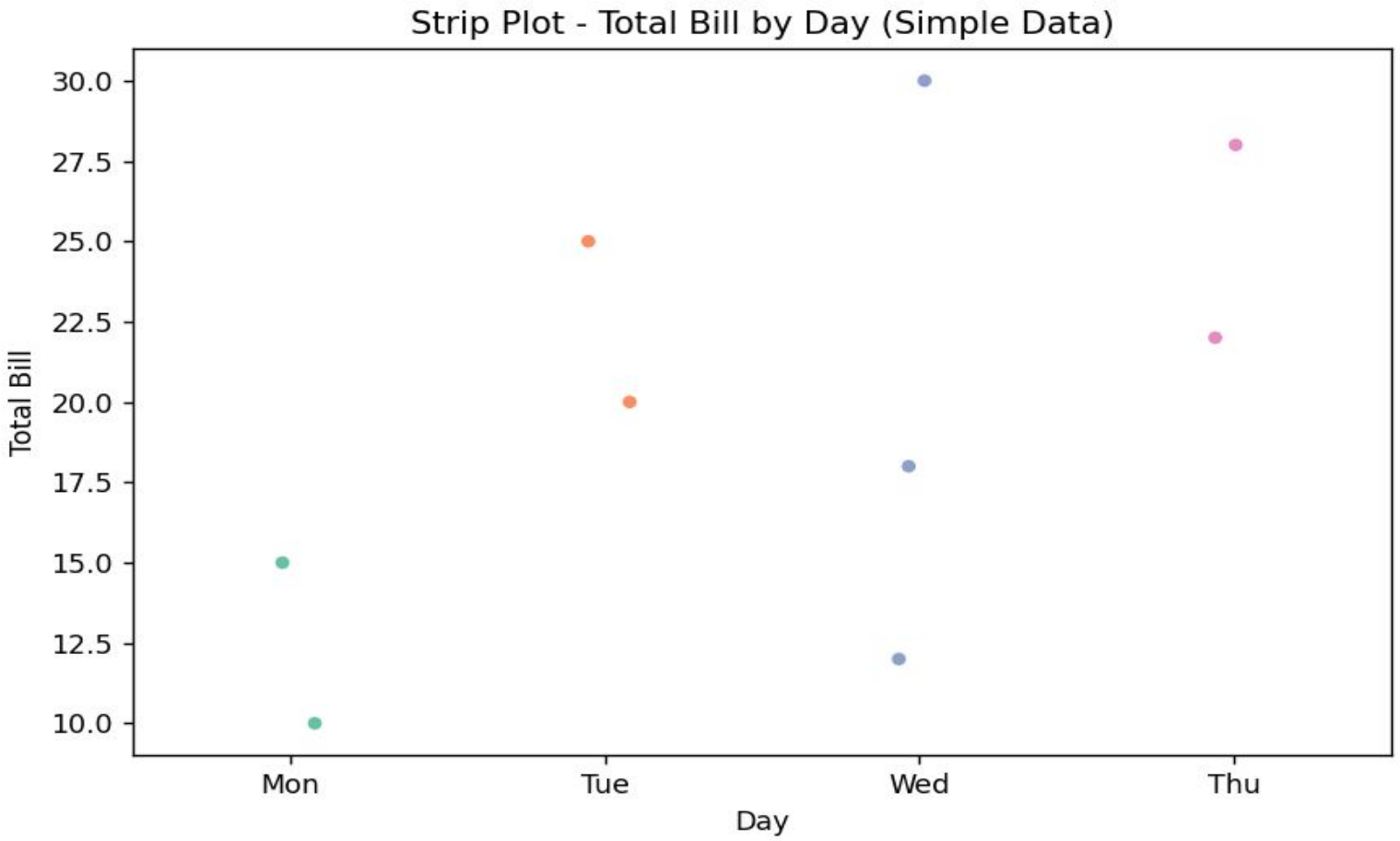
Strip Plots:

A strip plot is a type of data visualization that displays individual data points along an axis. It's mostly used to show the distribution of a dataset and how values are spread across different categories.

Unlike bar charts or box plots, a strip plot shows each individual data point, which allows you to easily identify clusters, gaps, or outliers.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Simple example data
days = ["Mon", "Mon", "Tue", "Tue", "Wed", "Wed", "Wed", "Thu",
"Thu"]
bills = [10, 15, 20, 25, 30, 12, 18, 22, 28]

# Create Strip Plot
plt.figure(figsize=(8,5))
sns.stripplot(x=days, y=bills, jitter=True, palette="Set2")

# Add title and labels
plt.title("Strip Plot - Total Bill by Day (Simple Data)")
plt.xlabel("Day")
plt.ylabel("Total Bill")

plt.show()
```

Output:


Strip Plot - Total Bill by Day (Simple Data)
Total Bill
30.0
27.5
25.0
22.5
20.0
17.5
15.0
12.5
10.0
Mon
Tue
Wed
Thu
Day

Swarm Plots:

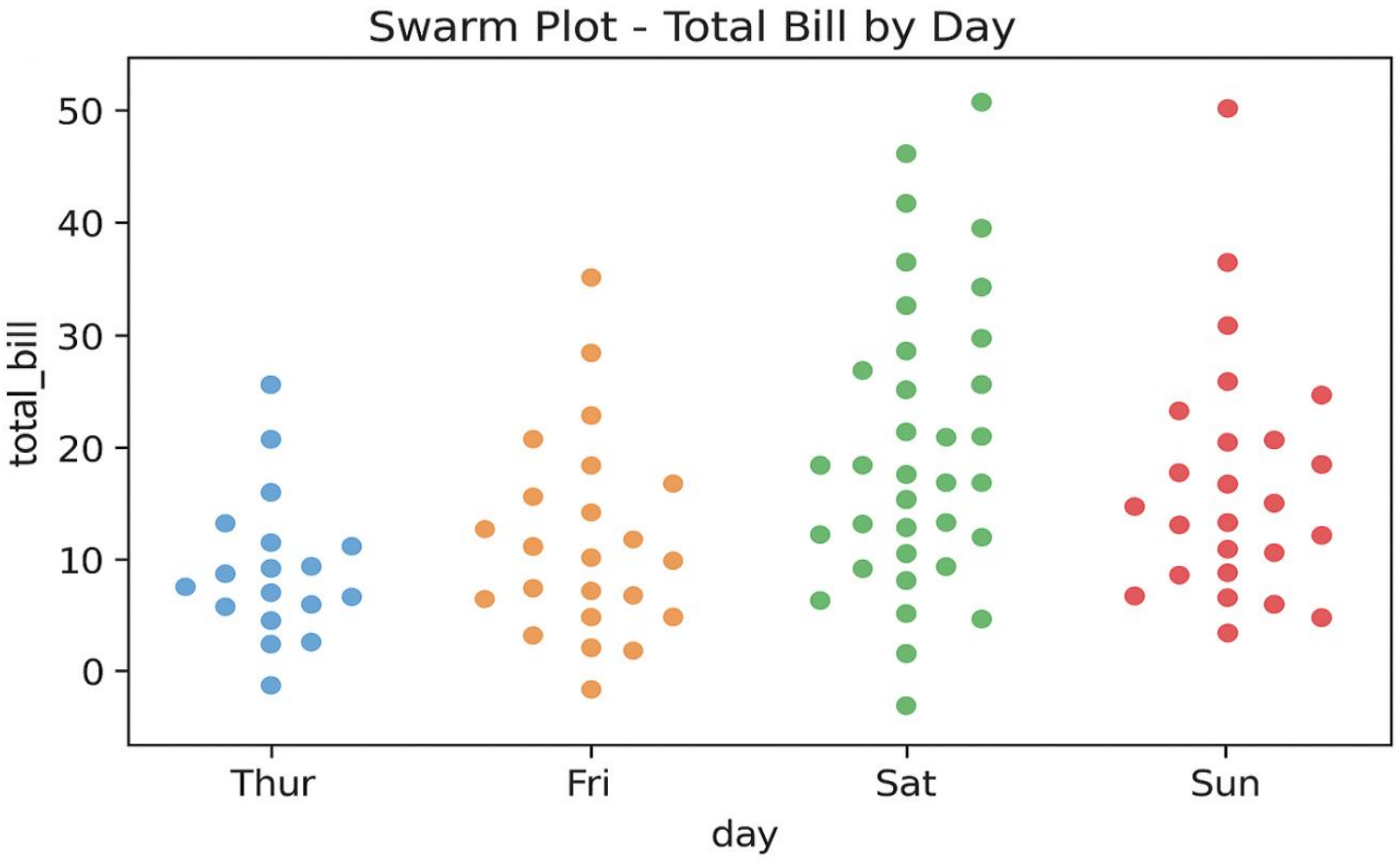
A Swarm Plot is a type of scatter plot used to display all individual data points in a dataset along one axis. Unlike a strip plot, a swarm plot avoids overlap of points by adjusting their positions along the categorical axis. This makes it easier to see the distribution of values for each category.

· Shows all data points.
· Avoids overlapping of points.
· Works well for small to medium-sized datasets.
· often combined with box plots or violin plots for more insights.

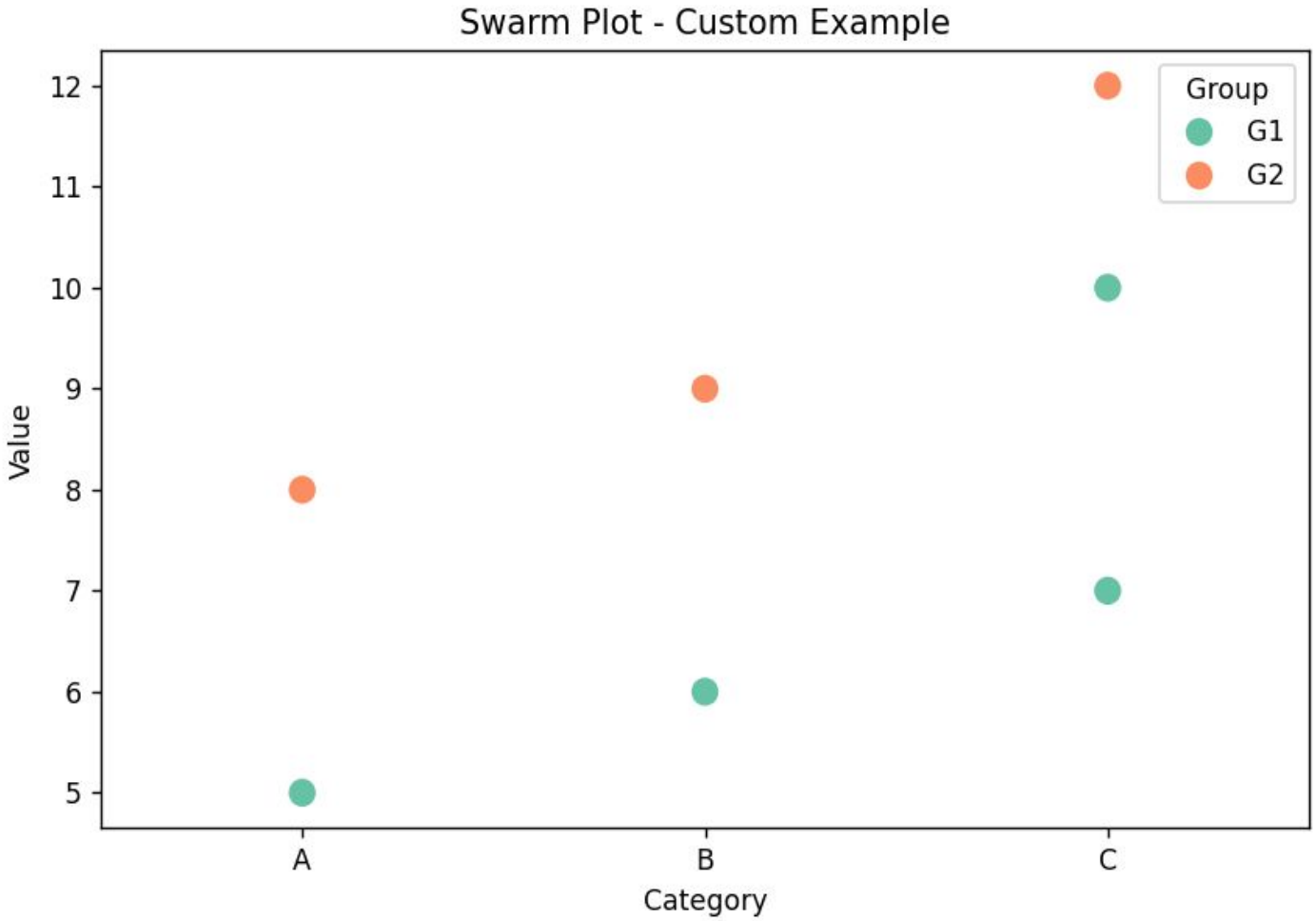
Example :

```
import seaborn as sns
import matplotlib.pyplot as plt
# Load example dataset
tips = sns.load_dataset ("tips")
# Create swarm plot
plt.figure (figsize= (8,6))
sns.swarmplot(x="day", y="total_bill",
    data=tips, palette="Set2")
plt.title("Swarm Plot - Total Bill by Day")
plt.show()
```

OUTPUT:


Swarm Plot - Total Bill by Day
total_bill
50
40
30
20
10
0
Thur
Fri
Sat
Sun
day

```
import seaborn as sns
import matplotlib.pyplot as plt

# Custom small dataset
categories = ["A", "A", "B", "B", "C", "C", "C"]
values = [5, 8, 6, 9, 7, 12, 10]
groups = ["G1", "G2", "G1", "G2", "G1", "G2", "G1"]  # optional hue

plt.figure(figsize=(8,5))

# Swarm Plot
sns.swarmplot(x=categories, y=values, hue=groups, palette="Set2", size=10)

plt.title("Swarm Plot - Custom Example")
plt.xlabel("Category")
plt.ylabel("Value")
plt.legend(title="Group")
plt.show()
```


Swarm Plot - Custom Example
Group
G1
G2
12
11
10
9
8
7
6
5
Value
A
B
C
Category

Comparison of Violin Plots, Strip Plots and Swarm Plots:

Feature	Violin Plot	Strip Plot	Swarm Plot
Shows distribution shape?	✅ Yes (via KDE)	❌ No	❌ No
Shows individual points?	Optional	✅ Yes	✅ Yes
Overlapping points?	Can overlap if points added	Can overlap	No overlap (spreads points)
Best use	Understanding density	Visualizing raw data points	Visualizing raw data points without overlap

Multivariate Visualization:

Multivariate visualization is the process of graphically representing the relationships among three or more variables in a dataset.
It helps us understand patterns, correlations, and interactions between multiple features simultaneously.

· **Univariate:** 1 variable (e.g., histogram of total_bill)
· **Bivariate:** 2 variables (e.g., scatter plot of total_bill vs tip)
· **Multivariate:** 3 or more variables (e.g., scatter plot with color representing gender and size representing group size)

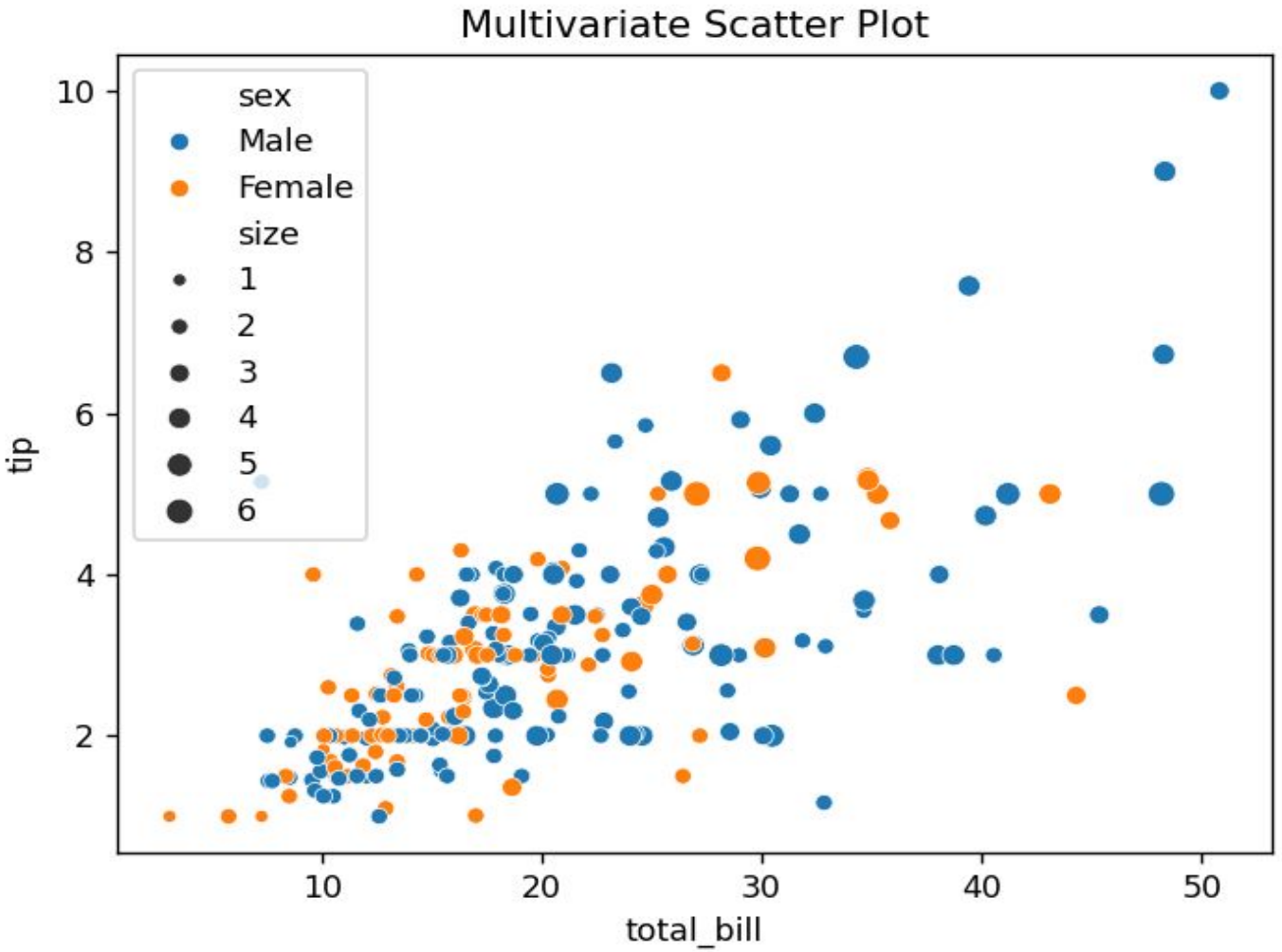
Purpose:

· To identify patterns and correlations among multiple variables.
· To detect clusters, trends, or outliers.
· To help in feature selection for machine learning models.

Ex:

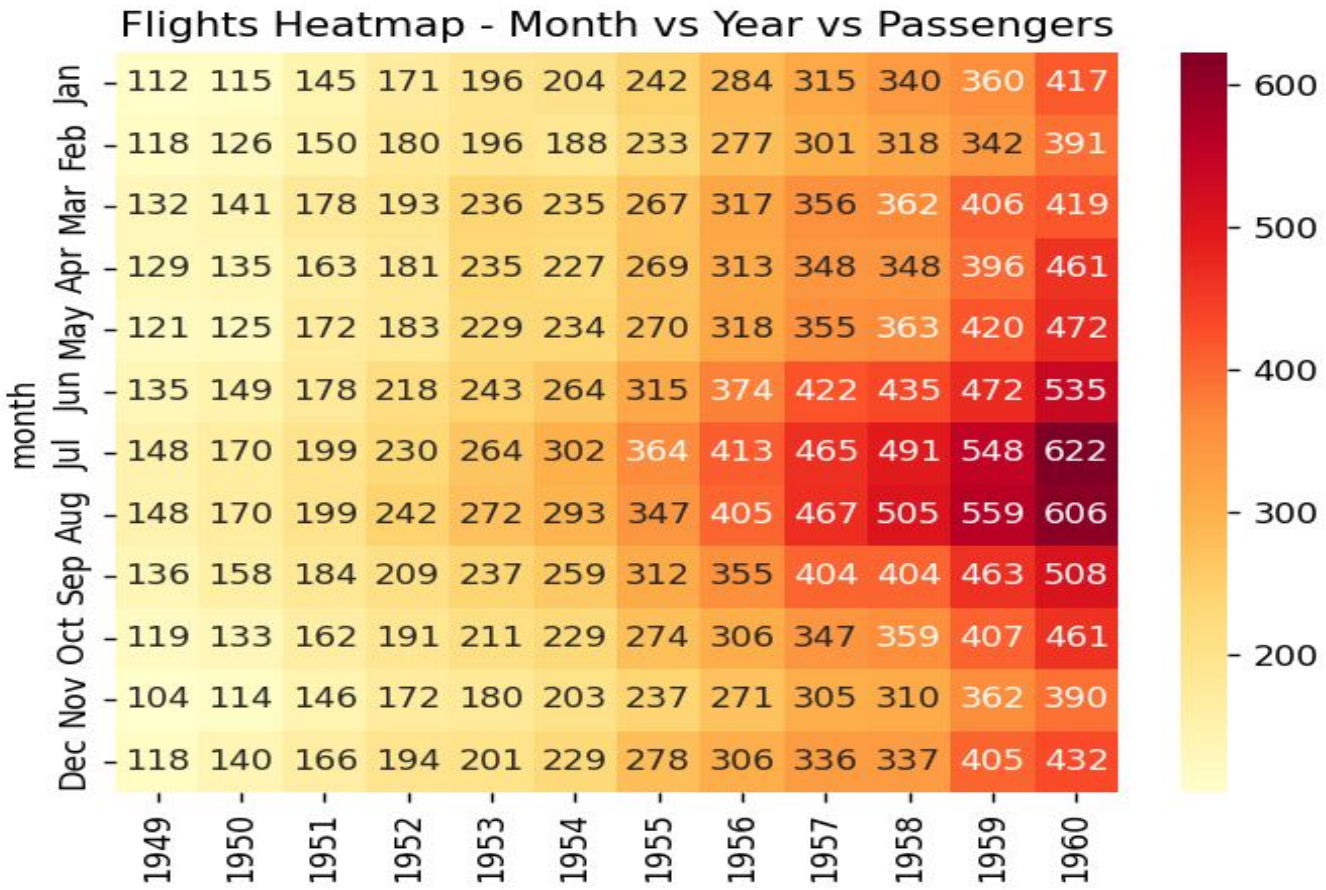
```
import seaborn as sns
import matplotlib.pyplot as plt
tips = sns.load_dataset("tips")

sns.scatterplot(
    data=tips,
    x="total_bill",
    y="tip",
    hue="gender",    # color represents gender
    size="size"   # size represents group size
)
plt.title("Multivariate Scatter Plot")
plt.show()
```

OUTPUT :


Multivariate Scatter Plot
sex
Male
Female
size
1
2
3
4
5
6
tip
total_bill
10
8
6
4
2
10
20
30
40
50

```
import seaborn as sns
import matplotlib.pyplot as plt

flights = sns.load_dataset("flights")

# Safe pivot_table and convert to int

pivot_table = flights.pivot_table(index="month", columns="year",
values="passengers").astype(int)

sns.heatmap(pivot_table, annot=True, fmt="d", cmap="YlOrRd")
plt.title("Flights Heatmap - Month vs Year vs Passengers")
plt.show()
```

OUTPUT :


Flights Heatmap - Month vs Year vs Passengers
month
Jan 112 115 145 171 196 204 242 284 315 340 360 417
Feb 118 126 150 180 196 188 233 277 301 318 342 391
Mar 132 141 178 193 236 235 267 317 356 362 406 419
Apr 129 135 163 181 235 227 269 313 348 348 396 461
May 121 125 172 183 229 234 270 318 355 363 420 472
Jun 135 149 178 218 243 264 315 374 422 435 472 535
Jul 148 170 199 230 264 302 364 413 465 491 548 622
Aug 148 170 199 242 272 293 347 405 467 505 559 606
Sep 136 158 184 209 237 259 312 355 404 404 463 508
Oct 119 133 162 191 211 229 274 306 347 359 407 461
Nov 104 114 146 172 180 203 237 271 305 310 362 390
Dec 118 140 166 194 201 229 278 306 336 337 405 432
1949 1950 1951 1952 1953 1954 1955 1956 1957 1958 1959 1960
600
500
400
300
200

Subplots:

Subplots are multiple plots drawn on a single figure. They allow you to visualize multiple graphs side by side or in a grid for easy comparison.

1) Instead of creating separate figures for each plot, you can organize them in rows and columns.
2) Each subplot is like a cell in a grid where a plot can be drawn.

Why use Subplots

· Compare multiple datasets visually.
· Save space instead of creating multiple figure windows.
· Maintain a single, coherent figure for reporting.

Creating Subplots in Matplotlib

We can create subplots using plt.subplot() or plt.subplots()

Using plt.subplot():

Syntax:

plt.subplot(nrows, ncols, index)

Here,

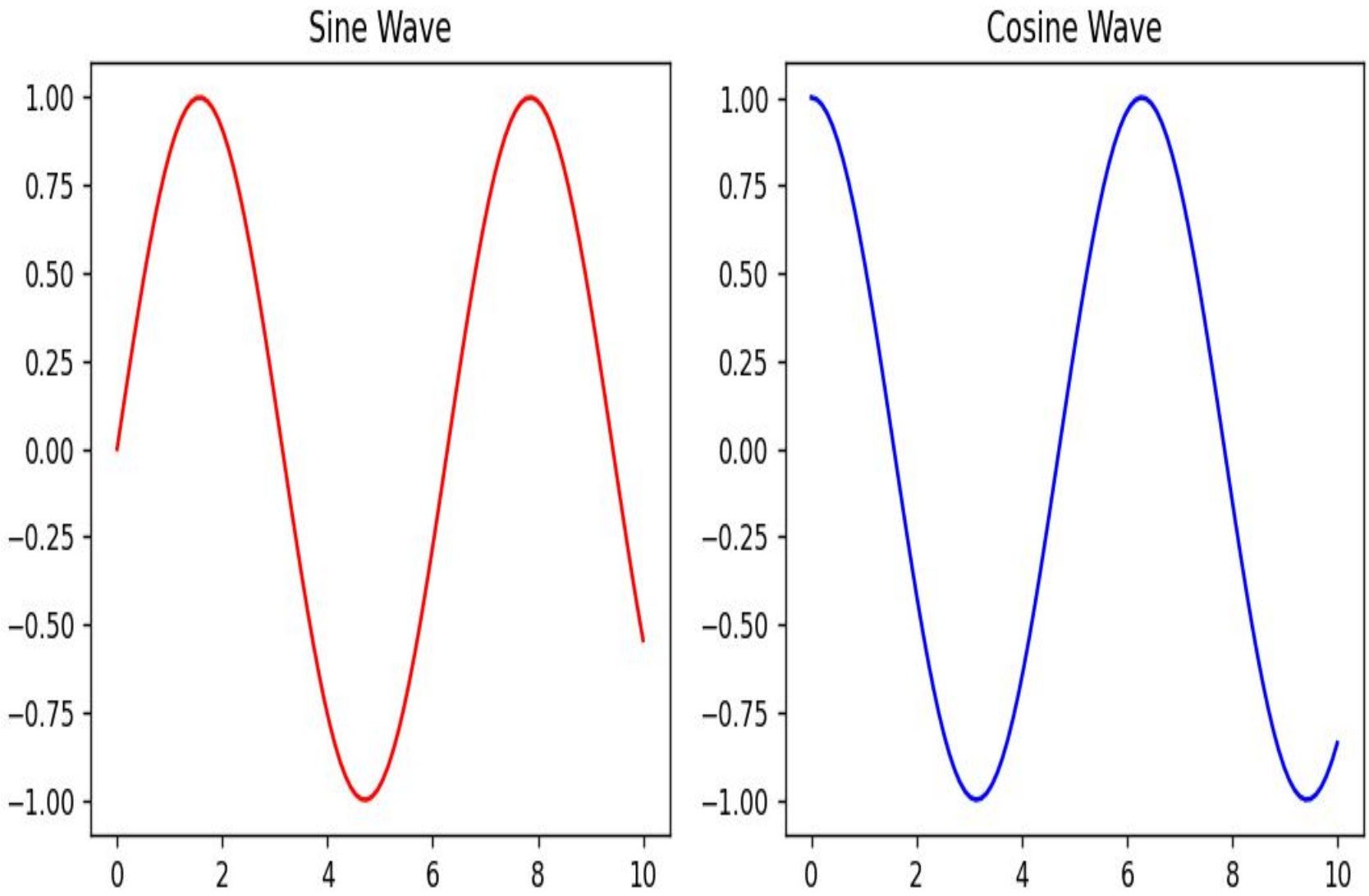
nrows = number of rows in the grid

ncols = number of columns in the grid

index= position of the current plot (starts at 1, goes left-to-right, top-to-bottom)

Example:

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 10, 100)          # x values from 0 to 10
y1 = np.sin(x)                       # sine values
y2 = np.cos(x)                      # cosine values
plt.figure(figsize=(10,4))           # figure size
# First subplot
plt.subplot(1, 2, 1)                 # 1 row, 2 columns, first plot
plt.plot(x, y1, 'r')                # plot sine in red
plt.title("Sine Wave")               # title
# Second subplot
plt.subplot(1, 2, 2)                 # 1 row, 2 columns, second plot
plt.plot(x, y2, 'b')                # plot cosine in blue
plt.title("Cosine Wave")             # title
plt.show()                          # show the figure
```

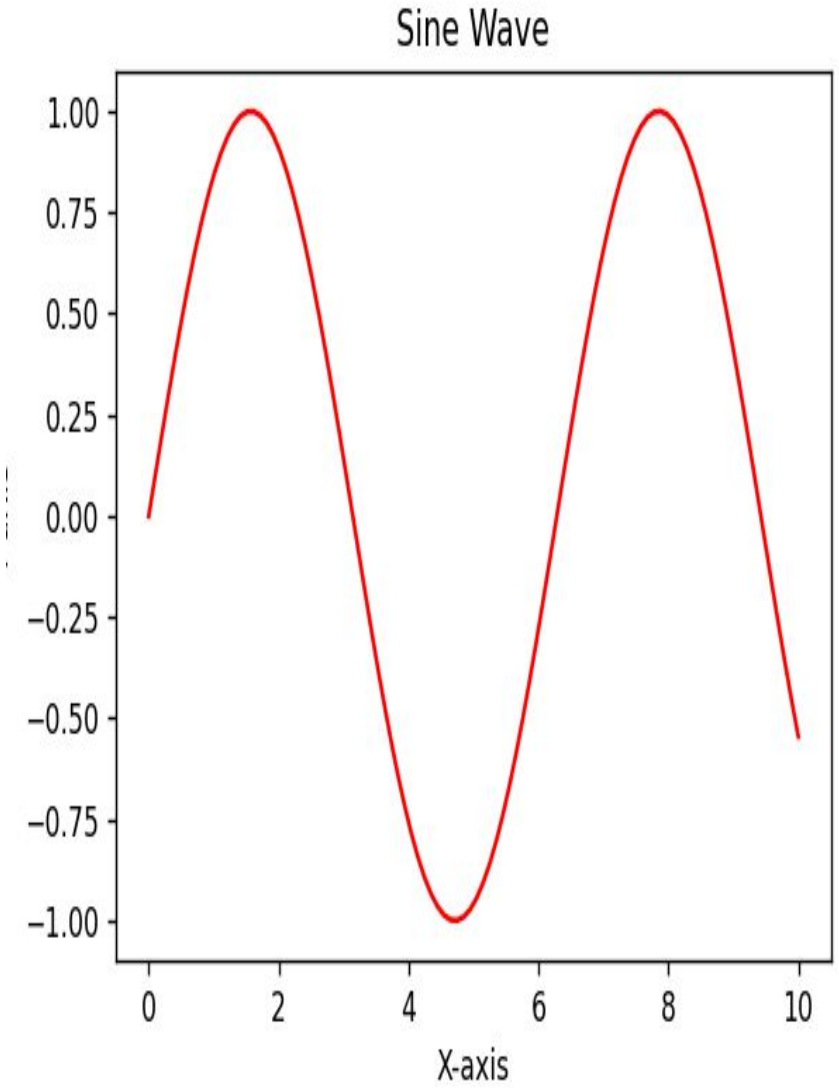

Sine Wave
Cosine Wave
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
2
4
6
8
10

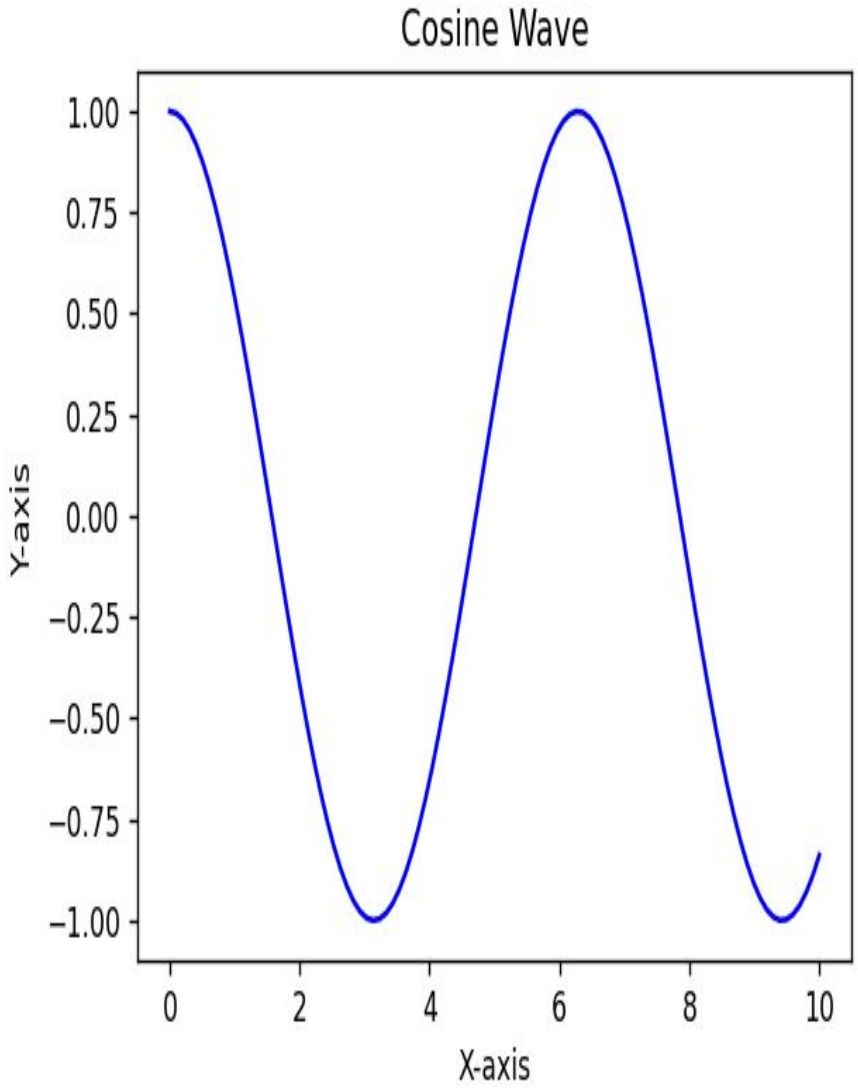
Using plt.subplots():

Syntax:

fig, axes = plt.subplots(nrows, ncols, figsize=(width, height))
Returns a fig (the figure) and axes (array of subplot axes).

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 10, 100)   # 0 to 10
y1 = np.sin(x)                # sine wave
y2 = np.cos(x)                # cosine wave
# Create a figure with 1 row and 2 columns of subplots
fig, axes = plt.subplots(1, 2, figsize=(10, 4))  # 1 row, 2 columns
# First subplot
axes[0].plot(x, y1, color='red')
axes[0].set_title("Sine Wave")   # title for first plot
axes[0].set_xlabel("X-axis")     # x-axis label
axes[0].set_ylabel("Y-axis")     # y-axis label
# Second subplot
axes[1].plot(x, y2, color='blue')
axes[1].set_title("Cosine Wave")  # title for second plot
axes[1].set_xlabel("X-axis")
axes[1].set_ylabel("Y-axis")
plt.tight_layout()
plt.show()
```


Sine Wave
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
2
4
6
8
10
X-axis


Cosine Wave
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Y-axis
0
2
4
6
8
10
X-axis

subplot() vs subplots() – Difference

Feature	`plt.subplot()`	`plt.subplots()`
Usage	Create **one subplot at a time**	Create **all subplots at once**
Return value	Nothing (just sets the current subplot)	Returns `fig` (figure) and `axes` (array of subplots)
Multiple plots	Must call multiple times for multiple plots	Single call creates **grid of subplots**
Ease of access	Harder to manage axes individually	Easy to access and modify each subplot using `axes[row, col]`
Recommended	Small/simple plots	Many subplots / clean organization / looping

Plotly:

Plotly is a Python library used to create interactive graphs. Unlike normal plots in Matplotlib, which are static pictures, Plotly plots allow you to:

1)Hover over points to see values
2)Zoom in or zoom out
3)Pan (move the plot around)
4)Toggle data series on or off by clicking the legend

Why Use Plotly

Interactivity: Great for exploring data.
Beautiful visualizations: Modern, colorful, and professional-looking plots.
Multiple chart types: Line, scatter, bar, pie, histograms, 3D plots, maps, heat maps, etc.
Dashboard friendly: Works with web dashboards using Dash.
Easy customization: You can change colors, sizes, labels, layout, and themes easily.

Ex:

```
import plotly.express as px
# Load example dataset
data = px.data.iris()
# Create an interactive scatter plot
fig = px.scatter(
    data_frame=data,
    x="sepal_width",
    y="sepal_length",
    color="species",       # Color points by species
    size="petal_length",   # Make marker size by petal length
    title="Iris Dataset Scatter Plot")
fig.show()  # Display the interactive plot
```


Iris Dataset Scatter Plot
species
setosa
versicolor
virginica
sepal_length
8
7.5
7
6.5
6
5.5
5
4.5
sepal_width
2
2.5
3
3.5
4
4.5

UNIT-V

EDA CASE STUDIES AND REAL-TIME DATA SETS

A hands-on guide you can follow step by step to explore data (EDA) using three well-known sample datasets. The three data sets are

1. Titanic (classification)
2. Iris (multiclass)
3. Sales (time series / regression)

Titanic (Classification):

The Titanic dataset is a famous dataset used in data science. The Titanic dataset helps us learn how to analyze data, find patterns, and understand which factors (like sex, class, age) affected survival.

The Data set contains rows and columns. Each row considers one passenger and each column contain some information (feature) about the passenger.

Important columns are:

Survived → 1 = survived, 0 = did not survive (target variable)

Pclass → Ticket class (1 = 1st class, 2 = 2nd class, 3 = 3rd class)

Sex → Male or Female

Age → Age of the passenger

Fare → Ticket price

Embarked → Port where passenger got on (C = Cherbourg, Q = Queenstown, S = Southampton)

Why is it used in Data Science

Beginner-Friendly Dataset

- It is small in size (only about 891 rows in training set).
- Easy to understand (real-world story → Titanic sinking).

- Doesn’t need powerful computers to process.

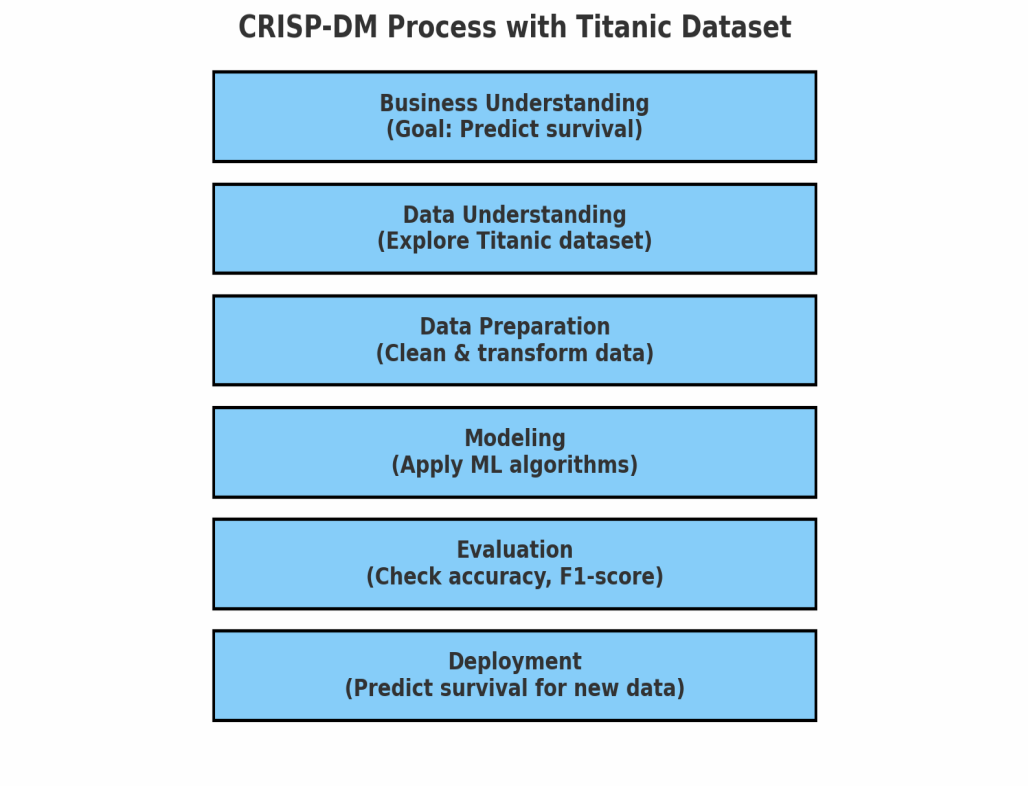
Covers All Important Concepts

Titanic dataset allows students/researchers to practice almost every step in data science:

- **Data cleaning** → Many missing values (Age, Cabin, Embarked).
- **Exploratory Data Analysis (EDA)** → checking survival patterns (e.g., women survived more).
- **Feature Engineering** → Converting text (Sex, Embarked) into numbers.
- **Model Building** → applying classification algorithms.
- **Evaluation** → Measuring accuracy, precision, recall.

How Titanic connects with each stage of Data Science

Here we are using CRISP-DM.CRISP-DM Means Cross-Industry Standard Process for Data Mining. It contains Six Phases.



Business Understanding

- Understand the problem and define the goal.
- Example (Titanic): Predict which passengers survived.

Data Understanding

- Collect and explore data.
- Example: Look at Titanic dataset, survival rates by gender, class, etc.

Data Preparation

- Clean, transform, and organize data.
- Example: Handle missing Age values, convert "Sex" (Male/Female) into numbers.

Modeling

- Apply machine learning algorithms.
- Example: Logistic Regression, Decision Trees, Random Forests.

Evaluation

- Test the model and check if it meets the business goal.
- Example: Accuracy, Precision, Recall, F1-score.

Deployment

- Put the model into real use (or submit predictions on Kaggle).
- Example: Predict survival for new Titanic passenger data.

Ex:

 # Step 1: Import libraries

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score


# Step 2: Load Titanic dataset (Kaggle CSV file)

# Make sure 'train.csv' is in your working directory

data = pd.read_csv("train.csv")

# Step 3: Select useful features

features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]

titanic = data[features + ["Survived"]]

# Step 4: Handle missing values

titanic["Age"].fillna(titanic["Age"].median(), inplace=True)

titanic["Embarked"].fillna(titanic["Embarked"].mode()[0], inplace=True)

# Step 5: Convert categorical → numeric

titanic = pd.get_dummies(titanic, columns=["Sex", "Embarked"], drop_first=True)

# Step 6: Split into train and test sets

X = titanic.drop("Survived", axis=1)

y = titanic["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 7: Train Logistic Regression model

model = LogisticRegression(max_iter=200)
```

```
model.fit(X_train, y_train)
```

 # Step 8: Make predictions

```
y_pred = model.predict(X_test)
```

 # Step 9: Evaluate accuracy

```
acc = accuracy_score(y_test, y_pred)

print("Accuracy of Titanic Logistic Regression Model:", acc)
```

Output:

Accuracy of Titanic Logistic Regression Model: 0.78

Iris (Multi Class):

A famous dataset in machine learning created by Ronald Fisher (1936).

It contains 150 flower samples from 3 species of Iris flowers:

1. Iris-setosa
2. Iris-versicolor
3. Iris-virginica

Each flower has 4 features**:**

- Sepal length
- Sepal width
- Petal length
- Petal width

The target variable is the species of the flower (3 classes).

Why is it Multi-Class Classification?

In Titanic → only 2 classes (Survived / Not Survived) = Binary Classification**.**

In Iris → 3 classes (Setosa, Versicolor, Virginica).
So it is a Multi-Class Classification problem**.**

Steps in Iris Classification

Load the dataset.

Explore features and targets.

Split into training and testing sets.

Train a classification model (Logistic Regression, Decision Tree, Random Forest, SVM, etc.).

Evaluate model performance (accuracy, confusion matrix).

Ex:

 # Step 1: Import libraries

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

 # Step 2: Load Iris dataset

```
iris = load_iris()
X = iris.data   # features: sepal length, sepal width, petal length, petal width
y = iris.target # target: 0 = Setosa, 1 = Versicolor, 2 = Virginica
```

 # Step 3: Split into train & test sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

 # Step 4: Train Logistic Regression model (multi-class)

```
model = LogisticRegression(max_iter=200)

model.fit(X_train, y_train)

# Step 5: Make predictions

y_pred = model.predict(X_test)

# Step 6: Evaluate

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred,
target_names=iris.target_names))
```

Output:

Accuracy: 1.0 (100% in many cases with Logistic Regression)

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11

Sales:

Sales prediction is a common regression problem in data science. The goal is to predict future sales based on past data.

- **Time Series Approach:** We focus on how sales change over time (daily, monthly, yearly).
- **Regression Approach:** We predict sales using other variables (like marketing spend, holidays, product price).

Why it is used in Data Science

Helps businesses plan inventory.

Optimizes marketing strategies.

Forecasts revenue for financial planning.

Detects trends, seasonality, and anomalies.

Methods

Time Series Methods

1. **Moving Average (MA)** – Smooths out fluctuations.
2. **Exponential Smoothing (ES)** – Gives more weight to recent data.
3. **ARIMA / SARIMA** – Advanced models for trend and seasonality.
4. **Prophet (by Facebook)** – Handles holidays and seasonality easily.

Regression Methods

1. **Linear Regression** – Simple if trend is linear.
2. **Polynomial Regression** – Captures non-linear trends.
3. **Multiple Regression** – Uses multiple features (e.g., price + marketing spend).

Ex :

```
import pandas as pd

from statsmodels.tsa.arima.model import ARIMA

import matplotlib.pyplot as plt

# Load sample sales data

data = pd.read_csv("sales.csv", parse_dates=['date'], index_col='date')

sales = data['sales']

# Plot sales

sales.plot(title="Sales Over Time")
```

```
plt.show()
```

```
# Fit ARIMA model
model = ARIMA(sales, order=(1,1,1))  # simple ARIMA
model_fit = model.fit()
# Forecast next 12 months
forecast = model_fit.forecast(steps=12)
print(forecast)
```

Output:

```
2025-01-31    245.21
2025-02-28    250.34
2025-03-31    255.12
2025-04-30    260.01
2025-05-31    265.23
2025-06-30    270.45
2025-07-31    275.67
2025-08-31    280.89
2025-09-30    286.10
2025-10-31    291.32
```

2025-11-30 296.54

2025-12-31 301.76

Freq: M, dtype: float64

Outlier Detection Techniques:

The Outlier Detection Techniques are

1. Statistical Methods
2. Visual Methods

Statistical Methods:

The Statistical Methods are

1. Z-Score Method
2. IQR (Interquartile Range) Method

Visual Methods:

The Visual Methods are

1. Box plot
2. Scatter Plot
3. Histogram

Feature Engineering Techniques in EDA:

Feature Engineering is the process of creating, transforming, or selecting features (columns) in your dataset to improve the performance of machine learning models.

In Exploratory Data Analysis (EDA), feature engineering helps us understand data better and extract meaningful patterns.

Why it is Important

- Raw data may not be suitable for ML models.
- Proper features can boost model accuracy**.**

- Helps in handling missing values, categorical variables, and scaling.

The Common Feature Engineering Techniques are

1. Handling Missing Values
2. Encoding Categorical Variables
3. Feature Transformation
4. Feature Creation
5. Feature Selection

Handling Missing Values:

Fill missing numeric values with mean, median, or mode.

Fill missing categorical values with most frequent category.

Create a new feature indicating if a value was missing (e.g., `is_missing`).

Encoding Categorical Variables:

Label Encoding: Converts categories to numbers (e.g., `Red=0, Blue=1`).

One-Hot Encoding: Creates binary columns for each category.

Target Encoding: Replaces category with the average target value for that category.

Feature Transformation:

Scaling: Min-Max scaling, Standardization to normalize data.

Log Transformation: Reduces skewness in numeric data.

Polynomial Features: Create interaction terms or higher-order features (x^2, x^3).

Feature Creation

Date/Time Features: Extract `day`, `month`, `year`, `weekday`, `hour`.

Aggregated Features: Sum, mean, count over a group (e.g., total sales per customer).

Binning: Convert numeric values into categories (e.g., age groups).

Feature Selection

Correlation Analysis: Remove highly correlated features.

Variance Threshold: Remove features with very low variance.

Model-Based Selection: Use feature importance from tree-based models (Random Forest, XGBoost).

Ex:

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, StandardScaler
# Sample data
data = pd.DataFrame({
    'sales': [100, 200, 150, 300, 250],
    'month': ['Jan', 'Feb', 'Jan', 'Mar', 'Feb']
})
# One-Hot Encoding
encoder = OneHotEncoder(sparse=False)
encoded = encoder.fit_transform(data[['month']])
encoded_df = pd.DataFrame(encoded, columns=encoder.get_feature_names_out(['month']))
```

```
data = pd.concat([data, encoded_df], axis=1)

data.drop('month', axis=1, inplace=True)

# Scaling sales

scaler = StandardScaler()

data['sales_scaled'] = scaler.fit_transform(data[['sales']])

print(data)
```

Output:

	sales	month_Jan	month_Feb	month_Mar	sales_scaled
0	100	1.0	0.0	0.0	-1.264911
1	200	0.0	1.0	0.0	0.000000
2	150	1.0	0.0	0.0	-0.632456
3	300	0.0	0.0	1.0	1.264911
4	250	0.0	1.0	0.0	0.632456

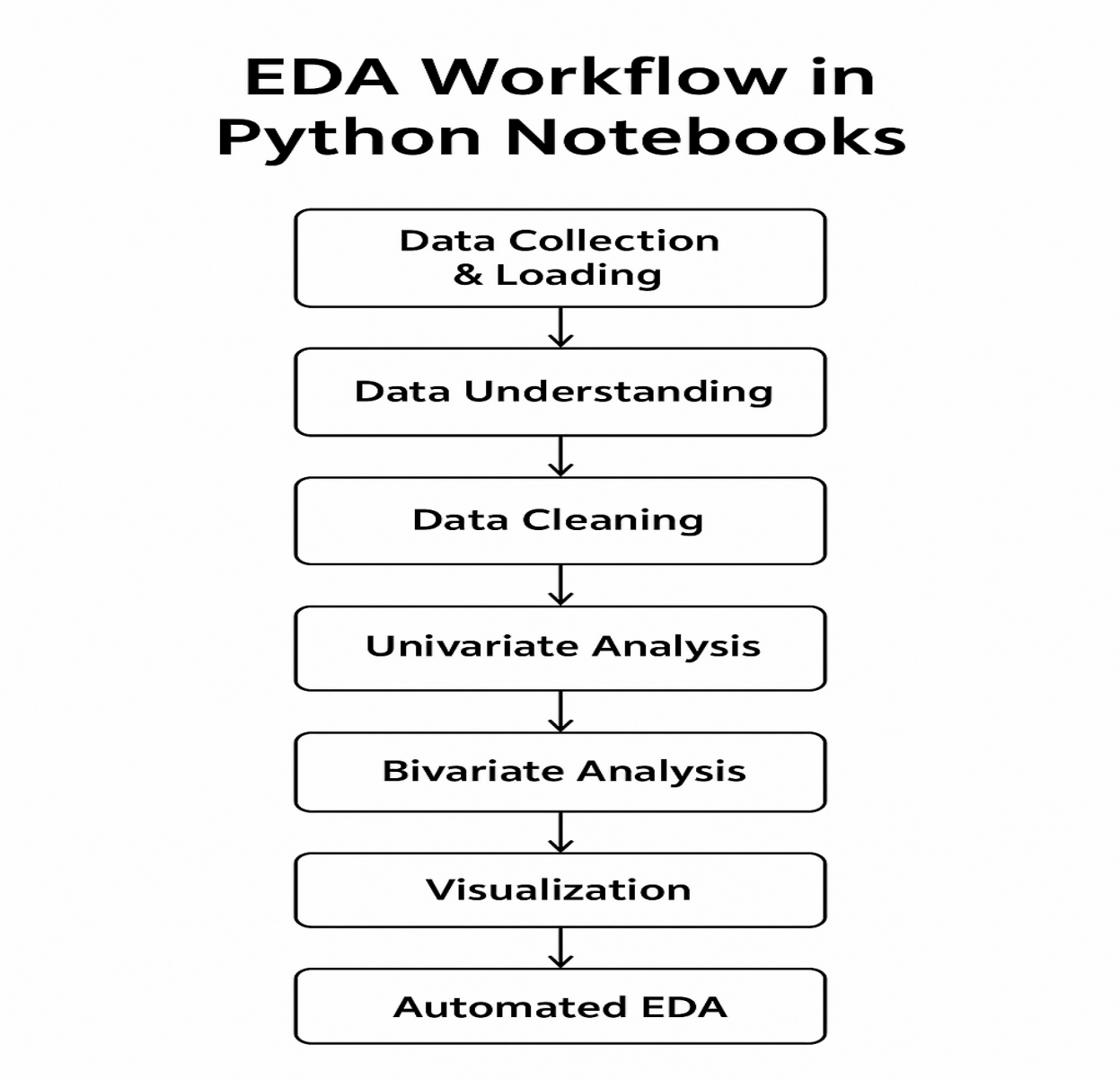
EDA Report Generation using Python Notebooks:

EDA is the process of analyzing and summarizing datasets to understand patterns, detect anomalies, test hypotheses, and check assumptions using statistical and graphical methods.

Purpose:

- Understand data structure and features
- Detect missing or incorrect data
- Identify relationships between variables

Assist in feature selection for machine learning



Steps in EDA

Step 1: Data Collection & Loading

- Collect dataset from CSV, Excel, database, or API.
- Load it into Python using **pandas** (pd.read_csv() or pd.read_excel()).

Step 2: Data Understanding

- Use functions like:
    - data.head() → view first few rows
    - data.info() → check column types, non-null values
    - data.describe() → summary statistics for numeric columns
- Check the **shape** of data (data.shape) to know rows and columns.

Step 3: Data Cleaning

- **Missing values:** Use data.isnull().sum() to find missing values. Fill or remove them.
- **Duplicates:** Use data.duplicated().sum() to detect duplicate rows.
- **Data type correction:** Ensure correct types (numeric, categorical, date).

Step 4: Univariate Analysis

- **Numerical columns:** Check distributions using histograms, boxplots, mean, median, standard deviation.
- **Categorical columns:** Use frequency tables or countplots to see distribution of categories.

Step 5: Bivariate Analysis

- **Numerical vs Numerical:** Correlation analysis (.corr()) and scatter plots.
- **Numerical vs Categorical:** Boxplots, violin plots to compare distributions across categories.
- **Categorical vs Categorical:** Cross-tabulation or stacked bar charts.

Step 6: Visualization

- Use libraries like **matplotlib** and **seaborn** for plotting.
- Visualizations help in identifying patterns, trends, outliers, and relationships.

Step 7: Automated EDA (Optional but Recommended)

- Use tools like:
    - **Sweetviz:** Generates interactive HTML reports with insights on target analysis, feature comparison, and data quality.

- **Pandas Profiling:** Creates comprehensive HTML reports including correlations, missing data, and distributions.

3. Importance of EDA in Data Science

1. **Data Quality Check:** Identify missing, incorrect, or duplicate data.
2. **Pattern Discovery:** Helps in identifying trends, correlations, and anomalies.
3. **Feature Selection:** Determines which features are important for modeling.
4. **Improves Model Performance:** Clean and well-understood data improves predictions.

4. EDA Report Generation in Python Notebooks

- **Why Python Notebooks?**
  - Interactive and allows step-by-step analysis.
  - Combines code, visualization, and narrative explanation.
- **Typical Notebook Workflow:**

1. Import libraries (pandas, numpy, matplotlib, seaborn)
2. Load dataset
3. Perform data understanding and cleaning
4. Visualize univariate and bivariate relationships
5. Generate automated report using Sweetviz or Pandas Profiling
6. Save results for future modeling

Preparing Data for Machine Learning Models

Machine learning models can only learn from **good-quality data**. Raw data is often messy, incomplete, or inconsistent. Preparing it ensures:

- Better model accuracy
- Faster training
- Fewer errors and biases

Steps to Prepare Data

Step 1: Data Collection

- Gather data from sources: CSV, databases, APIs, sensors, web scraping.
- Ensure the data represents the problem you want to solve.

Example: Collect historical sales data to predict future sales.

Step 2: Data Cleaning

- **Handle missing values:**
  - Remove rows/columns with too many missing values
  - Fill missing values using mean, median, or mode
- **Remove duplicates**
- **Correct errors** (like negative ages, wrong dates)

Step 3: Data Transformation

- **Scaling / Normalization:** Convert features to similar ranges so models perform better
  - Standardization: mean = 0, std = 1
  - Min-Max scaling: values between 0 and 1
- **Encoding categorical data:** Convert categories to numbers
  - Label Encoding: A→0, B→1
  - One-Hot Encoding: Creates binary columns for each category

Step 4: Feature Engineering

- Create new features that help models learn better
- Example: Combine `year` and `month` columns into year_month
- Remove irrelevant features that don't add value

Step 5: Splitting Data

- Divide dataset into:
  - **Training set** → used to train the model (usually 70–80%)
  - **Test set** → used to evaluate performance (usually 20–30%)

Step 6: Handling Imbalanced Data (if needed)

- If one class dominates in classification problems, models can be biased.

- Techniques:
    - Oversampling (e.g., SMOTE)
    - Under sampling
    - Class weighting

Step 7: Data Visualization & Exploration

- Visualize distributions, outliers, and relationships between features.
- Helps in making decisions for feature engineering or cleaning.